



รายงานวิจัย

เรื่อง

ระบบแนะนำสถานที่ท่องเที่ยวภายในประเทศไทย โดยใช้การกรองแบบอิงเนื้อหา
(A Content-Based Recommendation System for Tourist Attractions in Thailand)

จัดทำโดย

นางสาวกนกนาฏ มหาวนิช

นางสาวณัฐพรณ แปนทอง

รายงานวิจัยฉบับนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญาวิทยาศาสตรบัณฑิต

ภาควิชาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์ มหาวิทยาลัยรามคำแหง

ปีการศึกษา 2568

รายงานวิจัยฉบับนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญาวิทยาศาสตรบัณฑิต
ภาควิชาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์ มหาวิทยาลัยรามคำแหง

ปีการศึกษา 2568

ชื่อเรื่อง	ระบบแนะนำสถานที่ท่องเที่ยวภายในประเทศไทย โดยใช้การกรองแบบอิงเนื้อหา (A Content-Based Recommendation System for Tourist Attractions in Thailand)
ชื่อผู้เขียน	นางสาว กนกนาฏ มหาวนิช และ นางสาว ณิชพรพรณ แป้นทอง
หลักสูตร	ปริญญาวิทยาศาสตรบัณฑิต คณะวิทยาศาสตร์ มหาวิทยาลัยรามคำแหง
ชื่ออาจารย์ที่ปรึกษา	รศ.ดร.อัมรินทร์พันธุ์ รอดทุกข์

งานวิจัยนี้ได้รับการพิจารณาอนุมัติให้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญาวิทยาศาสตรบัณฑิต คณะวิทยาศาสตร์ มหาวิทยาลัยรามคำแหง

..... อาจารย์ที่ปรึกษา
(รศ.ดร.อัมรินทร์พันธุ์ รอดทุกข์)

กิตติกรรมประกาศ

งานวิจัยครั้งนี้สำเร็จลุล่วงได้ด้วยความอนุเคราะห์และช่วยเหลือสนับสนุนในการทำวิจัยจากทุกฝ่าย คณะผู้จัดทำขอขอบพระคุณ รศ.ดร.อัมรินทร์ รอดทุกข์ อาจารย์ที่ปรึกษาที่กรุณาให้คำปรึกษา ความช่วยเหลือ รวมถึงข้อเสนอแนะสำหรับการแก้ไขข้อบกพร่องต่าง ๆ ในงานวิจัยขึ้นนี้ด้วยความเอาใจใส่อย่างยิ่งตลอดจนจบ

ขอขอบพระคุณคณาจารย์ภาควิชาวิทยาการคอมพิวเตอร์ทุกท่านที่ได้อบรมสั่งสอนวิชาความรู้ และให้ความช่วยเหลือแก่คณะผู้จัดทำตลอดเวลาที่ได้ทำการศึกษาที่สาขาวิชาวิทยาการคอมพิวเตอร์เป็นต้นมา

ขอขอบคุณเพื่อนนักศึกษาของคณะผู้จัดทำที่ช่วยเหลือในเรื่องตัวอย่างระบบเพื่อใช้ในการพัฒนางานวิจัย และพูดคุยปรึกษาในระหว่างการทำงาน

สุดท้ายนี้ ขอขอบคุณครอบครัวของคณะผู้จัดทำที่ได้ให้การสนับสนุนและเป็นกำลังใจสำคัญมาโดยตลอดการศึกษา

หากงานวิจัยฉบับนี้มีข้อบกพร่องประการใด ทางคณะผู้จัดทำต้องขออภัยมา ณ โอกาสนี้และหวังเป็นอย่างยิ่งว่างานวิจัยฉบับนี้จะเป็นประโยชน์ไม่มากนักแก่ผู้ที่สนใจศึกษา

นางสาว กนกนาฏ มหาวนิช

และ

นางสาว ญัฐพรรณ แป้นทอง

บทคัดย่อ

ชื่อเรื่อง	ระบบแนะนำสถานที่ท่องเที่ยวภายในประเทศไทย โดยใช้การกรองแบบอิงเนื้อหา
ผู้เขียน	นางสาวกนกนาฏ มหาวนิช และ นางสาวณัฐพรณ แป้นทอง
ปริญญา	ปริญญาวิทยาศาสตรบัณฑิต คณะวิทยาศาสตร์ มหาวิทยาลัยรามคำแหง
ปีการศึกษา	2568
อาจารย์ที่ปรึกษา	รศ.ดร.อณณนุพันธ์ รอดทุกข์

การส่งเสริมและกระจายการท่องเที่ยวภายในประเทศจำเป็นต้องอาศัยระบบแนะนำที่มีประสิทธิภาพเพื่อช่วยให้นักท่องเที่ยวสามารถเข้าถึงข้อมูลที่ตรงกับความสนใจเฉพาะบุคคล งานวิจัยนี้มีวัตถุประสงค์เพื่อศึกษาและพัฒนาาระบบแนะนำสถานที่ท่องเที่ยวแบบอิงเนื้อหา (Content-Based Recommendation System) จากฐานข้อมูลรายละเอียดแหล่งท่องเที่ยวภายในประเทศไทยของการท่องเที่ยวแห่งประเทศไทย (ททท.) จำนวน 8,848 รายการ โดยเน้นการเปรียบเทียบประสิทธิภาพเชิงพฤติกรรมของการแปลงข้อความภาษาไทยให้อยู่ในรูปเวกเตอร์เชิงตัวเลขระหว่าง 4 อัลกอริทึมหลัก ได้แก่ TF-IDF, Thai2Vec, FastText และ Sentence-BERT ผ่านการวัดระดับความคล้ายคลึงของเนื้อหาด้วยค่าความคล้ายคลึงโคไซน์ (Cosine Similarity) พร้อมทั้งประเมินประสิทธิภาพด้านระยะเวลาในการประมวลผล

ผลลัพธ์จากงานวิจัยนี้สามารถใช้เป็นแนวทางปฏิบัติและกรอบงานวิจัยในการเลือกใช้อัลกอริทึมประมวลผลภาษาธรรมชาติที่เหมาะสมสำหรับการพัฒนาระบบสนับสนุนการค้นหาและแนะนำการท่องเที่ยวในสภาวะการทำงานจริงต่อไป

คำสำคัญ: ระบบแนะนำ, การกรองแบบอิงเนื้อหา, การประมวลผลภาษาธรรมชาติ, เวกเตอร์แทนความหมายข้อความ, ความคล้ายคลึงโคไซน์

Abstract

Title	A Content-Based Recommendation System for Tourist Attractions in Thailand
Author	Kanoknat Mahavanich and Nathaphan Pantong
Degree	Bachelor of Science
Academic Year	2025
Thesis Advisor	Assoc. Prof. Dr. Annupan Rodtook

Promoting and distributing domestic tourism requires effective recommendation systems that assist travelers in discovering attractions tailored to their personalized interests. This research aims to study and develop a content-based recommendation system using a dataset of 8,848 attractions across Thailand collected by the Tourism Authority of Thailand (TAT). The primary focus is to empirically compare the performance of four text representation algorithms: TF-IDF, Thai2Vec, FastText, and Sentence-BERT. The similarity of tourism descriptions is evaluated using Cosine Similarity, along with computational execution time analysis.

The findings of this study provide practical guidelines and a research framework for selecting optimal natural language processing techniques for production-grade tourism recommendation platforms.

Keywords: Recommendation System, Content-Based Filtering, Natural Language Processing, Text Representation, Cosine Similarity

สารบัญ

กิตติกรรมประกาศ	ก
บทคัดย่อ	ข
Abstract	ค
สารบัญ	ง
สารบัญตาราง	ช
สารบัญรูป	ญ
บทที่ 1 บทนำ	1
1.1 ความเป็นมาและความสำคัญของปัญหา	1
1.2 วัตถุประสงค์ของงานวิจัย	2
1.3 ขอบเขตของงานวิจัย	2
1.3.1 ขอบเขตด้านข้อมูล	2
1.3.2 ขอบเขตด้านการประมวลผลและโมเดล	2
1.3.3 ขอบเขตด้านการประเมินผล	2
1.4 ประโยชน์ที่คาดว่าจะได้รับ	3
บทที่ 2 ทฤษฎีและงานวิจัยที่เกี่ยวข้อง	4
2.1 ระบบแนะนำ (Recommendation System)	4
2.2 การกรองแบบอิงเนื้อหา (Content-Based Filtering)	5
2.3 การประมวลผลภาษาธรรมชาติ (Natural Language Processing)	7
2.4 ระบบประมวลผลภาษาไทย (Thai NLP)	9

2.5 วิธีการวัดความคล้ายคลึง	10
2.5.1 Term Frequency-Inverse Document Frequency (TF-IDF)	10
2.5.2 ความคล้ายคลึงโคไซน์ (Cosine Similarity)	11
2.6 อัลกอริทึมในการสร้างแบบจำลองเทคนิคการกรองข้อมูลแบบอิงเนื้อหา.....	12
2.6.1 Word2Vec / Thai2Vec (Word Embedding).....	12
2.6.2 FastText.....	14
2.6.3 Sentence-BERT	15
2.7 งานวิจัยที่เกี่ยวข้อง.....	17
บทที่ 3 วิธีดำเนินการวิจัย	22
3.1 กระบวนการสร้างแบบจำลอง	22
3.2 ข้อมูลที่นำมาใช้วิจัย.....	23
3.3 การสำรวจข้อมูล.....	23
3.4 การเตรียมข้อมูล.....	24
3.4.1 การจัดการข้อมูลที่มีค่าขาดหายไป.....	24
3.4.2 การจัดการข้อมูลที่มีค่าไม่เหมาะสม	25
3.4.3 การทำความสะอาดข้อมูล HTML	25
3.4.4 การจัดการข้อมูลซ้ำ และค่าว่าง	25
3.4.5 การคัดเลือกรหัสตัวแปร	26
3.4.6 การปรับชื่อฟิลด์	27
3.4.7 การสร้างข้อความสำหรับใช้เป็นคุณลักษณะ	27
3.4.8 การตัดคำภาษาไทย.....	28
3.4.9 การกำจัดคำหยุด	28
3.4.10 การเตรียมข้อมูลสำหรับแบบจำลองแต่ละประเภท.....	28
3.5 การออกแบบอัลกอริทึมที่ใช้ในการสร้างคำแนะนำและพารามิเตอร์	29

3.5.1 TF-IDF.....	29
3.5.2 Thai2Vec.....	31
3.5.3 FastText.....	32
3.5.4 Sentence-BERT	33
3.5.5 การคำนวณความคล้ายคลึงและการจัดอันดับ.....	34
3.6 การออกแบบการทดสอบและเปรียบเทียบ	34
3.6.1 การประเมินผลจากชื่อสถานที่ (By Name).....	34
3.6.2 การประเมินผลจากข้อความ (By Text)	35
3.6.3 การเปรียบเทียบผลลัพธ์ของแบบจำลอง	35
3.7 สรุปผลการดำเนินงาน.....	36
บทที่ 4 ผลการดำเนินการวิจัย.....	37
4.1 ผลการสร้างแบบจำลอง และการทดสอบด้วยชื่อสถานที่ (By Name)	37
4.1.1 ผลการทดสอบด้วยชื่อสถานที่ (By Name).....	37
4.1.2 ผลการเปรียบเทียบคะแนนความคล้ายคลึงโคไซน์ของการทดสอบด้วยชื่อสถานที่	55
4.1.3 ผลการเปรียบเทียบระยะเวลาในการสร้างแบบจำลอง.....	57
4.2 ผลการสร้างแบบจำลอง และการทดสอบด้วยข้อความ (By Text)	58
4.2.1 ผลการทดสอบด้วยข้อความ (By Text).....	58
4.2.2 ผลการเปรียบเทียบค่า Cosine ของ By Text	76
4.2.3 ผลการเปรียบเทียบระยะเวลาในการสร้างแบบจำลอง.....	79
บทที่ 5 สรุปผลการวิจัย อภิปรายผล และข้อเสนอแนะ.....	81
5.1 สรุปผลการวิจัย	81
5.1.1 สรุปผลการทดสอบด้วยชื่อสถานที่ (By Name).....	81
5.1.2 สรุปผลการทดสอบด้วยข้อความ (By Text).....	82
5.1.3 สรุปผลการเปรียบเทียบระยะเวลาในการสร้างแบบจำลอง.....	83

5.2 อภิปรายผลการวิจัย	83
5.2.1 การเปรียบเทียบผลการแนะนำของแบบจำลอง	83
5.2.2 การอภิปรายผลค่า Cosine Similarity.....	84
5.2.3 การอภิปรายผลด้านระยะเวลาในการสร้างแบบจำลอง.....	84
5.3 ข้อจำกัดการวิจัย	85
5.4 ข้อเสนอแนะ.....	86
5.4.1 ข้อเสนอแนะสำหรับการพัฒนาระบบในอนาคต.....	86
5.4.2 ข้อเสนอแนะด้านการประมวลผล	86
5.4.3 ข้อเสนอแนะด้านข้อมูลและแบบจำลอง.....	86
บรรณานุกรม.....	88
ภาคผนวก	90
ประวัติผู้เขียน.....	115

สารบัญตาราง

ตารางที่ 1 โครงสร้างชุดข้อมูลแหล่งท่องเที่ยว (attraction.csv).....	24
ตารางที่ 2 ตารางสรุปขั้นตอนการเตรียมข้อมูล.....	26
ตารางที่ 3 ตารางแสดงการปรับชื่อฟิลด์.....	27
ตารางที่ 4 การเตรียมข้อมูลสำหรับแบบจำลองแต่ละประเภท.....	29
ตารางที่ 5 ค่าพารามิเตอร์ในการสร้างแบบจำลองด้วยอัลกอริทึม TF-IDF Vectorizer.....	30
ตารางที่ 6 ค่าพารามิเตอร์สำหรับการฝึกแบบจำลอง Word2Vec กรณีไม่สามารถโหลด Thai2Vec ได้.....	32
ตารางที่ 7 ค่าพารามิเตอร์ในการสร้างแบบจำลองด้วยอัลกอริทึม FastText.....	32
ตารางที่ 8 ค่าพารามิเตอร์ในการสร้างแบบจำลองด้วยอัลกอริทึม Sentence-BERT.....	33
ตารางที่ 9 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “อุทยานแห่งชาติดอยอินทนนท์”	37
ตารางที่ 10 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “เขตห้ามล่าสัตว์ป่าถ้ำผาน้ำทิพย์”	39
ตารางที่ 11 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “เจดีย์พระศรีสุริโยทัย”	41
ตารางที่ 12 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “ตลาดน้ำคลองแห”	43
ตารางที่ 13 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “ถ้ำใหญ่น้ำหนาว”	44
ตารางที่ 14 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “นาเกลือบ้านแหลม”	46
ตารางที่ 15 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “พระธาตุมรุกขนคร”	48
ตารางที่ 16 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “วัดพระแก้ว”	49
ตารางที่ 17 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “ศูนย์ศึกษาธรรมชาติและสัตว์ป่าเขาพระแทว”	51
ตารางที่ 18 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “อุโมงค์ปิยะมิตร”	53
ตารางที่ 19 ค่าเฉลี่ย Top-1 Cosine Similarity จากการค้นหาโดยชื่อสถานที่ (N=10).....	56
ตารางที่ 20 ผลการเปรียบเทียบระยะเวลาในการสร้างแบบจำลองจากการทดสอบด้วยชื่อสถานที่ (N = 10)	57
ตารางที่ 21 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “ย่านเมืองเก่าที่มีอาคารชิโนโปรตุกีส และผสมผสานหลายวัฒนธรรม”	59

ตารางที่ 22 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “นาขั้นบันไดที่เหมาะสมสำหรับเที่ยวในฤดูฝน หรือชมหมอก พร้อมสัมผัสวิถีชีวิตดั้งเดิม”	60
ตารางที่ 23 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “หมู่บ้านเล็กๆ ในแถบภาคเหนือที่เด่นในเรื่องผ้ามัดย้อม สวนผลไม้และอยู่ท่ามกลางธรรมชาติ”	62
ตารางที่ 24 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “ดอยหรือป่าที่เหมาะสมสำหรับการเดินเขา หรือวิ่ง trail สำหรับผู้เริ่มต้น”	64
ตารางที่ 25 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “ดำน้าดูปะการัง ใกล้กรุงเทพ และอาหารทะเลสดอร่อย” ..	66
ตารางที่ 26 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “สถานที่ท่องเที่ยวใกล้กรุงเทพสำหรับคนชอบกิจกรรม adventure”	67
ตารางที่ 27 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “สถานที่ท่องเที่ยวเมืองเก่าที่เป็นที่นิยมในหมู่ชาวจีน”	69
ตารางที่ 28 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “วัดเก่าที่มีประวัติศาสตร์ยาวนานมากกว่า 100 ปี ที่แสดงถึงการผสมผสานวัฒนธรรมต่างชาติ”	71
ตารางที่ 29 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “ชุมชนริมน้ำที่เหมาะสมสำหรับการพักผ่อนและท่องเที่ยว”	73
ตารางที่ 30 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “สถานที่ท่องเที่ยวที่คล้ายกับอุทยานผาแต้ม หรือเขาปลาร้า มีภาพเขียนก่อนยุคประวัติศาสตร์”	74
ตารางที่ 31 ค่าเฉลี่ย Top-1 Cosine Similarity จากการค้นหาโดยข้อความ	77
ตารางที่ 32 ผลการเปรียบเทียบระยะเวลาในการสร้างแบบจำลอง และทดสอบด้วยข้อความ (N = 10).....	79

สารบัญรูป

รูปที่ 1 เทคนิคการกรองแบบอิงเนื้อหา	5
รูปที่ 2 ตัวอย่างการแนะนำสถานที่ท่องเที่ยวภายในประเทศไทย	6
รูปที่ 3 การสร้าง Place Vector	6
รูปที่ 4 Item Vector รายการสถานที่ท่องเที่ยว.....	7
รูปที่ 5 การคูณเวกเตอร์ 2 มิติ และผลลัพธ์การคูณ	7
รูปที่ 6 หลักการทำงานของระบบประมวลผลภาษาธรรมชาติ	8
รูปที่ 7 การหาความคล้ายคลึงของโคไซน์.....	12
รูปที่ 8 สถาปัตยกรรม CBOW (Continuous Bag-of-Words) ทำนายคำปัจจุบันจากคำที่อยู่ในบริบทโดยรอบ ขณะที่สถาปัตยกรรม Skip-gram ทำนายคำที่อยู่โดยรอบจากคำปัจจุบัน	13
รูปที่ 9 กระบวนการสร้างเวกเตอร์เอกสารด้วย Thai2Vec	14
รูปที่ 10 การแทนคำด้วย Character n-grams ของ FastText กรณีคำว่า "เขาใหญ่"	15
รูปที่ 11 เปรียบเทียบสถาปัตยกรรม Non-Siamese (cross-encoder) (ซ้าย) และสถาปัตยกรรม Siamese (bi- encoder) (ขวา) ในกระบวนการดึงข้อมูล embeddings ระดับประโยค	16
รูปที่ 12 สถาปัตยกรรมของ Sentence-BERT รูปแบบฟังก์ชันเป้าหมายแบบถดถอย (Regression Objective) สำหรับการวัดความคล้ายคลึงเชิงโคไซน์	17
รูปที่ 13 กรอบแนวคิดและกระบวนการสร้างระบบแนะนำสถานที่ท่องเที่ยวด้วยเทคนิค Content-Based Filtering.....	23
รูปที่ 14 กระบวนการทำงานของระบบ และขั้นตอนการเปรียบเทียบประสิทธิภาพแบบจำลอง	35
รูปที่ 15 แผนภูมิแสดงค่าความคล้ายคลึงโคไซน์จากการหาคำค้นหาด้วยชื่อสถานที่	55
รูปที่ 16 แผนภูมิแสดงค่าความคล้ายคลึงโคไซน์จากการหาคำค้นหาด้วยข้อความ	77

บทที่ 1

บทนำ

1.1 ความเป็นมาและความสำคัญของปัญหา

อุตสาหกรรมท่องเที่ยวของประเทศไทยมีการเติบโตอย่างต่อเนื่องและเป็นหนึ่งในอุตสาหกรรมหลักที่ช่วยขับเคลื่อนการพัฒนาเศรษฐกิจและสังคมของประเทศ อย่างไรก็ตาม การท่องเที่ยวไทยยังคงประสบปัญหาสำคัญหลายประการ (กระทรวงการท่องเที่ยวและกีฬา, 2560) ดังนี้

- 1) นักท่องเที่ยวจะจุกตัวในแหล่งท่องเที่ยวยอดนิยม ขณะที่แหล่งท่องเที่ยวอื่นมีผู้เข้าชมน้อย
- 2) รายได้จากการท่องเที่ยวยังกระจายตัวไม่ทั่วถึง และส่วนใหญ่รายได้กระจุกอยู่ในพื้นที่ที่ได้รับความนิยม
- 3) ข้อมูลแหล่งท่องเที่ยวมีจำนวนมาก และหลากหลาย ทำให้ค้นหาสถานที่ที่ตรงกับความต้องการได้ยาก
- 4) นักท่องเที่ยวยังต้องค้นหา และเปรียบเทียบข้อมูลด้วยตนเอง จึงอาจพลาดสถานที่ที่เหมาะสมกับความต้องการ

จากปัญหาข้างต้น เพื่อตอบรับกับแนวทางการพัฒนาอุตสาหกรรมท่องเที่ยวไทยยุคใหม่ การนำเทคโนโลยีสารสนเทศเข้ามาช่วยบริหารจัดการข้อมูลและส่งเสริมการท่องเที่ยวจึงมีความสำคัญอย่างยิ่ง

ระบบแนะนำ (Recommendation System) ถือเป็นเครื่องมือที่มีประสิทธิภาพในการแก้ปัญหาดังกล่าว โดยทำหน้าที่วิเคราะห์ข้อมูลพฤติกรรมและเนื้อหาเพื่อนำเสนอสถานที่ท่องเที่ยวที่ตรงกับความสนใจเฉพาะบุคคล (Personalized Recommendation) (Fayelyn, 2568) ช่วยยกระดับประสบการณ์ของนักท่องเที่ยวในการค้นหาข้อมูล และกระจายรายได้สู่ผู้ประกอบการท้องถิ่นและเศรษฐกิจในภาพรวม ในการพัฒนาระบบแนะนำสำหรับการท่องเที่ยวไทย การท่องเที่ยวแห่งประเทศไทย (ททท.) ได้จัดทำและรวบรวมชุดข้อมูลรายละเอียดแหล่งท่องเที่ยวทั่วประเทศไว้ในฐานข้อมูลมากกว่า 8,800 รายการ อย่างไรก็ตาม ด้วยปริมาณข้อมูลมหาศาลที่เป็นข้อมูล Unstructured Text Data และมีการเผยแพร่ในรูปแบบสองภาษา ได้แก่ ไทยและอังกฤษ จึงนำมาซึ่งความท้าทายในการประมวลผลภาษาธรรมชาติ (NLP) โดยเฉพาะภาษาไทย ซึ่งเป็นภาษาเขียนต่อเนื่องโดยไม่มีการเว้นวรรคระหว่างคำ ต่างจากภาษาอังกฤษที่มีโครงสร้างการแบ่งคำชัดเจน ทำให้การจัดการและสกัดคุณลักษณะของเนื้อหาเพื่อนำมาคำนวณความคล้ายคลึงมีความซับซ้อนที่แตกต่างกัน

ด้วยเหตุนี้ งานวิจัยนี้จึงมุ่งเน้นการพัฒนาระบบแนะนำแบบอิงเนื้อหา (Content-Based Recommendation System) บนชุดข้อมูลแหล่งท่องเที่ยว ททท. โดยออกแบบเป็นการพัฒนารอบงานวิจัย เพื่อศึกษากระบวนการประมวลผลภาษาธรรมชาติ เพื่อแปลงข้อความ Unstructured Text ให้อยู่ในรูปของเวกเตอร์เชิงตัวเลข ก่อนนำไปคำนวณหาความคล้ายคลึงด้วย Cosine Similarity โดยเทคนิคการสร้างเวกเตอร์ข้อความที่

จะศึกษาและเปรียบเทียบประสิทธิภาพของอัลกอริทึมในการสร้างระบบแนะนำแบบอิงเนื้อหาในงานวิจัยนี้ได้แก่ TF-IDF, Thai2Vec, FastText และ Sentence-BERT เพื่อวิเคราะห์ว่าอัลกอริทึมใดสามารถสะท้อนความคล้ายคลึงของรายละเอียดภาษาไทยของแหล่งท่องเที่ยวไทยได้แม่นยำและเหมาะสมที่สุด

1.2 วัตถุประสงค์ของงานวิจัย

- 1) เพื่อศึกษาและพัฒนาระบบแนะนำแหล่งท่องเที่ยวแบบอิงเนื้อหา (Content-Based Recommendation) จากฐานข้อมูลรายละเอียดแหล่งท่องเที่ยวภายในประเทศไทยของ การท่องเที่ยวแห่งประเทศไทย (ททท.)
- 2) เพื่อศึกษากระบวนการ Tokenization หรือการตัดคำ และการแปลงข้อความเป็นเวกเตอร์เชิงตัวเลขของแต่ละอัลกอริทึม
- 3) เพื่อเปรียบเทียบผลลัพธ์และประสิทธิภาพของโมเดลระบบแนะนำข้อมูลที่เป็นภาษาไทย โดยวิเคราะห์ผ่านค่าความคล้ายคลึง Cosine Similarity

1.3 ขอบเขตของงานวิจัย

1.3.1 ขอบเขตด้านข้อมูล

ใช้ข้อมูลรายละเอียดแหล่งท่องเที่ยวภายในประเทศไทยที่รวบรวมโดย ททท. สำนักงานในประเทศไทย รวมทั้งสิ้น 8,848 รายการ โดยเป็นชุดข้อมูลที่มีรายละเอียดเป็นภาษาไทยจำนวน 7,480 รายการ

1.3.2 ขอบเขตด้านการประมวลผลและโมเดล

- 1) ระบบแนะนำ ใช้เทคนิคการแนะนำแบบอิงเนื้อหา (Content-Based Recommendation System)
- 2) การประมวลผลข้อมูลและสร้างเวกเตอร์ เปรียบเทียบการแปลงข้อความรายละเอียดแหล่งท่องเที่ยวให้อยู่ในรูปเวกเตอร์เชิงตัวเลขด้วย 4 อัลกอริทึม ได้แก่ TF-IDF, Thai2Vec, FastText และ Sentence-BERT

1.3.3 ขอบเขตด้านการประเมินผล

ประเมินและเปรียบเทียบผลลัพธ์ความคล้ายคลึงของอาหาระหว่างทั้ง 4 อัลกอริทึม โดยใช้ค่า Cosine Similarity เป็นตัววัดผล เน้นการวิเคราะห์และเปรียบเทียบในลักษณะ Research Framework

1.4 ประโยชน์ที่คาดว่าจะได้รับ

- 1) ได้องค์ความรู้เชิงเปรียบเทียบด้านการประมวลผลภาษาธรรมชาติ ทำให้เข้าใจถึงพฤติกรรมและความแตกต่างการสกัดคุณลักษณะข้อมูลภาษาไทยบนชุดข้อมูลแหล่งท่องเที่ยวจริง
- 2) ได้ระบบต้นแบบการแนะนำแหล่งท่องเที่ยวแบบอิงเนื้อหา (Content-Based Recommendation) มีฟังก์ชันการแนะนำแหล่งท่องเที่ยวที่เหมาะสมกับลักษณะโครงสร้างข้อมูลภาษาไทยของ ททท. ซึ่งสามารถนำไปประยุกต์ใช้ในการกระจายการแนะนำไปยังแหล่งท่องเที่ยวเมืองรองอื่น ๆ ได้ต่อไป
- 3) ส่งเสริมการเข้าถึงข้อมูลแหล่งท่องเที่ยว เพิ่มประสิทธิภาพในการจับคู่ความสนใจของนักท่องเที่ยวกับเนื้อหาแหล่งท่องเที่ยวหรือสถานที่ท่องเที่ยวทางเลือกที่ตรงกับบริบทความสนใจของผู้ใช้ได้อย่างแม่นยำ

บทที่ 2

ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

การวิจัยครั้งนี้ทางผู้วิจัยได้ทำการศึกษาบทความทฤษฎีและงานวิจัยที่เกี่ยวข้องกับการพัฒนาระบบแนะนำสถานที่ท่องเที่ยวภายในประเทศไทย (Recommendation System) โดยใช้เทคนิคการกรองแบบอิงเนื้อหา (Content-Based Filtering) จึงได้นำเสนอหัวข้อ ดังนี้

1. ระบบแนะนำสถานที่ท่องเที่ยวภายในประเทศไทย (Recommendation System)
2. การกรองแบบอิงเนื้อหา (Content-Based Filtering)
3. การประมวลผลภาษาธรรมชาติ (Natural Language Processing)
4. ระบบประมวลผลภาษาไทย (Thai NLP)
5. วิธีการวัดค่าความคล้ายคลึง
6. อัลกอริทึมในการสร้างแบบจำลองทางเทคนิคการกรองข้อมูลแบบอิงเนื้อหา
7. งานวิจัยที่เกี่ยวข้อง

2.1 ระบบแนะนำ (Recommendation System)

ระบบแนะนำ (Recommendation System) เป็นระบบที่พัฒนาขึ้นเพื่อช่วยแนะนำข้อมูล สินค้า บริการ หรือสถานที่ที่มีความเหมาะสมกับความสนใจของผู้ใช้งาน โดยระบบสามารถนำข้อมูลที่เกี่ยวข้องกับผู้ใช้งานและข้อมูลของรายการที่ต้องการแนะนำมาวิเคราะห์ เพื่อคัดเลือกและจัดลำดับรายการที่มีความเหมาะสมสำหรับผู้ใช้งานแต่ละราย ระบบแนะนำมีบทบาทในการช่วยลดระยะเวลาในการค้นหาข้อมูล และช่วยให้ผู้ใช้งานสามารถเข้าถึงข้อมูลที่ตรงกับความสนใจได้สะดวกมากยิ่งขึ้น

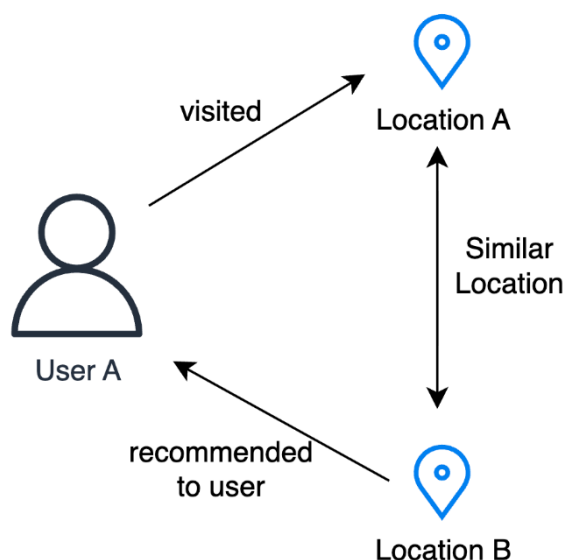
สำหรับระบบแนะนำสถานที่ท่องเที่ยว ระบบสามารถนำข้อมูลของสถานที่ท่องเที่ยว เช่น ชื่อสถานที่ ประเภทสถานที่ และรายละเอียดของสถานที่ มาใช้ในการวิเคราะห์เพื่อสร้างคำแนะนำให้แก่ผู้ใช้งาน โดยสามารถประยุกต์ใช้เทคนิคการประมวลผลภาษาธรรมชาติ (Natural Language Processing: NLP) เพื่อวิเคราะห์ข้อมูลที่อยู่ในรูปแบบข้อความ และสร้างตัวแทนของข้อมูลให้อยู่ในรูปแบบที่สามารถนำไปประมวลผลและเปรียบเทียบได้

การสร้างระบบแนะนำสามารถประยุกต์ใช้แนวทางได้หลายรูปแบบ โดยหนึ่งในแนวทางที่เหมาะสมกับข้อมูลสถานที่ท่องเที่ยวที่อยู่ในรูปแบบข้อความ คือ การกรองแบบอิงเนื้อหา (Content-Based Filtering) ซึ่ง

พิจารณาความคล้ายคลึงของเนื้อหาและคุณลักษณะของสถานที่ท่องเที่ยวที่ผู้ใช้งานสนใจ แล้วนำไปเปรียบเทียบกับสถานที่ท่องเที่ยวอื่นในระบบ เพื่อคัดเลือกสถานที่ที่มีลักษณะใกล้เคียงและนำมาแนะนำให้แก่ผู้ใช้งาน

2.2 การกรองแบบอิงเนื้อหา (Content-Based Filtering)

เทคนิคการกรองแบบอิงเนื้อหา คือการแนะนำสถานที่ หรือบริการที่ได้มาจากการพิจารณาความคล้ายคลึงโดยจะดูจากลักษณะของสถานที่ที่จะแนะนำ และทำการแนะนำสถานที่ที่มีลักษณะใกล้เคียงที่ผู้ใช้เคยไปหรือชื่นชอบ จากรูปที่ 1 ระบบจะทำการแนะนำสถานที่ที่มีความคล้ายคลึงกับสถานที่ที่ผู้ใช้งานเคยไปมาก่อนหน้า ซึ่งข้อดีของเทคนิคการกรองแบบอิงเนื้อหา นั้น คือไม่ต้องใช้ข้อมูลจำนวนมากในการประมวลผลเพื่อแนะนำสถานที่ และการแนะนำสถานที่ใหม่ที่ยังไม่มีข้อมูลสามารถทำได้เนื่องจากจะพิจารณาจากความคล้ายคลึงกับลักษณะสถานที่เดิม ส่วนข้อเสียของเทคนิคการกรองแบบอิงเนื้อหาจะไม่สามารถแนะนำสถานที่ที่มีความแตกต่างจากสถานที่ที่ผู้ใช้งานเคยท่องเที่ยวหรือใช้บริการ ส่งผลให้ผู้ใช้งานไม่ได้รับการแนะนำสถานที่ที่มีความหลากหลาย และจำกัดขอบเขตของการพบสถานที่ใหม่ ๆ



รูปที่ 1 เทคนิคการกรองแบบอิงเนื้อหา

จากรูปที่ 2 ตัวอย่างการแนะนำสถานที่ท่องเที่ยวภายในประเทศไทย โดยใช้ ดอยอินทนนท์ ในกรณีนี้ ดอยอินทนนท์ มีลักษณะเป็นแหล่งท่องเที่ยวทางธรรมชาติ ไม่มีความเกี่ยวข้องกับแหล่งท่องเที่ยวเชิงวัฒนธรรม และไม่ใช่วัดหรือแหล่งท่องเที่ยวสำหรับเด็ก



ดอยอินทนนท์

แหล่งท่องเที่ยวทาง ธรรมชาติ	สูง (9)
แหล่งท่องเที่ยวเชิง วัฒนธรรม	ไม่มีข้อมูล (0)
แหล่งท่องเที่ยวสำหรับ เด็ก	ต่ำ (-6)

รูปที่ 2 ตัวอย่างการแนะนำสถานที่ท่องเที่ยวภายในประเทศไทย

ดังนั้นก็จะสามารถสร้างเวกเตอร์สำหรับ ดอยอินทนนท์ ตามลักษณะของสถานที่ทั้ง 3 แห่ง ดังรูปที่ 3 ในระดับคะแนน -10 ถึง 10 เนื่องจาก ดอยอินทนนท์เป็นแหล่งท่องเที่ยวทางธรรมชาติ จึงกำหนดค่า 9 ให้กับแหล่งท่องเที่ยวทางธรรมชาติ และ ดอยอินทนนท์ไม่มีความเกี่ยวข้องกับ แหล่งท่องเที่ยวเชิงวัฒนธรรม จึงกำหนดค่า 0 ให้กับแหล่งท่องเที่ยวเชิงวัฒนธรรม และเนื่องจาก ดอยอินทนนท์ไม่ใช่แหล่งท่องเที่ยวสำหรับเด็ก แหล่งท่องเที่ยวสำหรับเด็กจึงกำหนดเป็น -6 ให้กับแหล่งท่องเที่ยวสำหรับเด็ก

สร้าง Vector ของ ดอยอินทนนท์

แหล่งท่องเที่ยวทาง ธรรมชาติ	แหล่งท่องเที่ยวเชิง วัฒนธรรม	แหล่งท่องเที่ยว สำหรับเด็ก
9	0	-6

Place Vector = (9, 0, -6)

รูปที่ 3 การสร้าง Place Vector

ดังนั้นเวกเตอร์ของ ดอยอินทนนท์ จึงมีค่าเท่ากับ (9, 0, -6) ตามลำดับ (แหล่งท่องเที่ยวทางธรรมชาติ, แหล่งท่องเที่ยวเชิงวัฒนธรรม, แหล่งท่องเที่ยวสำหรับเด็ก) และเวกเตอร์สำหรับรายการสถานที่ท่องเที่ยวคือ อุทยานประวัติศาสตร์พระนครศรีอยุธยา (0, 1, 0) สยามอะเมซิงปาร์ค (0, 0, 1) ตลาดน้ำอัมพวา (0, 1, 0) และ เกาะสมุย (1, 0, 0) ตามลำดับ ดังรูปที่ 4

Vector รายการสถานที่ท่องเที่ยว

สถานที่ท่องเที่ยว (ตัวอย่าง)	แหล่งท่องเที่ยวทาง ธรรมชาติ	แหล่งท่องเที่ยวเชิง วัฒนธรรม	แหล่งท่องเที่ยว สำหรับเด็ก	Item Vector
อุทยานประวัติศาสตร์ พระนครศรีอยุธยา	0	1	0	(0, 1, 0)
สยามอะเมซิ่งปาร์ค	0	0	1	(0, 0, 1)
ตลาดน้ำอัมพวา	0	1	0	(0, 1, 0)
เกาะสมุย	1	0	0	(1, 0, 0)

รูปที่ 4 Item Vector รายการสถานที่ท่องเที่ยว

จากรูปที่ 5 ได้ทำการคูณเวกเตอร์ 2 มิติสองตัว คือ เวกเตอร์ของสถานที่ และ เวกเตอร์ของรายการสถานที่ ดังนั้นการคูณเวกเตอร์ของ อุทยานประวัติศาสตร์พระนครศรีอยุธยา คือ 0 สยามอะเมซิ่งปาร์ค คือ -6 ตลาดน้ำอัมพวา คือ 0 และ เกาะสมุย คือ 9 จึงได้แนะนำ เกาะสมุย ซึ่งคล้ายกับ ดอยอินทนนท์ ที่เป็นแหล่งท่องเที่ยวทางธรรมชาติ และไม่ใช่แหล่งท่องเที่ยวสำหรับเด็ก ในลักษณะที่คล้ายกันนั้นสามารถแนะนำคำนวณการคูณเวกเตอร์ของเวกเตอร์รายการทั้งหมดของสถานที่ท่องเที่ยวภายในประเทศไทย และแนะนำสถานที่ท่องเที่ยว 10 อันดับแรกให้กับ ดอยอินทนนท์ ได้

คำนวณความสอดคล้อง (Dot Product)		Place Vector	Item Vector
		$(9, 0, -6)$	(i_1, i_2, i_3)
		$= (9 \cdot i_1, 0 \cdot i_2, -6 \cdot i_3)$	

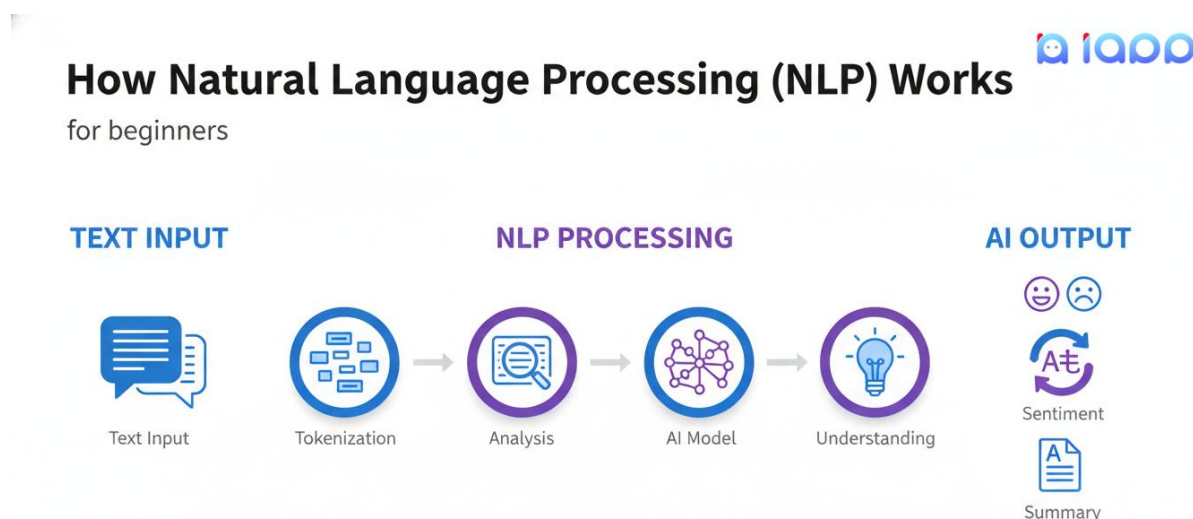
สถานที่ท่องเที่ยว	Item Vector	คำนวณ	Score
อุทยานประวัติศาสตร์ พระนครศรีอยุธยา	(0, 1, 0)	$(9 \cdot 0) + (0 \cdot 1) + (-6 \cdot 0)$	0
สยามอะเมซิ่งปาร์ค	(0, 0, 1)	$(9 \cdot 0) + (0 \cdot 0) + (-6 \cdot 1)$	-6
ตลาดน้ำอัมพวา	(0, 1, 0)	$(9 \cdot 0) + (0 \cdot 1) + (-6 \cdot 0)$	0
เกาะสมุย	(1, 0, 0)	$(9 \cdot 1) + (0 \cdot 0) + (-6 \cdot 0)$	9

ผลการแนะนำ Top-N Recommendation		
Rank	สถานที่ท่องเที่ยว	Score
1	เกาะสมุย	9
2	อุทยานประวัติศาสตร์พระนครศรีอยุธยา	0
3	ตลาดน้ำอัมพวา	0
4	สยามอะเมซิ่งปาร์ค	-6

รูปที่ 5 การคูณเวกเตอร์ 2 มิติ และผลลัพธ์การคูณ

2.3 การประมวลผลภาษาธรรมชาติ (Natural Language Processing)

Natural Language Processing (NLP) หรือการประมวลผลภาษาธรรมชาติ เป็นสาขาหนึ่งของปัญญาประดิษฐ์ที่ช่วยให้คอมพิวเตอร์เข้าใจ ตีความ และสร้างภาษามนุษย์ได้ มันเป็นสะพานเชื่อมระหว่างการสื่อสารของมนุษย์และความเข้าใจของคอมพิวเตอร์ (ดร.กอบกฤตย์ วิริยะยุทธกร, 2568)



รูปที่ 6 หลักการทำงานของระบบประมวลผลภาษาธรรมชาติ

(ดร.กอบกฤตย์ วิริยะยุทธกร, 2568)

NLP จะทำงานโดยการแปลงข้อความหรือประโยคในภาษาที่มนุษย์ใช้สื่อสารกันให้เป็นส่วนย่อย ๆ หรือชิ้นเล็ก ๆ ที่ทำให้คอมพิวเตอร์เข้าใจง่ายขึ้น ซึ่งสุดท้ายแล้วส่วนย่อย ๆ เหล่านี้จะอยู่ในรูปแบบของตัวเลข (หรือถ้าละเอียดลงไปอีกหน่อยก็คือจะอยู่ในรูปแบบของเวกเตอร์) จากนั้นจึงวิเคราะห์แต่ละชิ้นส่วนว่าการรวมตัวกันขององค์ประกอบนี้มีความสัมพันธ์กันในรูปแบบใด เช่น คำว่า “บริษัท” กับ “แบ็คยาร์ด” และคำว่า “แบ็คยาร์ด” กับ “จำกัด” นั้นมีการปรากฏลำดับก่อนหลังอย่างไร เป็นต้น โดยใช้อัลกอริทึมหรือกระบวนการทางคณิตศาสตร์หรือกระบวนการทางภาษาต่าง ๆ เข้ามาช่วย สุดท้ายเราจะได้รูปแบบที่เป็นตัวแทนของความสัมพันธ์ทั้งหมด หรือที่เรียกว่า “แบบจำลอง” (Model) นั้นเอง

ตัวอย่างการใช้งานของ NLP พบได้ทั้งในชีวิตประจำวัน เช่น ระบบแปลภาษาอย่าง Google Translate หรือ DeepL ผู้ช่วยเสียงอัจฉริยะอย่าง Siri, Alexa, และ Google Assistant โปรแกรมตรวจไวยากรณ์อย่าง

Grammarly รวมถึงแชทบอทตอบคำถามลูกค้า นอกจากนี้ยังใช้ในงานสรุปบทความหรือเอกสารอัตโนมัติ ช่วยให้ผู้ใช้งานเข้าใจข้อมูลได้รวดเร็วและมีประสิทธิภาพยิ่งขึ้น

NLP เป็นศาสตร์ที่คอมพิวเตอร์ใช้ประมวลผลภาษามนุษย์ ส่วน LLM (Large Language Model) เช่น GPT-4, Claude หรือ PaLM คือโมเดล AI ขนาดใหญ่ที่ถูกฝึกจากข้อมูลภาษาจำนวนมาก เพื่อให้ทำงานด้าน NLP ได้อย่างหลากหลาย เช่น เขียน สรุป แปล หรือสนทนาได้อย่างเป็นธรรมชาติ (บริษัท แบ็คยาร์ด จำกัด, 2568)

2.4 ระบบประมวลผลภาษาไทย (Thai NLP)

ภาษาไทยมีลักษณะทางโครงสร้างที่แตกต่างจากภาษาอังกฤษและภาษาอื่นในหลายด้าน โดยเฉพาะลักษณะการเขียนที่ไม่มีช่องว่างระหว่างคำ ทำให้การระบุขอบเขตของคำในข้อความภาษาไทยมีความซับซ้อน นอกจากนี้ ภาษาไทยยังประกอบด้วยพยัญชนะ สระ และวรรณยุกต์ที่สามารถประกอบกันในรูปแบบต่างๆ และคำบางคำอาจมีความหมายแตกต่างกันตามบริบทที่ปรากฏในประโยค ดังนั้น การนำข้อมูลภาษาไทยไปประมวลผลด้วยคอมพิวเตอร์จึงจำเป็นต้องมีกระบวนการที่เหมาะสมกับลักษณะเฉพาะของภาษาไทย (กอบกฤตย์ วิริยะยุทธกร, 2568)

การประมวลผลภาษาไทยเป็นกระบวนการหนึ่งของการประมวลผลภาษาธรรมชาติ (Natural Language Processing: NLP) ที่มุ่งเน้นการวิเคราะห์และประมวลผลข้อความภาษาไทยให้อยู่ในรูปแบบที่คอมพิวเตอร์สามารถนำไปใช้งานได้ โดยขั้นตอนสำคัญคือ การตัดคำ (Tokenization) ซึ่งเป็นการแบ่งข้อความออกเป็นหน่วยคำ เนื่องจากภาษาไทยไม่มีช่องว่างระหว่างคำอย่างชัดเจน เช่น ข้อความ “อุทยานแห่งชาติดอยอินทนนท์” จำเป็นต้องวิเคราะห์และแบ่งออกเป็นหน่วยคำก่อนที่จะนำไปประมวลผลในขั้นตอนต่อไป

หลังจากการตัดคำ ข้อมูลสามารถนำไปผ่านกระบวนการเตรียมข้อมูล (Text Preprocessing) เช่น การกำจัดอักขระหรือข้อมูลที่ซ้ำซ้อน การจัดรูปแบบข้อความให้เป็นมาตรฐาน และการจัดการข้อมูลที่ไม่เหมาะสม จากนั้นจึงนำข้อความที่ผ่านการเตรียมข้อมูลไปสร้างเป็นตัวแทนข้อมูลในรูปแบบตัวเลขหรือเวกเตอร์ เพื่อให้สามารถนำไปวิเคราะห์และเปรียบเทียบกับแบบจำลองทางคณิตศาสตร์ได้

การสร้างตัวแทนข้อความภาษาไทยสามารถทำได้หลายวิธี เช่น TF-IDF, Word2Vec, FastText และ BERT โดย TF-IDF เป็นวิธีที่พิจารณาความสำคัญของคำภายในเอกสารและชุดเอกสาร ส่วน Word2Vec และ FastText เป็นวิธีการสร้างตัวแทนคำในรูปแบบเวกเตอร์จากความสัมพันธ์ของคำและบริบท ขณะที่ BERT เป็นแบบจำลองที่สามารถสร้างตัวแทนคำโดยพิจารณาบริบทของคำที่ปรากฏในข้อความ

สำหรับงานวิจัยด้านการแนะนำสถานที่ท่องเที่ยว การประมวลผลภาษาไทยมีความสำคัญเนื่องจากข้อมูลสถานที่ท่องเที่ยวส่วนใหญ่อยู่ในรูปแบบข้อความภาษาไทย เช่น ชื่อสถานที่ ประเภทสถานที่ และรายละเอียด

สถานที่ท่องเที่ยว ข้อมูลดังกล่าวจำเป็นต้องผ่านกระบวนการประมวลผลก่อนนำไปสร้างตัวแทนข้อความ เพื่อให้สามารถนำข้อมูลมาเปรียบเทียบความคล้ายคลึงระหว่างสถานที่ท่องเที่ยวได้

ในงานวิจัยนี้ ข้อมูลข้อความของสถานที่ท่องเที่ยวจะถูกนำมาผ่านกระบวนการเตรียมข้อมูลและการตัดคำภาษาไทย จากนั้นนำข้อมูลที่ได้นำไปสร้างตัวแทนข้อความด้วยวิธี TF-IDF, Thai2Vec, FastText และ BERT แล้วนำเวกเตอร์ที่ได้มาคำนวณค่าความคล้ายคลึงด้วย Cosine Similarity เพื่อจัดอันดับสถานที่ท่องเที่ยวที่มีเนื้อหาใกล้เคียงกับสถานที่ที่ผู้ใช้งานสนใจ

2.5 วิธีการวัดความคล้ายคลึง

2.5.1 Term Frequency-Inverse Document Frequency (TF-IDF)

Term Frequency-Inverse Document Frequency (TF-IDF) คือ วิธีการบ่งบอกน้ำหนักของคำเฉพาะใช้เพื่อเปรียบเทียบความเหมือนกันของคำสองคำ โดยวัดจากค่าของ Term Frequency และ Inverse Document Frequency ดังสมการ (3)

Term Frequency คือ จำนวนครั้งที่แต่ละคำที่ปรากฏขึ้นในเอกสาร นำมาหารด้วยจำนวนคำทั้งหมดในเอกสารนั้น ดังสมการ (1)

$$TF(t, d) = \frac{\text{number of times } ter(t) \text{ appears in document}(d)}{\text{total number of terms in document}(d)} \quad (1)$$

$$\begin{aligned} \text{เมื่อ } t &= \text{คำที่พิจารณา} \\ d &= \text{เอกสารที่พิจารณา} \end{aligned}$$

Inverse Document Frequency หรือก็คือ จำนวนของเอกสารทั้งหมด หารด้วยจำนวนเอกสารที่แต่ละคำปรากฏอยู่แล้วทำการ take log เข้าไป ดังสมการ (2)

$$IDF(t, D) = \log\left(\frac{\text{total number of document}(D)}{\text{number of document with the term}(t) \text{ in it}}\right) \quad (2)$$

$$\text{เมื่อ } D = \text{จำนวนของเอกสารทั้งหมด}$$

t = จำนวนเอกสารที่มีคำ t ปรากฏอยู่

จากนั้นนำค่า TF และ IDF มาคูณกันเพื่อให้ได้น้ำหนัก TF-IDF ของคำ t ในเอกสาร d ดังสมการ (3)

$$TF(t, d) \times IDF(t, D) \quad (3)$$

2.5.2 ความคล้ายคลึงโคไซน์ (Cosine Similarity)

ความคล้ายคลึงแบบโคไซน์ คือ การหาความเหมือนของข้อมูล โดยใช้ค่าโคไซน์จากการวัดค่าข้อมูลเป็นลายลักษณ์อักษรในเทคนิคการกรองแบบอ้างอิงเนื้อหา (Content-Based Filtering) จะถูกใช้เพื่อวัดค่าความเหมือนระหว่างเวกเตอร์ของน้ำหนัก TF-IDF, Word2Vec/Thai2Vec, Fasttext และ Sentence-BERT โดยค่าโคไซน์จะมีค่าอยู่ระหว่าง -1 ถึง 1 ยกตัวอย่างเช่น ค่า 1 หมายถึงข้อมูล A และ ข้อมูล B มีลักษณะข้อมูลที่เหมือนกัน และ ส่วนของค่า 0 นั้นหมายถึง ข้อมูล A และ ข้อมูล B มีลักษณะข้อมูลที่ไม่เหมือนกัน ดังสมการ (4)

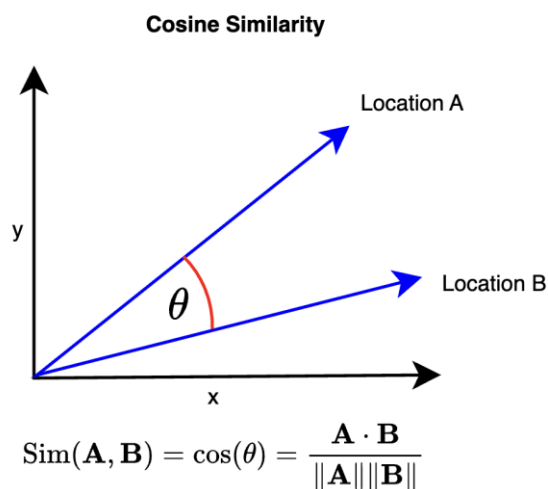
$$similarity = \cos(\theta) = \frac{A \cdot B}{||A|| ||B||} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (4)$$

เมื่อ A = รายการ A

B = รายการ B

n = จำนวนคู่ของรายการ A และ รายการ B ทั้งหมด

โดยมีที่มาจากกฎสามเหลี่ยม คือ $\cos(\theta) = \text{ชิด/ฉาก}$ โดยที่ A และ B คือรายการที่ต่างกัน ตามรูปที่ 7



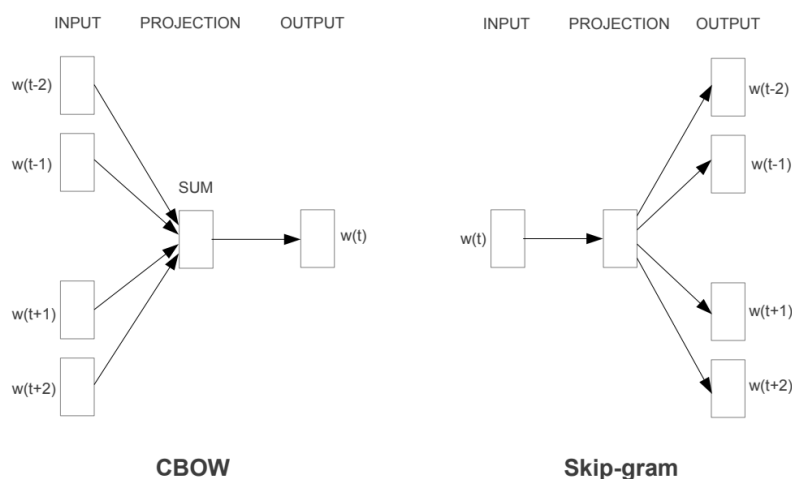
รูปที่ 7 การหาความคล้ายคลึงของโคไซน์

2.6 อัลกอริทึมในการสร้างแบบจำลองเทคนิคการกรองข้อมูลแบบอิงเนื้อหา

2.6.1 Word2Vec / Thai2Vec (Word Embedding)

Word2Vec เป็นเทคนิคสร้างตัวแทนคำในรูปแบบเวกเตอร์ (Word Embedding) ที่เสนอโดย Mikolov et al. (2013) โดยอาศัยแนวคิดที่ว่าคำที่ปรากฏร่วมกันในบริบทใกล้เคียงกันมักมีความหมายหรือความสัมพันธ์ทางภาษาใกล้เคียงกัน ดังนั้น คำเหล่านี้จึงมีแนวโน้มอยู่ใกล้กันในปริภูมิเวกเตอร์ วิธีการดังกล่าว ทำให้เวกเตอร์สามารถสะท้อนความสัมพันธ์เชิงความหมายและโครงสร้างทางภาษาได้

Word2Vec มีสถาปัตยกรรมหลัก 2 รูปแบบ ได้แก่ Continuous Bag-of-Words (CBOW) และ Skip-gram โดย CBOW ทำหน้าที่ทำนายคำเป้าหมายจากคำบริบทโดยรอบ ขณะที่ Skip-gram ทำหน้าที่ทำนายคำบริบทโดยรอบจากคำเป้าหมาย ทั้งสองสถาปัตยกรรมสามารถสร้างเวกเตอร์คำที่นำไปใช้วัดความคล้ายคลึงของคำ และประยุกต์ใช้ในงานประมวลผลภาษาธรรมชาติได้ (Mikolov et al., 2013)



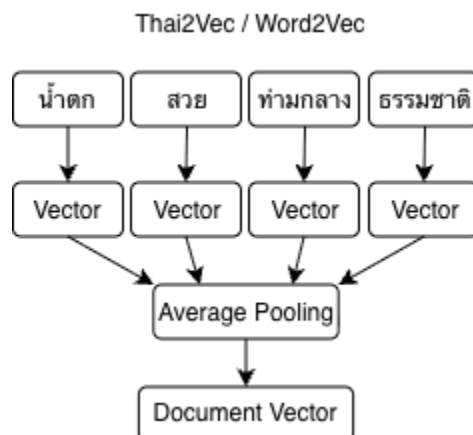
รูปที่ 8 สถาปัตยกรรม CBOW (Continuous Bag-of-Words) ทำนายคำปัจจุบันจากคำที่อยู่บริบทโดยรอบ ขณะที่สถาปัตยกรรม Skip-gram ทำนายคำที่อยู่โดยรอบจากคำปัจจุบัน

(Mikolov et al., 2013)

สำหรับภาษาไทย เนื่องจากไม่มีช่องว่างแบ่งคำ การตัดคำ (Word Segmentation) จึงเป็นขั้นตอนสำคัญก่อนนำข้อความไปใช้กับแบบจำลองเวกเตอร์คำ โดยไลบรารี PyThaiNLP มีโมดูล `word_vector` สำหรับใช้งานเวกเตอร์คำภาษาไทยที่ผ่านการฝึกไว้ล่วงหน้า เช่น `thai2fit_wv` และ `Large Thai Word2Vec (LTW2V)` (Phatthiyaphaibun et al., 2023)

Thai2Vec ในงานวิจัยนี้ หมายถึง เวกเตอร์คำภาษาไทยที่ผ่านการฝึกล่วงหน้าในชื่อ `thai2fit_wv` ซึ่งพัฒนาภายใต้โครงการ `thai2fit` และเผยแพร่ให้ใช้งานผ่าน PyThaiNLP (Polpanumas & Phatthiyaphaibun, 2021) เวกเตอร์คำชุดนี้ประกอบด้วยคำศัพท์ประมาณ 51,556 คำ โดยแต่ละคำแสดงด้วยเวกเตอร์ขนาด 300 มิติ สามารถนำไปใช้วิเคราะห์ความสัมพันธ์เชิงความหมายระหว่างคำ รวมถึงการสร้างเวกเตอร์ตัวแทนของประโยคหรือเอกสารได้

การสร้างเวกเตอร์ระดับเอกสารจาก Thai2Vec สามารถทำได้โดยนำเวกเตอร์ของคำที่ปรากฏในข้อความมารวมกัน เช่น การคำนวณค่าเฉลี่ยของเวกเตอร์คำ (Average Pooling) เพื่อให้ได้เวกเตอร์ตัวแทนของข้อความหนึ่งรายการ วิธีดังกล่าวเป็นการสร้างตัวแทนข้อความจากเวกเตอร์คำที่มีอยู่ในแบบจำลอง (Phatthiyaphaibun et al., 2023) ดังรูปที่ 9



รูปที่ 9 กระบวนการสร้างเวกเตอร์เอกสารด้วย Thai2Vec

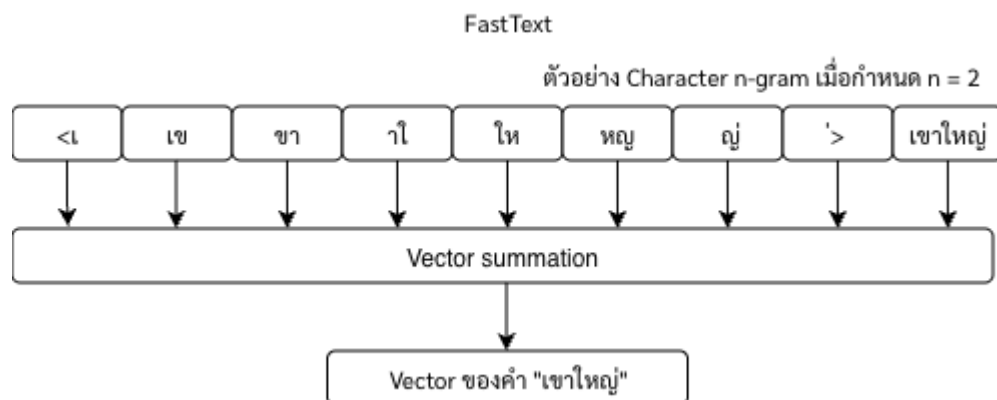
ในงานวิจัยนี้ Thai2Vec ถูกนำมาใช้แปลงชื่อและรายละเอียดสถานที่ให้เป็นเวกเตอร์ โดยรวมเวกเตอร์ของคำในข้อความเพื่อสร้างตัวแทนของแต่ละสถานที่ แล้วคำนวณความคล้ายคลึงด้วย Cosine Similarity ระหว่างเวกเตอร์ เพื่อวัดความใกล้เคียงเชิงความหมาย และจัดอันดับสถานที่ที่มีเนื้อหาใกล้เคียงกัน

ทั้งนี้ Thai2Vec แตกต่างจาก Word2Vec ที่ฝึกจากชุดข้อมูลของงานวิจัยโดยตรง กล่าวคือ Thai2Vec เป็นเวกเตอร์คำภาษาไทยที่ผ่านการฝึกล่วงหน้าแล้ว ขณะที่กรณีระบบไม่สามารถโหลด thai2fit_wv ได้ ระบบจะฝึก Word2Vec แบบ Skip-gram จากชุดข้อมูลแหล่งท่องเที่ยวเป็นกลไกสำรอง ซึ่งเรียกว่า Word2Vec ที่ฝึกจากชุดข้อมูลแหล่งท่องเที่ยว และไม่ถือเป็น Thai2Vec

2.6.2 FastText

Fasttext เป็นแบบจำลองสร้างตัวแทนคำในรูปแบบเวกเตอร์ที่พัฒนาโดย Bojanowski et al. (2017) ต่อยอดจาก Word2Vec โดยใช้ข้อมูลระดับอักขระย่อย (Subword) ร่วมกับข้อมูลระดับคำ ช่วยให้แบบจำลองเรียนรู้คำที่พบไม่บ่อยและคำที่มีรูปแบบคล้ายกันได้ดีขึ้น (Bojanowski et al., 2017)

หลักการสำคัญของ FastText คือการแทนคำหนึ่งคำด้วยกลุ่มของ Character n-grams โดยแต่ละ n-gram จะมีเวกเตอร์เป็นของตัวเอง จากนั้นเวกเตอร์ของคำจะถูกสร้างขึ้นจากการรวมเวกเตอร์ของ n-grams ที่ประกอบเป็นคำนั้น (ดังรูปที่ 10) ตัวอย่างเช่น หากกำหนดขนาดของ n-gram เป็น 2 ตัวอักษร คำว่า "เขาใหญ่" จะถูกแบ่งเป็นชุดอักขระย่อย <เ, เข, ขา, าเ, ไห, หญ, ญ, > พร้อมกับเวกเตอร์ของคำเต็ม แล้วนำมารวมกันเป็นเวกเตอร์ตัวแทนของคำ วิธีนี้ทำให้ FastText สามารถสร้างเวกเตอร์สำหรับคำที่ไม่เคยปรากฏในชุดข้อมูลฝึก (Out-of-Vocabulary) ได้ ซึ่งเป็นข้อได้เปรียบสำคัญเมื่อเทียบกับ Word2Vec แบบดั้งเดิม (Bojanowski et al., 2017)



รูปที่ 10 การแทนคำด้วย Character n-grams ของ FastText กรณีคำว่า "เขาใหญ่"

สำหรับภาษาไทยที่มีคำเฉพาะ ชื่อสถานที่ และรูปแบบการเขียนหลากหลาย การใช้ข้อมูลระดับอักขระย่อยช่วยให้ FastText จัดการคำใหม่หรือคำที่พบน้อยได้ดีกว่า Word Embedding ที่ใช้เฉพาะระดับคำ

ในงานวิจัยด้านระบบแนะนำสถานที่ท่องเที่ยว FastText ถูกนำมาใช้สร้างเวกเตอร์จากชื่อและรายละเอียดสถานที่หลังผ่านการตัดคำ แล้วรวมเวกเตอร์คำเป็นตัวแทนของแต่ละสถานที่ ก่อนคำนวณความคล้ายคลึงด้วย Cosine Similarity เพื่อจัดอันดับสถานที่ที่ใกล้เคียงกัน

อย่างไรก็ตาม FastText ยังเป็น Static Word Embedding ที่เวกเตอร์ของคำไม่เปลี่ยนแปลงตามบริบทประโยค จึงมีข้อจำกัดในการแยกแยะคำหลายความหมาย เมื่อเทียบกับแบบจำลอง Contextual Embedding เช่น BERT นอกจากนี้การจัดเก็บเวกเตอร์ของ n-grams ยังเพิ่มภาระด้านหน่วยความจำและพารามิเตอร์

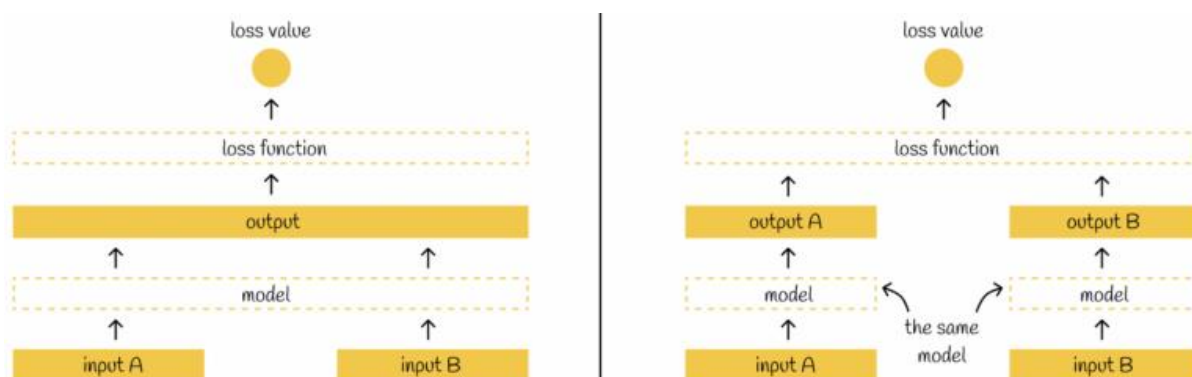
ในงานวิจัยนี้ FastText ถูกนำมาใช้เป็นหนึ่งในวิธีสร้างตัวแทนข้อความภาษาไทย โดยผลลัพธ์จะถูกเปรียบเทียบกับ TF-IDF, Thai2Vec และ BERT เพื่อประเมินประสิทธิภาพในการจัดอันดับสถานที่ท่องเที่ยวที่มีเนื้อหาใกล้เคียงกัน

2.6.3 Sentence-BERT

Sentence-BERT (SBERT) คือโมเดลโครงข่ายประสาทเทียมที่พัฒนาต่อยอดจาก BERT โดยใช้สถาปัตยกรรมแบบ Siamese Network เพื่อแปลงข้อความหรือประโยคให้ออกมาเป็นเวกเตอร์ (Sentence Embeddings) สามารถนำไปวัดความคล้ายคลึงเชิงความหมายได้อย่างมีประสิทธิภาพ (Reimers & Gurevych, 2019)

ข้อจำกัดของโมเดล BERT แบบดั้งเดิม (Vanilla BERT) ถูกออกแบบมาเพื่อสร้างเวกเตอร์แทนความหมายในระดับคำ (Word-level Embeddings) การจะวัดความคล้ายคลึงของประโยคด้วย BERT โดยตรง จำเป็นต้องนำประโยคเป็นคู่ใส่เข้าไปในแบบจำลองพร้อมกัน (Cross-encoder) ซึ่งส่งผลให้เกิดความซับซ้อนในการคำนวณสูง ($O(n^2)$) และไม่สามารถประมวลผลข้อความแยกอิสระได้ ส่วนการใช้วิธีค่าเฉลี่ยของเวกเตอร์คำ (Mean Pooling) หรือการใช้เวกเตอร์จากโทเคน [CLS] จาก BERT ดั้งเดิม มักให้ผลลัพธ์ด้านความคล้ายคลึงเชิงความหมายที่ไม่มีประสิทธิภาพเพียงพอ (Reimers & Gurevych, 2019)

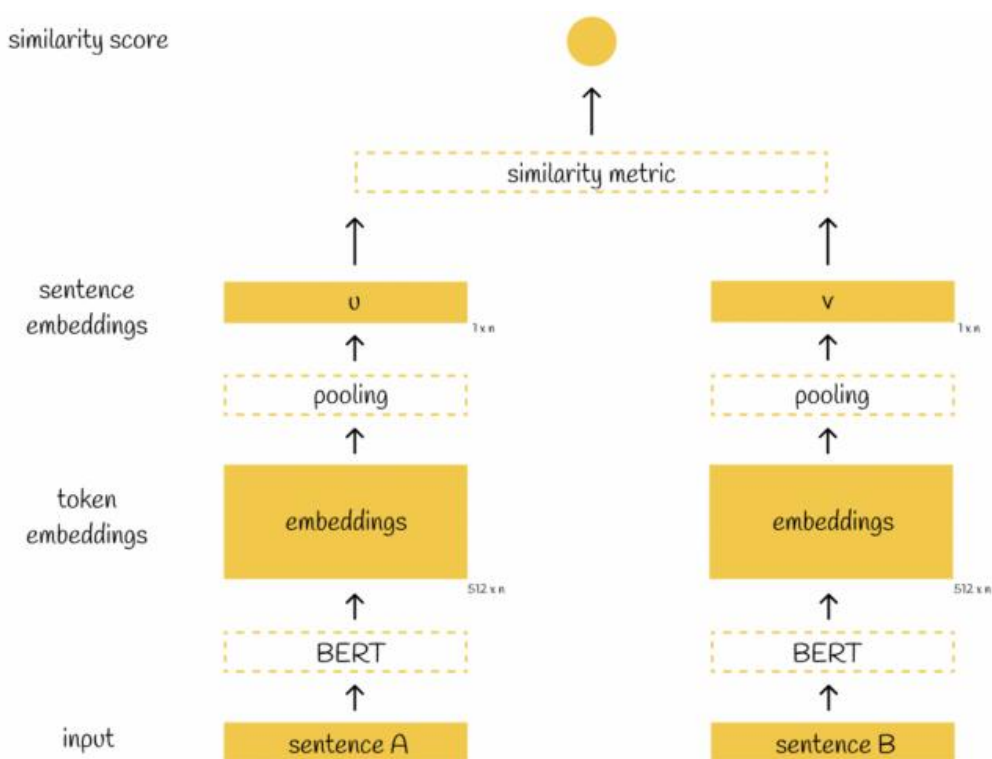
สถาปัตยกรรมของ Sentence-BERT (SBERT) แก้ปัญหาข้างต้นด้วยการใช้โครงสร้าง Siamese Network ทำให้สามารถป้อนประโยคและประมวลผลแยกจากกันได้อย่างเป็นอิสระ (Bi-encoder) เมื่อข้อความถูกส่งผ่านโครงสร้างย่อยของ BERT แบบจำลองจะทำการรวมเวกเตอร์แบบ Mean-pooling เพื่อแปลงเวกเตอร์คำหลาย ๆ คำ ให้กลายเป็นเวกเตอร์ตัวแทนของประโยคเพียงเวกเตอร์เดียว ซึ่งช่วยให้เวกเตอร์สามารถเก็บข้อมูลเชิงบริบทของประโยคได้อย่างครบถ้วน (Reimers & Gurevych, 2019)



รูปที่ 11 เปรียบเทียบสถาปัตยกรรม Non-Siamese (cross-encoder) (ซ้าย) และสถาปัตยกรรม Siamese (bi-encoder) (ขวา) ในกระบวนการดึงข้อมูล embeddings ระดับประโยค

(Efimov, S., 2023)

หลังจากได้เวกเตอร์ตัวแทนประโยค (u และ v) SBERT จะใช้ฟังก์ชันเป้าหมายในการปรับแต่งแบบจำลองให้เข้าใจความหมายเชิงลึก โดยการคำนวณค่าความคล้ายคลึงเชิงโคไซน์ (Cosine Similarity) ระหว่างเวกเตอร์ประโยค และใช้ฟังก์ชันความสูญเสียแบบ Mean Squared Error (MSE) ในการปรับค่าน้ำหนัก ซึ่งช่วยให้การคำนวณเปรียบเทียบในขั้นตอนการใช้งานจริง (Inference) สามารถทำได้อย่างรวดเร็วในระดับมิลลิวินาที (Efimov, 2023)



รูปที่ 12 สถาปัตยกรรมของ Sentence-BERT รูปแบบฟังก์ชันเป้าหมายแบบถดถอย (Regression Objective) สำหรับการวัดความคล้ายคลึงเชิงโคไซน์

(Efimov, S., 2023)

2.7 งานวิจัยที่เกี่ยวข้อง

งานวิจัยนี้ได้ทำการศึกษาค้นคว้างานวิจัยที่เกี่ยวข้องกับเทคนิคการกรองแบบอิงเนื้อหา (Content-Based Filtering) โดยงานวิจัยที่เกี่ยวข้องมีรายละเอียด ดังนี้

(1) บทความวิจัยเรื่อง ระบบแนะนำสินค้าอาหารโดยใช้ระบบแนะนำแบบผสมผสาน : Food recommendation system using a hybrid recommendation method (นิภาภรณ์ พันธนาม และ วราภรณ์ วิทยานนท์, 2563)

นิภาภรณ์ พันธนาม และ วราภรณ์ วิทยานนท์ ได้ศึกษาระบบแนะนำสินค้าอาหารโดยใช้ระบบแนะนำแบบผสมผสาน (Hybrid Recommendation) โดยนำแนวคิดของระบบแนะนำหลายรูปแบบมาประยุกต์ใช้ร่วมกัน เพื่อเพิ่มประสิทธิภาพในการแนะนำสินค้าอาหารให้เหมาะสมกับผู้ใช้งาน งานวิจัยดังกล่าวมีความเกี่ยวข้องกับการพัฒนาระบบแนะนำ เนื่องจากแสดงให้เห็นถึงการประยุกต์ใช้ข้อมูลและเทคนิคการแนะนำ

เพื่อสร้างผลลัพธ์ที่เหมาะสมกับความสนใจของผู้ใช้งาน ซึ่งสามารถนำมาเป็นแนวทางในการพัฒนาระบบแนะนำสถานที่ท่องเที่ยวในงานวิจัยนี้ได้

(2) บทความวิจัยเรื่อง Intelligent tourism recommender system: A survey (Borràs, Moreno และ Valls, 2014)

Borràs ได้ศึกษาระบบแนะนำด้านการท่องเที่ยวอัจฉริยะ โดยทบทวนแนวทางการพัฒนาระบบแนะนำสำหรับการท่องเที่ยว ซึ่งครอบคลุมการแนะนำสถานที่ท่องเที่ยว จุดหมายปลายทาง และการวางแผนการเดินทาง ผลการศึกษาพบว่าสามารถประยุกต์ใช้เทคนิคการกรองข้อมูลหลายรูปแบบ เช่น Content-Based Filtering และ Collaborative Filtering เพื่อสร้างคำแนะนำที่เหมาะสมกับผู้ใช้งาน

(3) บทความวิจัยเรื่อง How smart is e-tourism? A systematic review of smart tourism recommendation system applying data management (Hamid et al, 2021)

Hamid et al ได้ศึกษาทบทวนระบบแนะนำด้านการท่องเที่ยวอัจฉริยะ โดยรวบรวมและวิเคราะห์งานวิจัยที่เกี่ยวข้องกับระบบแนะนำ พบว่าวิธีการที่ได้รับความนิยม ได้แก่ Collaborative Filtering, Content-Based Filtering, Context-Based และ Hybrid Recommendation ซึ่งสามารถนำข้อมูลเกี่ยวกับผู้ใช้งาน สถานที่ท่องเที่ยว และบริบทต่าง ๆ มาใช้ในการสร้างคำแนะนำ

(4) บทความวิจัยเรื่อง A survey on point-of-interest recommendations leveraging heterogeneous data (Wang, Höpken และ Jannach, 2025)

Wang ได้ศึกษาระบบแนะนำจุดสนใจ (Point-of-Interest Recommendation) โดยทบทวนการนำข้อมูลหลายประเภทมาใช้ในการแนะนำสถานที่ เช่น ข้อมูลสถานที่ พฤติกรรมของผู้ใช้งาน และข้อมูลบริบท ผลการศึกษาพบว่า การนำข้อมูลที่หลากหลายมาประกอบสร้างระบบแนะนำสามารถช่วยเพิ่มความสามารถของระบบในการคาดการณ์สถานที่ที่ผู้ใช้งานมีแนวโน้มสนใจ

(5) บทความวิจัยเรื่อง Efficient Estimation of Word Representations in Vector Space (Mikolov et al., 2013)

Mikolov et al. ได้นำเสนอแบบจำลอง Word2Vec สำหรับสร้างตัวแทนคำในรูปแบบเวกเตอร์ โดยมีวัตถุประสงค์เพื่อเรียนรู้ความสัมพันธ์ระหว่างคำจากบริบทที่คำปรากฏในข้อความ แบบจำลองประกอบด้วย

แนวทางสำคัญ ได้แก่ Continuous Bag-of-Words (CBOW) และ Skip-gram ซึ่งสามารถเรียนรู้ตัวแทนคำจากความสัมพันธ์ระหว่างคำและบริบทได้ ผลการศึกษาพบว่าเวกเตอร์ที่ได้สามารถสะท้อนความสัมพันธ์เชิงความหมายและความสัมพันธ์ระหว่างคำได้ จึงสามารถนำไปประยุกต์ใช้กับงานประมวลผลภาษาธรรมชาติหลายประเภท

(6) บทความวิจัยเรื่อง Distributed Representations of Words and Phrases and their Compositionality (Mikolov et al., 2013)

Mikolov et al. ได้ศึกษาและปรับปรุงแนวทางการสร้างตัวแทนคำและวลีด้วย Word2Vec โดยเน้นการเรียนรู้ความสัมพันธ์ของคำจากบริบทและการสร้างตัวแทนที่สามารถสะท้อนความสัมพันธ์เชิงความหมายของคำ งานวิจัยได้นำเสนอแนวทาง Skip-gram และวิธีการปรับปรุงประสิทธิภาพของการฝึกแบบจำลอง เช่น Negative Sampling ผลการศึกษาพบว่าแบบจำลองสามารถสร้างตัวแทนคำที่มีคุณภาพและสามารถนำไปใช้กับงานที่ต้องการวิเคราะห์ความสัมพันธ์ระหว่างคำได้

(7) เอกสารเรื่อง PyThaiNLP: Thai Natural Language Processing in Python (Phatthiyaphaibun et al., 2023)

Phatthiyaphaibun et al. ได้พัฒนา PyThaiNLP ซึ่งเป็นไลบรารีสำหรับการประมวลผลภาษาธรรมชาติภาษาไทยในภาษา Python และมีเครื่องมือสำหรับการประมวลผลข้อความภาษาไทย รวมถึงการใช้งาน Word Embedding และ Thai2Vec เอกสารดังกล่าวแสดงแนวทางการนำ Word Vector มาใช้กับข้อมูลภาษาไทย และช่วยสนับสนุนการประยุกต์ใช้ Word2Vec กับงานประมวลผลภาษาธรรมชาติภาษาไทย

(8) เอกสาร Thai2Vec Embeddings Examples (PyThaiNLP, n.d.)

PyThaiNLP ได้นำเสนอตัวอย่างการใช้งาน Thai2Vec ซึ่งเป็นตัวแทนคำภาษาไทยในรูปแบบเวกเตอร์ โดยสามารถนำเวกเตอร์มาใช้ในการค้นหาคำที่มีความสัมพันธ์หรือความคล้ายคลึงกัน และนำไปประยุกต์ใช้กับงานประมวลผลภาษาธรรมชาติภาษาไทยได้ ตัวอย่างดังกล่าวแสดงให้เห็นแนวทางการนำ Thai2Vec มาใช้ในการวิเคราะห์ความสัมพันธ์ของคำและข้อความ

(9) บทความวิจัยเรื่อง Enriching Word Vectors with Subword Information (Bojanowski et al., 2017)

Bojanowski et al. ได้นำเสนอแบบจำลอง FastText ซึ่งเป็นแบบจำลองสำหรับสร้างตัวแทนคำในรูปแบบเวกเตอร์ โดยพัฒนาต่อยอดจากแนวคิดของ Word2Vec และเพิ่มการใช้ข้อมูลระดับอักขระย่อย (Subword Information) เข้ามาในการสร้างตัวแทนคำ แบบจำลองจะแทนคำด้วยกลุ่มของ Character n-grams และนำข้อมูลดังกล่าวมาใช้ร่วมกับข้อมูลระดับคำ ผลการศึกษาพบว่าแนวทางดังกล่าวช่วยปรับปรุงการสร้างตัวแทนคำ โดยเฉพาะคำที่พบบ่อย และสามารถสร้างตัวแทนสำหรับคำที่ไม่ปรากฏในชุดข้อมูลฝึกได้ งานวิจัยนี้จึงเป็นพื้นฐานสำคัญสำหรับการนำ FastText ไปประยุกต์ใช้กับงานประมวลผลภาษาธรรมชาติ

(10) บทความวิจัยเรื่อง Learning Word Vectors for 157 Languages (Grave et al., 2018)

Grave et al. ได้นำเสนอเวกเตอร์คำที่ผ่านการฝึกด้วย FastText สำหรับภาษาจำนวน 157 ภาษา โดยใช้ข้อมูลจาก Wikipedia ในการฝึกแบบจำลอง งานวิจัยมีวัตถุประสงค์เพื่อจัดเตรียมตัวแทนคำที่สามารถนำไปประยุกต์ใช้กับงานประมวลผลภาษาธรรมชาติในหลายภาษา ผลงานดังกล่าวแสดงให้เห็นถึงความสามารถของ FastText ในการสร้างตัวแทนคำสำหรับภาษาที่มีความหลากหลาย และเป็นแหล่งข้อมูลสำหรับการนำ Pre-trained Word Vectors ไปประยุกต์ใช้ในงานวิจัยด้านภาษา

(11) บทความวิจัยเรื่อง Bag of Tricks for Efficient Text Classification (Joulin et al., 2017)

Joulin et al. ได้นำเสนอแนวทางการจำแนกข้อความที่มีประสิทธิภาพโดยใช้แนวคิด Bag of Words และ N-grams ร่วมกับแบบจำลองที่มีโครงสร้างไม่ซับซ้อน เพื่อให้สามารถฝึกแบบจำลองกับข้อมูลจำนวนมากได้อย่างรวดเร็ว งานวิจัยแสดงให้เห็นว่าการใช้ข้อมูลระดับ N-gram สามารถช่วยให้แบบจำลองเรียนรู้ข้อมูลจากองค์ประกอบย่อยของข้อความได้ และสามารถนำแนวคิดดังกล่าวไปประยุกต์ใช้กับงานประมวลผลข้อความประเภทต่าง ๆ ได้

(12) บทความวิจัยเรื่อง Sentence-BERT: Sentence embeddings using Siamese BERT-networks (Reimers และ Gurevych, 2019)

Reimers และ Gurevych ได้นำเสนอโครงสร้าง Siamese Network เพื่อประยุกต์ใช้กับ BERT ในการสร้างเวกเตอร์แทนประโยค (Sentence Embedding) ซึ่งสามารถวัดความคล้ายคลึงเชิงความหมายได้รวดเร็วและมีประสิทธิภาพสูงกว่าการใช้ BERT ดั้งเดิม งานวิจัยนี้มีความสำคัญต่อการเลือกใช้อัลกอริทึมการสร้างตัวแทนข้อความในระดับประโยค ซึ่งเป็นส่วนสำคัญที่ช่วยให้ระบบแนะนำสามารถวิเคราะห์ความหมายของคำค้นและรายละเอียดสถานที่ท่องเที่ยวได้ลึกซึ้งขึ้น

จากการทบทวนวรรณกรรมข้างต้น พบว่างานวิจัยที่ผ่านมาส่วนใหญ่ศึกษาเทคนิคการแทนข้อความเพียงหนึ่งหรือสองเทคนิค และมักดำเนินการบนคลังข้อมูลภาษาอังกฤษซึ่งมีตัวค้นคำชัดเจน ในขณะที่ภาษาไทยมีลักษณะเฉพาะคือ ไม่มีการเว้นวรรคระหว่างคำ ทำให้ผลลัพธ์ของการตัดคำส่งผลโดยตรงต่อคุณภาพของเวกเตอร์ นอกจากนี้งานวิจัยด้านระบบแนะนำแหล่งท่องเที่ยวไทยจำนวนหนึ่งยังประเมินผลด้วยค่าความคล้ายคลึงเชิงตัวเลขเพียงอย่างเดียว งานวิจัยนี้จึงเติมเต็มช่องว่างดังกล่าว ด้วยการเปรียบเทียบเทคนิคการแทนข้อความ 4 เทคนิคบนคลังข้อมูลภาษาไทยชุดเดียวกันภายใต้กระบวนการเตรียมข้อมูลที่ควบคุมให้เหมือนกัน

บทที่ 3

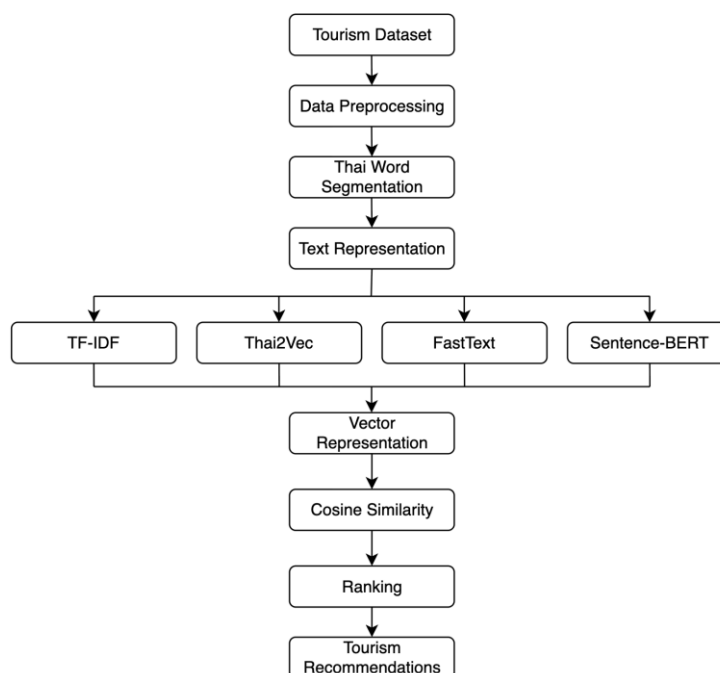
วิธีดำเนินการวิจัย

ในการดำเนินการวิจัย ผู้วิจัยได้ดำเนินการดังต่อไปนี้

1. กระบวนการสร้างแบบจำลอง
2. ข้อมูลที่นำมาใช้วิจัย
3. การสำรวจข้อมูล
4. การเตรียมข้อมูล
5. การออกแบบอัลกอริทึมที่ใช้ในการสร้างคำแนะนำและพารามิเตอร์
6. การออกแบบการทดสอบและเปรียบเทียบ
7. สรุปผลการดำเนินงาน

3.1 กระบวนการสร้างแบบจำลอง

ในการพัฒนาระบบแนะนำสถานที่ท่องเที่ยว ผู้วิจัยได้สร้างแบบจำลองโดยประยุกต์ใช้เทคนิคการกรองเนื้อหา (Content-Based Filtering) โดยใช้ข้อมูลแหล่งท่องเที่ยวจากไฟล์ attraction.csv เป็นชุดข้อมูลสำหรับการพัฒนาแบบจำลอง ขั้นแรกดำเนินการตรวจสอบและเตรียมความพร้อมของข้อมูล (Data Preprocessing) จากนั้นทำการตัดคำภาษาไทย (Thai Word Segmentation) และแปลงข้อมูลข้อความให้อยู่ในรูปแบบที่เหมาะสมสำหรับการประมวลผล (Text Representation) โดยประยุกต์ใช้เทคนิคจำนวน 4 วิธี ได้แก่ TF-IDF, Thai2Vec, FastText และ Sentence-BERT เพื่อสร้างตัวแทนข้อมูลในรูปแบบเวกเตอร์ (Vector Representation) หลังจากนั้นจึงคำนวณค่าความคล้ายคลึงระหว่างข้อมูลด้วยวิธี Cosine Similarity และจัดอันดับ (Ranking) ผลลัพธ์ตามระดับความคล้ายคลึง เพื่อนำเสนอเป็นคำแนะนำสถานที่ท่องเที่ยว ดังแสดงในรูปที่



รูปที่ 13 กรอบแนวคิดและกระบวนการสร้างระบบแนะนำสถานที่ท่องเที่ยวด้วยเทคนิค Content-Based Filtering

3.2 ข้อมูลที่นำมาใช้วิจัย

การสร้างแบบจำลองใช้ชุดข้อมูลแหล่งท่องเที่ยว (การท่องเที่ยวแห่งประเทศไทย., 2026) ซึ่งประกอบด้วยข้อมูลของสถานที่ท่องเที่ยวภายในประเทศไทย ทั้งหมด 8,848 รายการ ในรูปแบบของไฟล์ csv จำนวน 1 ไฟล์ ได้แก่ ชุดข้อมูล attraction.csv คือชุดข้อมูลที่อธิบายเกี่ยวกับชื่อสถานที่ คำอธิบายสถานที่ และข้อมูลที่เกี่ยวข้องกับสถานที่

3.3 การสำรวจข้อมูล

การวิจัยครั้งนี้ผู้วิจัยได้นำชุดข้อมูลสถานที่ จาก TAT Data Catalog โดยใช้ไฟล์ข้อมูลจำนวน 1 ชุด ได้แก่ ชุดข้อมูล attraction.csv โดยชุดข้อมูลต้นฉบับประกอบด้วยข้อมูลจำนวน 71 คอลัมน์ อย่างไรก็ตามผู้วิจัยได้คัดเลือกเฉพาะตัวแปรที่เกี่ยวข้องมาทั้งหมด 6 คอลัมน์ ดังตาราง 1 ทางผู้วิจัยได้นำมาทดสอบโดยใช้เทคนิคการกรองแบบอิงเนื้อหา (Content-Based Filtering) เพื่อนำมาวิเคราะห์ในการแนะนำสถานที่ท่องเที่ยวภายในประเทศไทย

ตารางที่ 1 โครงสร้างชุดข้อมูลแหล่งท่องเที่ยว (attraction.csv)

ข้อมูลตัวแปร (Variable)	คำอธิบายข้อมูล (Description)
ATT_ID	รหัสของสถานที่
ATT_NAME_TH	ชื่อสถานที่
ATT_CATEGORY_LABEL	ประเภทของสถานที่
ATT_DETAIL_TH	รายละเอียดสถานที่ภาษาไทย
ATT_DETAIL_EN	รายละเอียดสถานที่ภาษาอังกฤษ
PROVINCE_NAME_TH	จังหวัดที่อยู่ของสถานที่

ผลจากการสำรวจข้อมูลจะถูกนำไปใช้เป็นแนวทางในการคัดเลือกและเตรียมข้อมูลในขั้นตอนถัดไป เพื่อให้ข้อมูลมีความเหมาะสมสำหรับการสร้างแบบจำลองระบบแนะนำ

3.4 การเตรียมข้อมูล

การเตรียมข้อมูล (Data Preprocessing) เป็นขั้นตอนสำคัญก่อนนำข้อมูลไปใช้ในการสร้างแบบจำลองระบบแนะนำสถานที่ท่องเที่ยว โดยมีวัตถุประสงค์เพื่อจัดการข้อมูลที่ไม่สมบูรณ์ ข้อมูลที่ไม่เหมาะสมสำหรับการวิเคราะห์ ทำความสะอาดข้อมูล กำจัดข้อมูลซ้ำ การคัดเลือกตัวแปร การปรับชื่อฟิลด์ การสร้างข้อความสำหรับใช้เป็นคุณลักษณะ การตัดคำภาษาไทย และการกำจัดคำหยุดเฉพาะโดเมน ทั้งนี้ การเตรียมข้อความจะแตกต่างกันตามลักษณะของแบบจำลองที่นำมาใช้

ในการวิจัยครั้งนี้ ผู้วิจัยดำเนินการเตรียมข้อมูลจากชุดข้อมูล attraction.csv ตามลำดับขั้นตอนดังต่อไปนี้

3.4.1 การจัดการข้อมูลที่มีค่าขาดหายไป

ผู้วิจัยตรวจสอบความสมบูรณ์ของข้อมูลตัวแปร ATT_DETAIL_TH ซึ่งเป็นรายละเอียดของแหล่งท่องเที่ยวภาษาไทย และเป็นข้อมูลสำคัญที่นำไปใช้ในการสร้างแบบจำลอง ส่งผลให้ข้อมูลที่ไม่มีค่าในตัวแปร ATT_DETAIL_TH ถูกนำออกจากชุดข้อมูลด้วยการตรวจสอบเฉพาะตัวแปรดังกล่าว โดยใช้วิธี dropna() ส่งผลให้จำนวนข้อมูลลดลงจาก 8,848 รายการ เหลือ 7,766 รายการ

3.4.2 การจัดการข้อมูลที่มีค่าไม่เหมาะสม

หลังจากจัดการข้อมูลที่มีค่าขาดหาย ผู้วิจัยได้ตรวจสอบข้อมูลในตัวแปร ATT_DETAIL_TH เพิ่มเติม โดยพบว่าในบางรายการใช้เครื่องหมายขีด (-) หรือเครื่องหมายดอกจัน (*) แทนข้อมูลรายละเอียดของสถานที่ท่องเที่ยว ดังนั้น ผู้วิจัยจึงคัดกรองข้อมูลที่มีค่าเป็น - และ * ออกจากชุดข้อมูล เนื่องจากข้อมูลดังกล่าวไม่ได้ให้รายละเอียดที่สามารถนำไปใช้ในการวิเคราะห์เนื้อหาได้ หลังจากดำเนินการคัดกรองแล้ว จำนวนข้อมูลลดลงจาก 7,766 รายการ เหลือ 7,516 รายการ

3.4.3 การทำความสะอาดข้อมูล HTML

ผู้วิจัยดำเนินการทำความสะอาดข้อมูลข้อความในตัวแปร ATT_DETAIL_TH และ ATT_DETAIL_EN เนื่องจากข้อมูลบางรายการอาจประกอบด้วยโครงสร้าง HTML โดยใช้ไลบรารี BeautifulSoup เพื่อแปลงข้อความที่มีโครงสร้าง HTML ให้เป็นข้อความปกติ กระบวนการประกอบด้วยการตรวจสอบค่าที่สูญหาย การแปลงข้อมูลให้อยู่ในรูปแบบข้อความ การแยกวิเคราะห์โครงสร้าง HTML การลบองค์ประกอบประเภท script และ style การดึงเฉพาะข้อความ และการตัดช่องว่างส่วนเกิน การดำเนินการดังกล่าวช่วยลดองค์ประกอบที่ไม่ใช่เนื้อหาของสถานที่ท่องเที่ยว และทำให้ข้อมูลข้อความมีความเหมาะสมสำหรับการประมวลผลในขั้นตอนถัดไป

3.4.4 การจัดการข้อมูลซ้ำ และค่าว่าง

หลังจากทำความสะอาดข้อมูล HTML ผู้วิจัยดำเนินการตรวจสอบที่มีค่าว่างในตัวแปร ATT_NAME_TH อีกครั้ง เนื่องจากบางรายการอาจเหลือเพียงแท็ก HTML หรืออักขระที่ไม่ใช่เนื้อหาหลังผ่านการทำความสะอาดแล้วโดยพบข้อมูลค่าว่างจำนวน 1 รายการ จึงนำออกจากชุดข้อมูล ส่งผลให้จำนวนข้อมูลคงเหลือ 7,515 รายการ

จากนั้น ผู้วิจัยตรวจสอบข้อมูลซ้ำโดยพิจารณาาร่วมกันจากตัวแปร ATT_NAME_TH และ ATT_NAME_EN หากพบข้อมูลที่มีทั้งชื่อและรายละเอียดภาษาไทยซ้ำกันจะถูกเก็บไว้เพียงรายการแรก โดยใช้คำสั่ง drop_duplicate() และกำหนดให้พิจารณาข้อมูลซ้ำจากตัวแปรทั้ง 2 ตัวร่วมกัน หลังจากการกำจัดข้อมูลซ้ำ

จำนวนข้อมูลลดลงจาก 7,515 รายการ เหลือ 7,478 รายการ ดังนั้นมีข้อมูลซ้ำที่ถูกลำเอียงจำนวน 37 รายการ โดยชุดข้อมูลสุดท้าย 7,478 รายการ

ตารางที่ 2 ตารางสรุปขั้นตอนการเตรียมข้อมูล

ขั้นตอน	จำนวนข้อมูลคงเหลือ	จำนวนที่ถูกลำเอียง
ข้อมูลเริ่มต้น	8,848	-
นำข้อมูลที่ไม่มี ATT_DETAIL_TH ออก	7,766	1,082
นำข้อมูลที่มีค่า - และ * ใน ATT_DETAIL_TH ออก	7,516	250
นำข้อมูลที่เป็นค่าว่าง ใน ATT_DETAIL_TH ออก หลังการทำความสะอาด HTML	7,515	1
กำจัดข้อมูลซ้ำ	7,478	37

รวมข้อมูลที่ถูกลำเอียงทั้งหมด 1,370 รายการ

จากกระบวนการเตรียมข้อมูลดังกล่าว ชุดข้อมูลที่ได้มีจำนวน 7,478 รายการ ซึ่งเป็นข้อมูลผ่านการคัดกรองและทำความสะอาดเบื้องต้นแล้ว ก่อนนำไปใช้ในขั้นตอนการคัดเลือกตัวแปร การปรับชื่อฟิลด์ และการประมวลผลข้อความสำหรับสร้างแบบจำลองระบบแนะนำต่อไป

3.4.5 การคัดเลือกตัวแปร

จากชุดข้อมูลต้นฉบับจำนวน 71 ตัวแปร ผู้วิจัยได้คัดเลือกเฉพาะตัวแปรที่มีความเกี่ยวข้องกับการพัฒนาระบบแนะนำสถานที่ท่องเที่ยวจำนวน 6 ตัวแปร ได้แก่ รหัสสถานที่ ชื่อสถานที่ ประเภทสถานที่ รายละเอียดภาษาไทย รายละเอียดภาษาอังกฤษ และจังหวัดที่ตั้ง การคัดเลือกตัวแปรดังกล่าวช่วยลดความซับซ้อนของชุดข้อมูล และทำให้การประมวลผลมุ่งเน้นไปยังข้อมูลที่เกี่ยวข้องกับการอธิบายลักษณะของสถานที่ท่องเที่ยว โดยข้อมูล ATT_CATEGORY_LABEL และ ATT_DETAIL_TH ถูกลำเอียงใช้เป็นข้อมูลข้อความหลัก

สำหรับสร้างตัวแทนของสถานที่ท่องเที่ยว ขณะที่ ATT_ID, ATT_NAME_TH และ PROVINCE_NAME_TH ใช้สำหรับระบุและนำเสนอผลลัพธ์ของสถานที่ที่ได้รับการแนะนำ

3.4.6 การปรับชื่อฟิลด์

หลังจากคัดเลือกตัวแปร ผู้วิจัยได้ปรับชื่อฟิลด์ (Column Mapping) จากชื่อเดิมของชุดข้อมูล ต้นฉบับให้เป็นชื่อมาตรฐานที่สะดวกต่อการนำไปใช้งานในกระบวนการประมวลผลและการสร้างแบบจำลอง โดยการปรับชื่อฟิลด์ทำให้ข้อมูลมีโครงสร้างมาตรฐานเดียวกัน และสามารถนำไปใช้ร่วมกันในกระบวนการสร้างแบบจำลองทั้ง 4 วิธีได้สะดวกยิ่งขึ้น ดังแสดงในตารางที่ 3

ตารางที่ 3 ตารางแสดงการปรับชื่อฟิลด์

ชื่อฟิลด์ต้นฉบับ	ชื่อฟิลด์ที่ใช้ในระบบ
ATT_ID	place_id
ATT_NAME_TH	name
ATT_CATEGORY_LABEL	category
ATT_DETAIL_TH	description_th
ATT_DETAIL_EN	description_en
PROVINCE_NAME_TH	location

3.4.7 การสร้างข้อความสำหรับใช้เป็นคุณลักษณะ

ในการสร้างตัวแทนข้อมูล ผู้วิจัยใช้ข้อมูลประเภทของสถานที่ (category) และรายละเอียดสถานที่ภาษาไทย (description_th) เป็นข้อมูลข้อความหลัก โดยนำข้อมูลทั้งสองฟิลด์มารวมกันเพื่อใช้แทนลักษณะของสถานที่ท่องเที่ยว

สำหรับการสร้างแบบจำลอง TF-IDF, Thai2Vec และ FastText มีการกำหนดให้น้ำหนักของ category มาเพิ่มเข้าในข้อความคุณลักษณะเพื่อเพิ่มน้ำหนัก และความสำคัญของประเภทสถานที่ในการสร้างเวกเตอร์ข้อความ วิธีดังกล่าวช่วยให้แบบจำลองสามารถพิจารณาคุณลักษณะของสถานที่ได้ทั้งในเชิงหมวดหมู่ และรายละเอียดเชิงเนื้อหา

สำหรับ Sentence-BERT ผู้วิจัยใช้ข้อความที่รวมจาก category และ description_th ในรูปแบบข้อความต่อเนื่อง โดยไม่ตัดคำ และไม่กำจัดคำหยุด เพื่อรักษาบริบทของข้อความให้ครบถ้วนสำหรับการสร้าง Sentence Embedding

3.4.8 การตัดคำภาษาไทย

เนื่องจากภาษาไทยไม่มีการเว้นวรรคระหว่างคำทุกคำอย่างชัดเจน ผู้วิจัยจึงดำเนินการตัดคำภาษาไทยสำหรับแบบจำลองที่ใช้คำเป็นหน่วยในการวิเคราะห์ ได้แก่ TF-IDF ในระดับคำ FastText และ การสร้างเวกเตอร์ระดับเอกสารจาก Thai2Vec โดยการตัดคำภาษาไทยดำเนินการด้วยไลบรารี PyThaiNLP โดยใช้ตัวตัดคำ newmm ซึ่งเหมาะสมกับการประมวลผลข้อความภาษาไทยทั่วไป หลังจากนั้นจึงนำคำที่ได้ไปใช้ในการสร้างตัวแทนข้อความของแต่ละแบบจำลอง

อย่างไรก็ตาม TF-IDF สามารถกำหนดการวิเคราะห์ข้อความในระดับ Character n-gram ได้ ซึ่งไม่จำเป็นต้องใช้การตัดคำภาษาไทยในลักษณะเดียวกัน ขณะที่ Sentence-BERT สามารถประมวลผลข้อความต่อเนื่องเพื่อสร้างตัวแทนระดับข้อความ หรือประโยคได้โดยตรง

3.4.9 การกำจัดคำหยุด

ผู้วิจัยกำจัดคำหยุด (Stopword Removal) สำหรับกระบวนการประมวลผลข้อความแบบ Word-based ได้แก่ TF-IDF ในระดับคำ FastText และ Thai2Vec เพื่อลดอิทธิพลของคำที่ปรากฏบ่อยแต่มีความสำคัญน้อยต่อการแยกความแตกต่างของสถานที่ท่องเที่ยว นอกจากนี้ ยังมีการกำหนดชุดคำหยุดเฉพาะโดเมนการท่องเที่ยว (Domain-Specific Stopwords) เช่น “นักท่องเที่ยว” “สถานที่” “เหมาะ” “เหมาะสำหรับ” “ชม” และ “เที่ยว” รวมถึงคำทั่วไปที่ปรากฏบ่อยในข้อความ

ทั้งนี้ ขั้นตอนการกำจัดคำหยุดไม่ถูกนำไปใช้กับ Sentence-BERT เนื่องจากแบบจำลองดังกล่าวอาศัยบริบท และโครงสร้างของข้อความต่อเนื่องในการสร้าง Sentence Embedding

3.4.10 การเตรียมข้อมูลสำหรับแบบจำลองแต่ละประเภท

เนื่องจากแบบจำลองแต่ละประเภทมีวิธีการประมวลผลข้อความที่แตกต่างกัน ผู้วิจัยจึงจัดเตรียมข้อมูลให้เหมาะสมกับแต่ละแบบจำลอง

ตารางที่ 4 การเตรียมข้อมูลสำหรับแบบจำลองแต่ละประเภท

แบบจำลอง	การเตรียมข้อมูล
TF-IDF	ทำความสะอาดข้อความและสร้างตัวแทนในระดับ Word หรือ Character n-gram ตามการกำหนด
Thai2Vec	ใช้แบบจำลอง thai2fit_wv ที่ผ่านการฝึกมาล่วงหน้า ตัดคำภาษาไทย และกำจัดคำหยุด ก่อนนำเวกเตอร์คำมารวมเป็นเวกเตอร์ระดับเอกสาร
FastText	ตัดคำภาษาไทย กำจัดคำหยุด และนำคำไปใช้สร้างตัวแทนข้อความ
Sentence-BERT	ใช้ข้อความต่อเนื่องที่ผ่านการทำความสะอาดแล้ว โดยไม่ตัดคำ และไม่กำจัดคำหยุด เพื่อสร้าง Sentence Embedding

3.5 การออกแบบอัลกอริทึมที่ใช้ในการสร้างคำแนะนำและพารามิเตอร์

หลังจากดำเนินการเตรียมข้อมูลตามขั้นตอนที่กล่าวมาในหัวข้อ 3.4 ผู้วิจัยได้นำข้อมูลข้อความของแหล่งท่องเที่ยวไปสร้างแบบจำลองระบบแนะนำด้วยเทคนิคการกรองแบบอิงเนื้อหา (Content-Based Filtering) จำนวน 4 วิธี ได้แก่ TF-IDF, Thai2Vec, FastText และ Sentence-BERT โดยแต่ละวิธีทำหน้าที่สร้างตัวแทนของข้อมูลสถานที่ท่องเที่ยวให้อยู่ในรูปแบบที่สามารถนำไปคำนวณหาความคล้ายคลึงระหว่างสถานที่ได้

ในการสร้างคำแนะนำ ข้อมูลของสถานที่ท่องเที่ยวแต่ละรายการจะถูกแปลงเป็นเวกเตอร์ตามวิธีของแต่ละแบบจำลอง จากนั้นนำเวกเตอร์ที่ได้มาคำนวณค่าความคล้ายคลึงด้วยวิธี Cosine Similarity และจัดอันดับสถานที่ท่องเที่ยวตามค่าความคล้ายคลึง เพื่อแนะนำสถานที่ที่มีเนื้อหาใกล้เคียงกับข้อมูลนำเข้า

3.5.1 TF-IDF

ในการสร้างแบบจำลองด้วย TF-IDF ผู้วิจัยนำข้อมูลข้อความที่ผ่านกระบวนการเตรียมข้อมูลมาแปลงเป็นเวกเตอร์ โดยกำหนดการวิเคราะห์ข้อความทั้งในระดับคำ (Word-level) และระดับอักขระ (Character-level) เพื่อเปรียบเทียบผลลัพธ์ของรูปแบบการวิเคราะห์ข้อความ

สำหรับการวิเคราะห์ระดับคำ ผู้วิจัยกำหนดให้วิเคราะห์ข้อความด้วย analyzer='word' และใช้ช่วง N-gram ตั้งแต่ Unigram ถึง Bigram หรือ ngram_range=(1,2) ส่วนการวิเคราะห์ระดับอักขระใช้ analyzer='char_wb' และกำหนดช่วง Character N-gram เป็น ngram_range=(3,5)

นอกจากนี้ ผู้วิจัยกำหนดค่า min_df เพื่อกำหนดความถี่ขั้นต่ำของคำที่นำมาพิจารณา กำหนดค่า max_df เพื่อคัดกรองคำที่ปรากฏในสัดส่วนสูงเกินไป ใช้ sublinear_tf=True สำหรับการปรับค่าความถี่ของคำในรูปแบบลอการิทึม และใช้ norm='l2' เพื่อปรับเวกเตอร์ให้อยู่ในรูปแบบ L2-normalization

ตารางที่ 5 ค่าพารามิเตอร์ในการสร้างแบบจำลองด้วยอัลกอริทึม TF-IDF Vectorizer

Alogorithm	Parameter	Value	Description
TF-IDF Vectorizer	analyzer	= 'word'	วิเคราะห์ระดับคำ
	ngram_range	= (1,2)	กำหนดความยาวช่วง Unigram ถึง Bigram
	min_df	= 1	เกณฑ์กรองความถี่คำขั้นต่ำ
	max_df	= 0.9	ตัดค่าที่มีความถี่สัดส่วนสูงเกินค่าพารามิเตอร์ไปออก
	sublinear_tf	= True	การสเกลความถี่แบบ Logarithmic ($1 + \log(tf)$)
	norm	= 'l2'	การทำ Normalization ในรูปแบบ L2-norm
TF-IDF Vectorizer	analyzer	= 'char_wb'	วิเคราะห์ระดับอักขระ
	ngram_range	= (3, 5)	กำหนดความยาวช่วง Character N-gram

	min_df	= 1	เกณฑ์กรองความถี่คำขั้นต่ำ
	max_df	= 0.95	ตัดคำที่มีความถี่สัดส่วนสูงเกินค่าพารามิเตอร์ไป ออก
	sublinear_tf	= True	การสเกลความถี่แบบ Logarithmic ($1 + \log(\text{tf})$)
	norm	= 'l2'	การทำ Normalization ในรูปแบบ L2-norm

3.5.2 Thai2Vec

ในการทดลอง ผู้วิจัยใช้แบบจำลองเวกเตอร์คำภาษาไทยที่ผ่านการฝึกไว้ล่วงหน้า ได้แก่ thai2fit_wv จากไลบรารี PyThaiNLP เป็นแบบจำลองหลักสำหรับวิธี Thai2Vec โดยผลการทดลองที่นำเสนอในบทที่ 4 ได้มาจากแบบจำลองที่ผ่านการฝึกล่วงหน้าดังกล่าว

อย่างไรก็ตาม ระบบได้จัดเตรียมกลไกสำรองสำหรับกรณีที่ไม่สามารถโหลดแบบจำลองที่ผ่านการโหลดล่วงหน้าได้ โดยระบบจะฝึกแบบจำลอง Word2Vec แบบ Skip-gram จากชุดข้อมูลแหล่งท่องเที่ยวภายในงานวิจัยแทน ทั้งนี้ แบบจำลองที่ฝึกขึ้นเองดังกล่าวเรียกว่า “Word2Vec ที่ฝึกจากชุดข้อมูลแหล่งท่องเที่ยว” และไม่นับเป็น Thai2Vec เนื่องจากไม่ได้ใช้เวกเตอร์คำภาษาไทยที่ผ่านการฝึกล่วงหน้า

สำหรับกรณีที่ไม่สามารถโหลดแบบจำลอง Thai2Vec ได้ การฝึก Word2Vec จากชุดข้อมูลแหล่งท่องเที่ยว โดยกำหนดสถาปัตยกรรมของแบบจำลองเป็น Skip-gram ด้วยค่า sg=1 กำหนดขนาดของเวกเตอร์เท่ากับ 100 มิติ (vector_size=100) และกำหนดขนาดของหน้าต่างบริบทเท่ากับ 5 คำ (window=5) โดยกำหนดให้คำที่ปรากฏอย่างน้อย 1 ครั้งสามารถนำมาใช้ในการสร้างแบบจำลองได้ด้วยค่า min_count=1 แบบจำลองถูกกำหนดให้เรียนรู้จำนวน 300 รอบ (epochs=300) และกำหนดค่าเริ่มต้นแบบสุ่มด้วย seed=42 เพื่อให้สามารถควบคุมความสม่ำเสมอของผลลัพธ์ในการทดลอง

ตารางที่ 6 ค่าพารามิเตอร์สำหรับการฝึกแบบจำลอง Word2Vec กรณีไม่สามารถโหลด Thai2Vec ได้

Alogorithm	Parameter	Value	Description
Word2Vec	sg	= 1	สถาปัตยกรรมโมเดล Skip-gram
	vector_size	= 100	ขนาดมิติเวกเตอร์
	window	= 5	ขนาดหน้าต่างบริบท
	min_count	= 1	เกณฑ์กรองความถี่ค่าขั้นต่ำ
	epochs	= 300	จำนวนรอบการเรียนรู้
	seed	= 42	กำหนดค่าเริ่มต้นสำหรับการสุ่ม

3.5.3 FastText

ในการสร้างแบบจำลองด้วย FastText ผู้วิจัยนำข้อมูลข้อความที่ผ่านกระบวนการเตรียมข้อมูลมาใช้ในการสร้างเวกเตอร์แทนคำ โดยกำหนดสถาปัตยกรรมของแบบจำลองเป็น Skip-gram ด้วยค่า sg=1 ผู้วิจัยกำหนดขนาดของเวกเตอร์เป็น 100 มิติ และกำหนดขนาดหน้าต่างบริบทเป็น 5 คำ นอกจากนี้ กำหนดช่วงความยาวของหน่วยคำย่อย (Character n-gram) ตั้งแต่ 3 ถึง 6 อักขระ ด้วยค่า min_n=3 และ max_n=6 แบบจำลองถูกกำหนดให้เรียนรู้จำนวน 300 รอบ เพื่อสร้างตัวแทนเวกเตอร์ที่สามารถนำไปใช้ในการเปรียบเทียบความคล้ายคลึงของข้อมูลสถานที่ท่องเที่ยว

ตารางที่ 7 ค่าพารามิเตอร์ในการสร้างแบบจำลองด้วยอัลกอริทึม FastText

Alogorithm	Parameter	Value	Description
FastText	sg	= 1	สถาปัตยกรรมโมเดล Skip-gram

	vector_size	= 100	ขนาดมิติเวกเตอร์
	window	= 5	ขนาดหน้าต่างบริบท
	min_n	= 3	กำหนดช่วงความยาวตัวอักษรย่อของคำขั้นต่ำ
	max_n	= 6	กำหนดช่วงความยาวตัวอักษรย่อของคำไม่เกินค่าพารามิเตอร์
	epochs	= 300	จำนวนรอบการเรียนรู้

3.5.4 Sentence-BERT

ในการสร้างแบบจำลองด้วย Sentence-BERT ผู้วิจัยใช้ข้อความที่ผ่านกระบวนการเตรียมข้อมูลเป็นข้อมูลนำเข้าเพื่อสร้างตัวแทนข้อความในรูปแบบ Sentence Embedding โดยใช้แบบจำลอง sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 ผู้วิจัยกำหนดขนาดของชุดข้อมูลที่ประมวลผลในแต่ละครั้ง (batch_size) เท่ากับ 32 และกำหนดให้ทำการปรับเวกเตอร์ที่ได้ให้อยู่ในรูปแบบ Normalized Embedding ด้วยค่า normalize_embeddings=True เวกเตอร์ที่ได้จาก Sentence-BERT ถูกนำไปใช้ในการคำนวณค่าความคล้ายคลึงกับข้อมูลสถานที่ท่องเที่ยว เพื่อจัดอันดับและสร้างคำแนะนำ

ตารางที่ 8 ค่าพารามิเตอร์ในการสร้างแบบจำลองด้วยอัลกอริทึม Sentence-BERT

Alogorithm	Parameter	Value	Description
Sentence-BERT	model_name	= sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2	ชื่อโมเดลภาษาที่ใช้

	batch_size	= 32	ขนาดการประมวลผลชุดข้อมูล
	normalize_embeddings	= True	การทำ Normalization

3.5.5 การคำนวณความคล้ายคลึงและการจัดอันดับ

หลังจากสร้างตัวแทนข้อมูลของสถานที่ท่องเที่ยวด้วยแบบจำลองทั้ง 4 วิธี ผู้วิจัยนำเวกเตอร์ที่ได้มาใช้ในการคำนวณค่าความคล้ายคลึงด้วยวิธี Cosine Similarity ซึ่งหลักการและสมการได้อธิบายไว้ในบทที่ 2 แล้ว เพื่อวัดความใกล้เคียงระหว่างข้อมูลนำเข้าและข้อมูลสถานที่ท่องเที่ยวในชุดข้อมูล

สำหรับการทดสอบด้วยชื่อสถานที่ (By Name) ระบบจะนำเวกเตอร์ของสถานที่ต้นทางมาเปรียบเทียบกับเวกเตอร์ของสถานที่ท่องเที่ยวรายการอื่น และไม่นำสถานที่ต้นทางมาแสดงซ้ำในผลการแนะนำ

สำหรับการทดสอบด้วยข้อความ (By Text) ระบบจะนำข้อความที่ผู้ใช้ระบุมาแปลงเป็นเวกเตอร์ตามกระบวนการของแต่ละแบบจำลอง จากนั้นคำนวณค่าความคล้ายคลึงกับข้อมูลสถานที่ท่องเที่ยวทั้งหมด

ผลลัพธ์จากทั้งสองรูปแบบการทดสอบจะถูกจัดเรียงตามค่าความคล้ายคลึงจากมากไปน้อย และเลือกสถานที่ตามจำนวนผลลัพธ์ที่กำหนดเพื่อนำเสนอเป็นคำแนะนำ

3.6 การออกแบบการทดสอบและเปรียบเทียบ

การวิจัยครั้งนี้ดำเนินการเปรียบเทียบเทียบผลลัพธ์ของระบบแนะนำสถานที่ท่องเที่ยวที่สร้างด้วยเทคนิค TF-IDF, Thai2Vec, FastText และ Sentence-BERT โดยกำหนดรูปแบบการทดสอบออกเป็น 2 รูปแบบ ได้แก่ การแนะนำจากชื่อสถานที่ (By Name) และ การแนะนำจากข้อความ (By Text)

3.6.1 การประเมินผลจากชื่อสถานที่ (By Name)

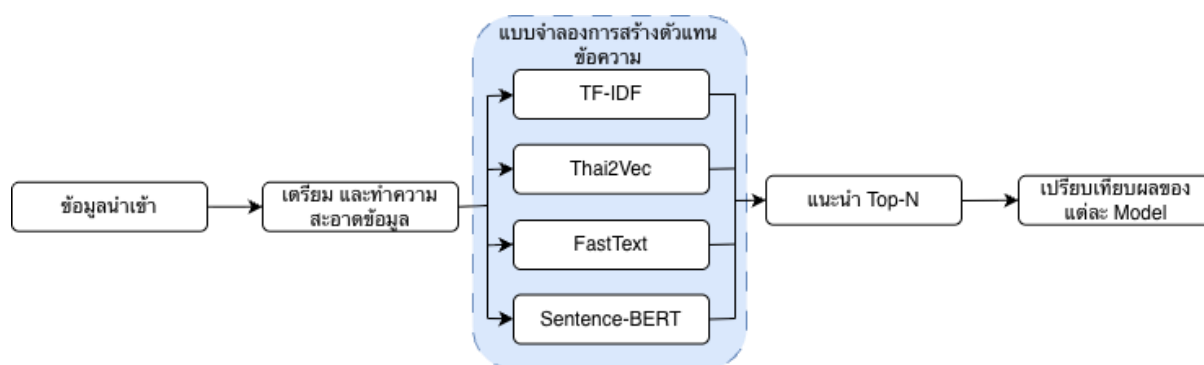
การประเมินผลรูปแบบนี้ใช้ “ชื่อสถานที่ท่องเที่ยว” เป็นข้อมูลนำเข้า ระบบจะดึงข้อมูลคุณลักษณะของสถานที่ดังกล่าวจากชุดข้อมูลมาเป็นสถานที่ต้นทาง (Source Location) เพื่อนำไปสร้างตัวแทนข้อมูลตามเทคนิคของแต่ละแบบจำลอง ได้แก่ TF-IDF, Thai2Vec, FastText และ Sentence-BERT จากนั้นระบบจะคำนวณค่าความคล้ายคลึงด้วยวิธี Cosine Similarity และจัดเรียงผลลัพธ์เพื่อแสดงสถานที่ที่คำแนะนำ

ตัวอย่างเช่น หากผู้ใช้ระบุชื่อสถานที่ท่องเที่ยว “อุทยานแห่งชาติดอยอินทนนท์” ระบบจะนำเวกเตอร์ของสถานที่ดังกล่าวไปเปรียบเทียบกับสถานที่อื่น เพื่อวิเคราะห์ความแตกต่างของรายการที่ได้รับการแนะนำและประสิทธิภาพของแต่ละแบบจำลอง ตามขั้นตอนที่แสดงในรูปที่ 14

3.6.2 การประเมินผลจากข้อความ (By Text)

การประเมินรูปแบบนี้ใช้ “ข้อความค้นหา (Text Query)” หรือคำสำคัญที่อธิบายลักษณะของสถานที่แทนการระบุชื่อสถานที่โดยตรง เช่น “สถานที่ท่องเที่ยวที่คล้ายกับอุทยานผาแต้ม หรือเขาปลาร้า มีภาพเขียนก่อนยุคประวัติศาสตร์” ระบบจะนำข้อความดังกล่าวเข้าสู่กระบวนการประมวลผลและสร้างตัวแทนข้อมูลโดยตรงผ่านแบบจำลองทั้ง 4 วิธี โดยไม่จำเป็นต้องดึงข้อมูลจากสถานที่ต้นทางก่อน ผลลัพธ์จะถูกจัดอันดับตามค่าความคล้ายคลึงเพื่อแนะนำสถานที่ที่ตรงกับความหมายของข้อความนำเข้ามากที่สุด ดังแสดงในขั้นตอนการทำงานของรูปที่ 14

การทดสอบรูปแบบนี้ช่วยประเมินความสามารถของแต่ละแบบจำลองในการทำความเข้าใจ “ความหมายเชิงบริบท” (Semantic Context) ของข้อความค้นหา ซึ่งเป็นการทดสอบในเชิงลึกมากกว่าการระบุเพียงชื่อสถานที่



รูปที่ 14 กระบวนการทำงานของระบบ และขั้นตอนการเปรียบเทียบประสิทธิภาพแบบจำลอง

3.6.3 การเปรียบเทียบผลลัพธ์ของแบบจำลอง

ผลการแนะนำจากทั้ง 2 รูปแบบการทดสอบ ได้แก่ By Name และ By Text จะถูกนำมาเปรียบเทียบระหว่างแบบจำลองจำนวน 4 วิธี ได้แก่ TF-IDF, Thai2Vec, FastText และ Sentence-BERT การเปรียบเทียบผลลัพธ์พิจารณาจากข้อมูลสำคัญ ได้แก่

- 1) สถานที่ที่ได้รับการแนะนำ (Recommended Location)

พิจารณาความแตกต่างของรายการสถานที่ที่ได้รับการแนะนำจากแต่ละแบบจำลอง

- 2) ลำดับของคำแนะนำ (Ranking)

เปรียบเทียบลำดับของสถานที่ท่องเที่ยวที่ปรากฏในผลลัพธ์ของแต่ละแบบจำลอง

- 3) ค่าความคล้ายคลึง (Similarity Score)

เปรียบเทียบค่าความคล้ายคลึงที่ได้จากการคำนวณของแต่ละแบบจำลอง

- 4) ระยะเวลาของการสร้างแบบจำลอง (Fit Time)

เปรียบเทียบระยะเวลาที่ใช้ในการสร้างหรือเตรียมแบบจำลองของแต่ละวิธี โดยวัดระยะเวลาเป็นหน่วยวินาที เพื่อใช้ประกอบการพิจารณาประสิทธิภาพด้านระยะเวลาในการประมวลผล

ผลการเปรียบเทียบจะนำเสนอในรูปแบบตาราง โดยสามารถแยกผลการทดลองออกเป็น 2 ส่วน คือ ผลการแนะนำจากชื่อสถานที่ และ ผลการแนะนำจากข้อความ

จากนั้นจะพิจารณาความสอดคล้องของผลลัพธ์กับข้อมูลนำเข้า โดยพิจารณาว่าสถานที่ที่ระบบแนะนำมีลักษณะหรือเนื้อหาที่สอดคล้องกับข้อมูลนำเข้าหรือไม่ โดยในกรณีการทดสอบแบบ By Name จะพิจารณาความสอดคล้องกับลักษณะของสถานที่ต้นทาง ส่วนการทดสอบแบบ By Text จะพิจารณาความสอดคล้องกับข้อความหรือคำสำคัญที่ผู้ใช้ระบุ

3.7 สรุปผลการดำเนินงาน

การดำเนินการวิจัยในบทนี้เริ่มต้นจากการนำชุดข้อมูลแหล่งท่องเที่ยวมาใช้ในการสร้างระบบแนะนำ จากนั้นดำเนินการสำรวจและเตรียมข้อมูล โดยมีการคัดเลือกตัวแปรที่เกี่ยวข้อง การปรับชื่อฟิลด์ การคัดกรองข้อมูล การทำความสะอาดข้อความ การตัดคำภาษาไทย และการกำจัดคำหยุดตามความเหมาะสมของแบบจำลอง

หลังจากนั้น ข้อมูลที่ผ่านการเตรียมจะถูกนำไปสร้างตัวแทนข้อความด้วยเทคนิค TF-IDF, Thai2Vec, FastText และ Sentence-BERT จากนั้นจึงคำนวณความคล้ายคลึงระหว่างสถานที่ท่องเที่ยวด้วยวิธี Cosine Similarity และจัดอันดับผลลัพธ์เพื่อสร้างคำแนะนำ

สุดท้าย ผลลัพธ์จากแบบจำลองทั้ง 4 วิธีจะถูกนำมาเปรียบเทียบ โดยพิจารณารายการสถานที่ที่ได้รับการแนะนำ ลำดับของผลลัพธ์ และค่าความคล้ายคลึง เพื่อวิเคราะห์ความแตกต่างของการสร้างคำแนะนำจากแต่ละแบบจำลอง

บทที่ 4

ผลการดำเนินการวิจัย

ในการวิจัยครั้งนี้ ผู้วิจัยได้ศึกษา และพัฒนาระบบแนะนำสถานที่ท่องเที่ยวภายในประเทศไทย โดยประยุกต์ใช้เทคนิคการกรองแบบอิงเนื้อหา (Content-Based Filtering) เพื่อเปรียบเทียบผลการแนะนำจากแบบจำลองจำนวน 4 วิธี ได้แก่ TF-IDF, Thai2Vec, FastText และ Sentence-BERT

ผลการดำเนินการวิจัยแบ่งออกเป็น 2 รูปแบบ ได้แก่

1. ผลการสร้างแบบจำลอง และการทดสอบด้วยชื่อสถานที่ (By Name)
2. ผลการสร้างแบบจำลอง และการทดสอบด้วยข้อความ (By Text)

4.1 ผลการสร้างแบบจำลอง และการทดสอบด้วยชื่อสถานที่ (By Name)

การทดสอบในรูปแบบ By Name เป็นการนำชื่อของสถานที่ท่องเที่ยวที่มีอยู่ในชุดข้อมูลมาใช้เป็นข้อมูลนำเข้า โดยระบบจะค้นหารายการสถานที่ท่องเที่ยวอื่นที่มีเนื้อหา หรือรายละเอียดใกล้เคียงกับสถานที่ต้นทาง จากนั้นจึงจัดอันดับผลการแนะนำจากค่าความคล้ายคลึงจากมากไปน้อย

ในการทดลองครั้งนี้ ผู้วิจัยได้เปรียบเทียบผลการแนะนำจากแบบจำลองจำนวน 4 วิธี ได้แก่ TF-IDF, Thai2Vec, FastText และ Sentence-BERT โดยแสดงผลการแนะนำสถานที่ท่องเที่ยวจำนวน 5 อันดับแรก พร้อมทั้งแสดงค่าความคล้ายคลึงของแต่ละรายการ

4.1.1 ผลการทดสอบด้วยชื่อสถานที่ (By Name)

ในการทดสอบด้วยชื่อสถานที่ ผู้วิจัยได้นำชื่อสถานที่ท่องเที่ยวที่มีอยู่ในชุดข้อมูลมาใช้เป็นข้อมูลนำเข้า โดยระบบจะค้นหาสถานที่ที่มีความเกี่ยวข้องหรือมีลักษณะใกล้เคียงกับสถานที่ต้นทาง และแสดงผลลัพธ์จำนวน 5 อันดับแรกตามค่าความคล้ายคลึง

ตารางที่ 9 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “อุทยานแห่งชาติดอยอินทนนท์”

TF-IDF		
อันดับ	ชื่อสถานที่	Score

1	อุทยานแห่งชาติแม่ว่าง	0.2542
2	ดอยหลวงเชียงดาว	0.2363
3	ยอดดอยอินทนนท์	0.2262
4	อุทยานแห่งชาติออบหลวง	0.2195
5	น้ำตกแม่กลาง	0.1813
Thai2Vec		
อันดับ	ชื่อสถานที่	Score
1	ดอยหลวงเชียงดาว	0.8812
2	อุทยานแห่งชาติดอยสุเทพ-ปุย	0.8716
3	อุทยานแห่งชาติทุ่งแสลงหลวง (หนองแม่นา)	0.8617
4	อุทยานแห่งชาติถ้ำหลวง-ขุนน้ำนางนอน (เตรียมการ)	0.8557
5	อุทยานแห่งชาติผาแดง	0.8469
FastText		
อันดับ	ชื่อสถานที่	Score
1	ดอยหลวงเชียงดาว	0.9286
2	อุทยานแห่งชาติแม่ว่าง	0.9222
3	อุทยานแห่งชาติถ้ำหลวง-ขุนน้ำนางนอน (เตรียมการ)	0.9108
4	อุทยานแห่งชาติผาแดง	0.8995
5	อุทยานแห่งชาติออบหลวง	0.8983
Sentence-BERT		

อันดับ	ชื่อสถานที่	Score
1	ดอยหลวงเชียงดาว	0.8758
2	อุทยานแห่งชาติห้วยน้ำดัง	0.8504
3	อ่างกาหลวง	0.8501
4	อุทยานแห่งชาติถ้ำหลวง-ขุนน้ำนางนอน (เตรียมการ)	0.8477
5	อุทยานแห่งชาติแม่ว้าง	0.8466

Thai2Vec, FastText และ Sentence-BERT ต่างแนะนำ “ดอยหลวงเชียงดาว” เป็นอันดับ 1 ตรงกันทั้งสามวิธี ซึ่งเป็นอุทยานแห่งชาติบนพื้นที่ภูเขาสูงในภาคเหนือเช่นเดียวกับดอยอินทนนท์ สะท้อนว่าทั้งสามเทคนิคจับความสัมพันธ์เชิงความหมายระหว่างสถานที่ประเภทอุทยานแห่งชาติบนยอดดอยได้สอดคล้องกัน ขณะที่ TF-IDF แนะนำ “อุทยานแห่งชาติแม่ว้าง” ซึ่งมีคำว่า “อุทยานแห่งชาติ” ปรากฏร่วมกันโดยตรง แสดงว่าแม้หลักการต่างกัน แต่ผลลัพธ์ของทุกวิธียังคงอยู่ในกลุ่มประเภทอุทยานแห่งชาติภาคเหนือซึ่งเกี่ยวข้องกับข้อมูลนำเข้าอย่างสมเหตุสมผล

ตารางที่ 10 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “เขตห้ามล่าสัตว์ป่าถ้ำผาน้ำทิพย์”

TF-IDF		
อันดับ	ชื่อสถานที่	Score
1	ผาหมอกมียาย	0.2964
2	อุทยานแห่งชาติป่าหินงาม	0.1999
3	อุทยานแห่งชาติเขาพนมเบญจา	0.1743
4	อุทยานแห่งชาติเขาหลัก-ลำรู่	0.1736
5	อุทยานแห่งชาติน้ำหนาว	0.1725

Thai2Vec		
อันดับ	ชื่อสถานที่	Score
1	เขตห้ามล่าสัตว์ป่าอ่าวคุ้งกระเบน	0.8862
2	วนอุทยานปราณบุรี	0.8757
3	ผาสามเกลอ	0.8751
4	อุทยานแห่งชาติทุ่งแสลงหลวง	0.8695
5	อุทยานแห่งชาติศรีสัชนาลัย	0.8689
FastText		
อันดับ	ชื่อสถานที่	Score
1	อุทยานแห่งชาติเขื่อนศรีนครินทร์	0.9349
2	ผามอกมียวาย	0.9320
3	ผาสามเกลอ	0.9236
4	อุทยานแห่งชาติป่าหินงาม	0.9201
5	อุทยานแห่งชาติภูผายล	0.9169
Sentence-BERT		
อันดับ	ชื่อสถานที่	Score
1	เขตรักษาพันธุ์สัตว์ป่าแม่ตื่น	0.8874
2	เขาพุร้อน	0.8782
3	ประตูโขลง	0.8660
4	วนอุทยานถ้ำเขาน้อย	0.8615

5	ถ้ำยงวี	0.8613
---	---------	--------

Thai2Vec แนะนำ “เขตห้ามล่าสัตว์ป่าอ่าวคุ้งกระเบน” ซึ่งมีคำว่า “เขตห้ามล่าสัตว์ป่า” ตรงกับสถานที่ต้นทางโดยตรง และ Sentence-BERT แนะนำ “เขตรักษาพันธุ์สัตว์ป่าแม่ตื่น” ซึ่งเป็นพื้นที่อนุรักษ์สัตว์ป่าประเภทเดียวกัน แสดงถึงความเข้าใจเชิงหมวดหมู่ ขณะที่ FastText แนะนำ “อุทยานแห่งชาติเขื่อนศรีนครินทร์” ซึ่งเป็นพื้นที่อนุรักษ์ธรรมชาติเช่นกันแต่ต่างประเภทย่อย ส่วน TF-IDF แนะนำ “ผาหมอกมียวาย” ซึ่งเกี่ยวข้องกับลักษณะภูมิประเทศแบบผาและจุดชมวิวมากกว่าหมวดหมู่การอนุรักษ์สัตว์ป่าโดยตรง กรณีนี้แสดงให้เห็นความไม่แน่นอนของความสัมพันธ์ระหว่างคำว่า “เขตห้ามล่าสัตว์ป่า” กับสถานที่ธรรมชาติประเภทอื่นในบางเทคนิค

ตารางที่ 11 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “เจดีย์พระศรีสุริโยทัย”

TF-IDF		
อันดับ	ชื่อสถานที่	Score
1	พระบรมราชานุสาวรีย์สมเด็จพระนเรศวรมหาราช	0.1973
2	พระราชานุสาวรีย์สมเด็จพระศรีสุริโยทัย	0.1656
3	วัดแก้วฟ้า	0.1535
4	พิพิธภัณฑสถานแห่งชาติเจ้าสามพระยา	0.1519
5	วัดพระธาตุเสด็จ	0.1518
Thai2Vec		
อันดับ	ชื่อสถานที่	Score
1	ศาลสมเด็จพระนเรศวรมหาราช	0.8776
2	วัดอัมพวันเจติยาราม	0.8768
3	วัดเขาวง ถ้ำนารายณ์	0.8711

4	วัดพระธาตุมหาชัย	0.8701
5	พระบรมธาตุเจดีย์กาญจนาภิเษก	0.8654
FastText		
อันดับ	ชื่อสถานที่	Score
1	วัดวิเชียรบำรุง	0.9107
2	วัดท่าสุทธาวาส	0.9103
3	วัดบางคาง	0.9083
4	วัดสวนตาล	0.9047
5	วัดไตรตรึงธาราม	0.9045
Sentence-BERT		
อันดับ	ชื่อสถานที่	Score
1	วัดเถรพลาย	0.9104
2	วัดพระธาตุจอมรุ่ง	0.9005
3	วัดศาลโคดม	0.8937
4	วัดต้นสน อ่างทอง	0.8913
5	วัดทิพย์สุคนธาราม	0.8892

TF-IDF และ Thai2Vec ต่างแนะนำสถานที่ที่เกี่ยวข้องกับ “สมเด็จพระนเรศวรมหาราช” ซึ่งเชื่อมโยงทางประวัติศาสตร์กับเจดีย์พระศรีสุริโยทัยในฐานะวีรกรรมสมัยเดียวกัน แสดงว่าทั้งสองวิธีจับความสัมพันธ์เชิงเนื้อหาทางประวัติศาสตร์ได้ ขณะที่ FastText และ Sentence-BERT แนะนำ “วัดวิเชียรบำรุง” และ “วัดเถรพลาย” ตามลำดับ ซึ่งเป็นวัดทั่วไปที่มีลักษณะทางสถาปัตยกรรมใกล้เคียงกันแต่ไม่ได้เชื่อมโยงกับบริบททางประวัติศาสตร์ของเจดีย์พระศรีสุริโยทัยโดยตรง กรณีนี้แสดงความแตกต่างระหว่างการจับคู่เชิงเนื้อหาทางประวัติศาสตร์กับการจับคู่เชิงลักษณะสถานที่ทางศาสนา

ตารางที่ 12 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “ตลาดน้ำคลองแห”

TF-IDF		
อันดับ	ชื่อสถานที่	Score
1	ตลาดน้ำท่าคา	0.2015
2	ตลาดน้ำดอนหวาย	0.1997
3	ตลาดคลองแล	0.1591
4	ตลาดน้ำคลองลัดมะยม	0.1541
5	ตลาดน้ำหลวงพ่อบึง	0.1534
Thai2Vec		
อันดับ	ชื่อสถานที่	Score
1	ตลาดน้ำบางคล้า	0.9041
2	ตลาดน้ำวัดบางกระเจ็ด	0.9016
3	ตลาดคลองสวน 100 ปีฉะเชิงเทรา	0.8984
4	ตลาดโบราณนครเนื่องเขต	0.8962
5	ตลาดร้อยปีปากพนัง	0.8917
FastText		
อันดับ	ชื่อสถานที่	Score
1	ตลาดน้ำบางคล้า	0.9282
2	ตลาดร้อยปีปากพนัง	0.9264

3	ตลาดคลองแล	0.9213
4	ตลาดน้ำท่าคา	0.9211
5	ตลาดน้ำวัดบางกระเจ็ด	0.9185
Sentence-BERT		
อันดับ	ชื่อสถานที่	Score
1	ตลาดหลวงแพ่ง 108 ปี	0.8639
2	ตลาดน้ำคลองลัดมะยม	0.8431
3	ตลาดน้ำคลองลัดพลี	0.8391
4	ตลาดน้ำหลวงพ่อบึง	0.8365
5	ตลาดริมยม 2437	0.8301

TF-IDF, Thai2Vec และ FastText ต่างแนะนำตลาดน้ำเป็นอันดับ 1 โดย TF-IDF ให้ “ตลาดน้ำท่าคา” ส่วน Thai2Vec และ FastText ให้ “ตลาดน้ำบางคล้า” ตรงกัน ซึ่งทั้งสามคำตอบยังคงอยู่ในหมวดหมู่ “ตลาดน้ำ” เช่นเดียวกับข้อมูลนำเข้า แสดงถึงความสัมพันธ์เชิงคำศัพท์และเชิงความหมายที่แข็งแกร่งร่วมกัน ขณะที่ Sentence-BERT แนะนำ “ตลาดหลวงแพ่ง 108 ปี” ซึ่งยังคงเป็นตลาดเช่นกันแต่เน้นมิติความเก่าแก่มากกว่าคำว่า “ตลาดน้ำ” โดยตรง กรณีนี้จึงสอดคล้องกับข้อมูลนำเข้าในทุกวิธี เพียงแต่ต่างระดับความใกล้เคียงของคำ

ตารางที่ 13 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “ถ้าใหญ่น้ำหนาว”

TF-IDF		
อันดับ	ชื่อสถานที่	Score
1	อุทยานแห่งชาติน้ำหนาว	0.2106
2	ถ้ำแก้วสุรภานต์	0.2056

3	เขาผาแรด	0.1917
4	ถ้ำเขาทะนนาน	0.1907
5	ถ้ำน้ำผาผางาม	0.1834
Thai2Vec		
อันดับ	ชื่อสถานที่	Score
1	ถ้ำหงส์	0.8876
2	ถ้ำน้ำเขาศิ่วะ	0.8841
3	ถ้ำธารลอดน้อย	0.8774
4	ถ้ำพุหวาย	0.8761
5	ถ้ำแก้วสุรกานต์	0.8761
FastText		
อันดับ	ชื่อสถานที่	Score
1	ถ้ำห้วยบ่อน	0.9378
2	ถ้ำเดือน ถ้ำดาว	0.9370
3	ถ้ำเมืองแพม	0.9359
4	ถ้ำหงส์	0.9317
5	ถ้ำเขาบิน	0.9315
Sentence-BERT		
อันดับ	ชื่อสถานที่	Score
1	ถ้ำห้วยบ่อน	0.8980

2	ถ้าพวย	0.8920
3	ถ้าหงส์	0.8773
4	ถ้าเดือน ถ้าดาว	0.8751
5	ถ้ารูนกสีก	0.8746

TF-IDF แนะนำ “อุทยานแห่งชาติน้ำหนาว” ซึ่งมีคำว่า “น้ำหนาว” ตรงกับข้อมูลนำเข้าในระดับคำ แต่เป็นอุทยานไม่ใช่ถ้า ขณะที่ Thai2Vec, FastText และ Sentence-BERT ต่างแนะนำสถานที่ประเภทถ้า เช่นเดียวกัน โดย FastText และ Sentence-BERT ให้ “ถ้าห้วยบ่อน” ตรงกัน และ Thai2Vec ให้ “ถ้าหงส์” แสดงว่าทั้งสามวิธีที่อาศัยการแทนความหมายจับหมวดหมู่ “ถ้า” ได้แม่นยำกว่า TF-IDF ที่ยึดติดกับคำว่า “น้ำหนาว” มากกว่าคำว่า “ถ้า”

ตารางที่ 14 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “นาเกลือบ้านแหลม”

TF-IDF		
อันดับ	ชื่อสถานที่	Score
1	บ้านคลองนาเกลือ	0.1283
2	นาเกลือ	0.1240
3	ชุมชนย่านเมืองเก่าริมน้ำเพชรบุรี	0.1161
4	น้ำตกทับลาน	0.1114
5	ชุมชนย่านเมืองเก่าริมน้ำเพชรบุรี (เช้า)	0.1082
Thai2Vec		
อันดับ	ชื่อสถานที่	Score
1	ชุมชนบ้านไม้รูด	0.8722

2	อุทยานแห่งชาติแจ้ซ้อน	0.8603
3	ท่องเที่ยวเกษตรเชิงอนุรักษ์ตำบลกมลา	0.8597
4	ลานค้าชุมชนห้วยเตี๋	0.8583
5	พันธุ์ตแสนไคร	0.8579
FastText		
อันดับ	ชื่อสถานที่	Score
1	บ่อเกลือสินเธาว์	0.8941
2	วิสาหกิจท่องเที่ยวโดยชุมชน บ้านสามช่องเหนือ	0.8565
3	บ้านคลองนาเกลือ	0.8539
4	นาเกลือ	0.8499
5	บ้านจำรุง	0.8480
Sentence-BERT		
อันดับ	ชื่อสถานที่	Score
1	ตลาดโรงเกลือ	0.8095
2	บ่อเกลือสินเธาว์	0.7519
3	นาเกลือ	0.7509
4	บ้านคลองนาเกลือ	0.7068
5	หนึ่งในสยาม เนินทรายงามที่ชุมพร	0.6839

TF-IDF แนะนำ “บ้านคลองนาเกลือ” ซึ่งมีคำว่า “นาเกลือ” ปรากฏร่วมกันโดยตรง จึงมีความสัมพันธ์เชิงคำศัพท์ที่ชัดเจนที่สุด ส่วน FastText แนะนำ “บ่อเกลือสินเธาว์” และ Sentence-BERT แนะนำ “ตลาดโรงเกลือ” ซึ่งยังคงอยู่ในกลุ่มความหมายเกี่ยวกับ “เกลือ” แม้ไม่ใช่คำว่า “นาเกลือ” โดยตรง แสดงถึง

ความสามารถในการจับความสัมพันธ์เชิงความหมายของคำที่เกี่ยวข้องกัน ขณะที่ Thai2Vec แนะนำ “ชุมชนบ้านไม้รูด” ซึ่งไม่มีความเชื่อมโยงกับคำว่า “เกลือ” อย่างชัดเจน อาจเกิดจากข้อจำกัดของ static word embedding ที่ไม่สามารถจับคำเฉพาะที่พบไม่บ่อยในคลังคำได้ดีเท่า FastText

ตารางที่ 15 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “พระธาตุมรุกขนคร”

TF-IDF		
อันดับ	ชื่อสถานที่	Score
1	วัดพระธาตุท่าอุเทน	0.2827
2	หาดแห่ (บ้านน้ำก่ำ)	0.2395
3	พระธาตุ ก.ว. วัดดอนนางหงส์	0.2108
4	วัดดงลาน	0.2018
5	แหล่งเรียนรู้ร้อยห้าไดโนเสาร์ท่าอุเทน	0.1786
Thai2Vec		
อันดับ	ชื่อสถานที่	Score
1	วัดพระธาตุท่าอุเทน	0.9129
2	วัดนารายณ์การาม	0.8822
3	พระธาตุเจดีย์ทะเมนชัย	0.8813
4	วัดอนาลโยทิพยาราม	0.8709
5	วัดพระธาตุจอมแจ้ง	0.8696
FastText		
อันดับ	ชื่อสถานที่	Score

1	วัดพระธาตุท่าอุเทน	0.9210
2	พระธาตุพนมจำลอง	0.8876
3	วัดเข้างัฏฐาคาร	0.8819
4	วัดถ้ำขัวญเมือง	0.8807
5	พระธาตุ ก.ว. วัดดอนนางหงส์	0.8788
Sentence-BERT		
อันดับ	ชื่อสถานที่	Score
1	วัดพระธาตุจอมรุ่ง	0.8813
2	วัดสุทธิกาวาส	0.8734
3	วัดพระธาตุถิ่นแกนหลวง	0.8672
4	วัดพระธาตุคุดยคำ	0.8636
5	ค่ายเจ้าเมืองระนอง	0.8589

TF-IDF, Thai2Vec และ FastText ต่างแนะนำ “วัดพระธาตุท่าอุเทน” เป็นอันดับ 1 ตรงกันทั้งสามวิธี ซึ่งเป็นวัดที่มีพระธาตุเช่นเดียวกับพระธาตุมรุกขนคร แสดงถึงความสัมพันธ์ที่แข็งแกร่งทั้งในระดับคำศัพท์และระดับความหมาย ขณะที่ Sentence-BERT แนะนำ “วัดพระธาตุจอมรุ่ง” ซึ่งยังคงเป็นวัดพระธาตุเช่นกัน เพียงแต่เป็นคนละแห่งกับที่แบบจำลองอื่นเลือก กรณีนี้จึงเป็นตัวอย่างที่ชัดเจนที่สุดกรณีหนึ่งที่ทุกวิธีให้ผลลัพธ์สอดคล้องกับข้อมูลนำเข้าในหมวดหมู่เดียวกัน

ตารางที่ 16 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “วัดพระแก้ว”

TF-IDF		
อันดับ	ชื่อสถานที่	Score

1	วัดราชโอรสาราม ราชวรวิหาร	0.1790
2	เจดีย์หลวงปู่ขาว	0.1680
3	วัดท่าอุเทน	0.1679
4	พระธาตุภูแล	0.1667
5	วัดแม่อาวหลวง	0.1663
Thai2Vec		
อันดับ	ชื่อสถานที่	Score
1	วัดกลาง	0.9072
2	วัดบางหลวง	0.9035
3	วัดไถ่เตี้ย	0.9025
4	วัดมหาธาตุ	0.8942
5	วัดอรัญญิก	0.8920
FastText		
อันดับ	ชื่อสถานที่	Score
1	วัดมหาธาตุ	0.9453
2	วัดอรัญญิก	0.9400
3	วัดแม่นางปลื้ม	0.9268
4	วัดไตรตรึงษาราม	0.9263
5	พระธาตุดวงเดียว	0.9215
Sentence-BERT		

อันดับ	ชื่อสถานที่	Score
1	วัดม่วยต่อ	0.9034
2	วัดอโยธยา	0.9018
3	วัดมหาธาตุ	0.8986
4	วัดมณีชลขันธ์	0.8963
5	วัดพระประโทนเจดีย์วรวิหาร	0.8795

ผลลัพธ์อันดับ 1 ของทั้งสี่วิธีล้วนเป็นวัดเช่นเดียวกับวัดพระแก้ว โดย TF-IDF แนะนำ “วัดราชโอรสาราม ราชวรวิหาร” Thai2Vec แนะนำ “วัดกลาง” FastText แนะนำ “วัดมหาธาตุ” และ Sentence-BERT แนะนำ “วัดม่วยต่อ” ซึ่งทุกคำตอบมีคำว่า “วัด” ตรงกับข้อมูลนำเข้าโดยตรง แสดงถึงความสามารถในการจับหมวดหมู่ “วัด” ได้ดีทั้งสี่วิธี อย่างไรก็ตาม TF-IDF ให้คะแนนเพียง 0.1790 ขณะที่อีกสามวิธีให้คะแนนสูงถึง 0.90 ขึ้นไป ทั้งที่คำตอบมีความสอดคล้องเชิงหมวดหมู่เท่ากัน สะท้อนว่าความแตกต่างของคะแนนเกิดจากลักษณะพื้นที่คุณลักษณะ (feature space) ของแต่ละวิธี มากกว่าจะบ่งชี้ความเกี่ยวข้องของคำตอบ

ตารางที่ 17 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “ศูนย์ศึกษาธรรมชาติและสัตว์ป่าเขาพระแหว”

TF-IDF		
อันดับ	ชื่อสถานที่	Score
1	สถานีพัฒนาและส่งเสริมการอนุรักษ์สัตว์ป่าเขาพระแหว	0.2301
2	ดอยหัวหมู	0.1559
3	น้ำตกสายใจ	0.1519
4	น้ำตกโตนปริวรรต	0.1476
5	น้ำตกวังน้ำว้าง	0.1471

Thai2Vec		
อันดับ	ชื่อสถานที่	Score
1	น้ำตกบางแป	0.8854
2	อ่าวนุ่น	0.8795
3	อุทยานแห่งชาติดอยผาหม่นปก	0.8781
4	จุดชมวิวิทิวทัศน์เขาพังเหย	0.8772
5	สถานีพัฒนาและส่งเสริมการอนุรักษ์สัตว์ป่าเขาพระแทว	0.8757
FastText		
อันดับ	ชื่อสถานที่	Score
1	แก่งสนสามพันปี	0.9200
2	ดอยหัวหมู	0.9181
3	น้ำตกภูละออ	0.9171
4	น้ำตกบางแป	0.9147
5	สวนรุกขชาติไพศาลี	0.9138
Sentence-BERT		
อันดับ	ชื่อสถานที่	Score
1	สถานีพัฒนาและส่งเสริมการอนุรักษ์สัตว์ป่าเขาพระแทว	0.8698
2	น้ำตกบริพัตร	0.8509
3	น้ำตกแก้งแม่ปอง	0.8487
4	น้ำตกแม่ระเมิง	0.8472

5	อุทยานแห่งชาติน้ำตกพาเจริญ	0.8437
---	----------------------------	--------

TF-IDF และ Sentence-BERT ต่างแนะนำ “สถานีพัฒนาและส่งเสริมการอนุรักษ์สัตว์ป่าเขาพระแทว” ตรงกัน ซึ่งเป็นหน่วยงานอนุรักษ์สัตว์ป่าในพื้นที่เดียวกับข้อมูลนำเข้า สะท้อนว่าทั้งสองวิธีจับความหมายเชิง “หน่วยงาน/ภารกิจอนุรักษ์” ได้ตรงประเด็น ขณะที่ Thai2Vec และ FastText แนะนำ “น้ำตกบางแป” และ “แก่งสนสามพันปี” ตามลำดับ ซึ่งเป็นแหล่งท่องเที่ยวทางธรรมชาติเช่นกันแต่เน้นลักษณะภูมิประเทศมากกว่าหน่วยงานอนุรักษ์ กรณีนี้ชี้ให้เห็นว่า word embedding อาจให้น้ำหนักกับคำอธิบายเชิงธรรมชาติมากกว่าคำที่ระบุประเภทหน่วยงาน

ตารางที่ 18 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “อุโมงค์ปิยะมิตร”

TF-IDF		
อันดับ	ชื่อสถานที่	Score
1	หมู่บ้านจุฬาภรณ์พัฒนา 9	0.1885
2	หมู่บ้านจุฬาภรณ์พัฒนา 10	0.1769
3	วิสาหกิจชุมชนการท่องเที่ยวโดยชุมชนบ้านจุฬาภรณ์พัฒนา 12	0.1344
4	สวนไม้ดอกเมืองหนาวเบตง	0.1274
5	บ่อน้ำร้อนเบตง	0.1106
Thai2Vec		
อันดับ	ชื่อสถานที่	Score
1	เขาค้อ	0.8836
2	เขตรักษาพันธุ์สัตว์ป่าทุ่งใหญ่นเรศวร-ห้วยขาแข้ง	0.8733
3	บ้านลุงโฮจิมินห์ (บ้านนาจอก)	0.8718

4	ถนนคนเดินตรอกโรงยา	0.8707
5	ชุมชนบ้านไม้รูด	0.8684
FastText		
อันดับ	ชื่อสถานที่	Score
1	หมู่บ้านจุฬาภรณ์พัฒนา 9	0.9021
2	เขาค้อ	0.8886
3	เขานาบน้ำ	0.8877
4	หมู่บ้านจุฬาภรณ์พัฒนา 10	0.8826
5	อุโมงค์เบตงมงคลฤทธิ์	0.8821
Sentence-BERT		
อันดับ	ชื่อสถานที่	Score
1	อุโมงค์เบตงมงคลฤทธิ์	0.8336
2	แหล่งหินตัด	0.8014
3	วัดถ้ำขวัญเมือง	0.7895
4	ถ้ำผาตั้ง	0.7877
5	วัดถ้ำตับเตา	0.7876

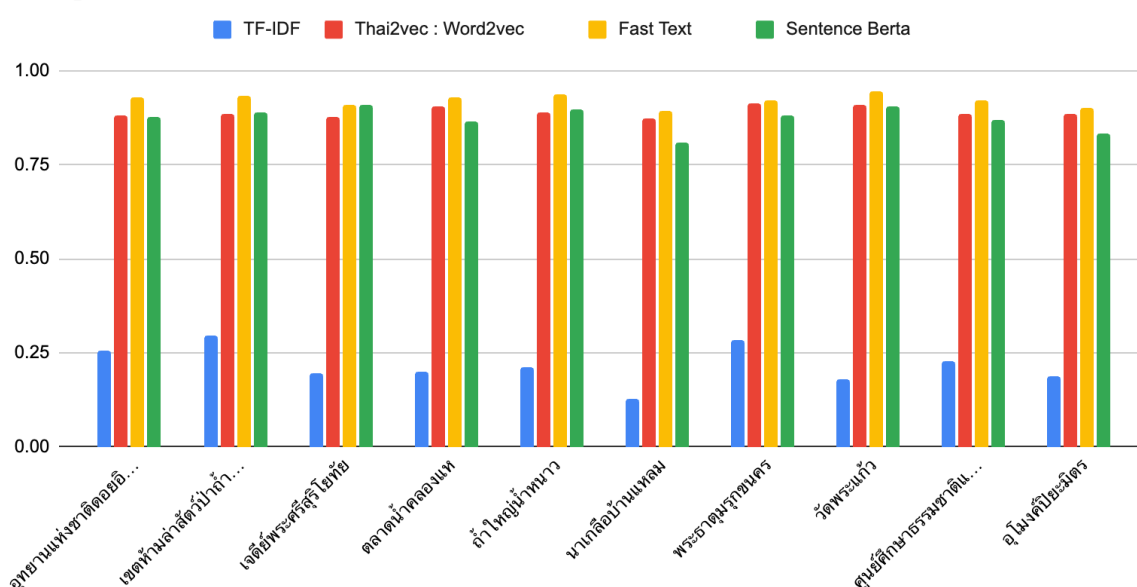
TF-IDF และ FastText ต่างแนะนำ “หมู่บ้านจุฬาภรณ์พัฒนา 9” ตรงกัน ซึ่งเป็นหมู่บ้านในพื้นที่ประวัติศาสตร์คอมมิวนิสต์เดียวกับอุโมงค์ปิยะมิตร จึงมีความสัมพันธ์เชิงบริบททางประวัติศาสตร์ร่วมกัน ขณะที่ Sentence-BERT แนะนำ “อุโมงค์เบตงมงคลฤทธิ์” ซึ่งเป็นอุโมงค์เช่นเดียวกับข้อมูลนำเข้าโดยตรง แสดงถึงความสามารถในการจับความหมายระดับประเภทสถานที่ได้แม่นยำ ส่วน Thai2Vec แนะนำ “เขาค้อ” ซึ่งอยู่ในพื้นที่ใกล้เคียงและมีความเกี่ยวข้องทางประวัติศาสตร์เช่นกัน แต่ไม่ตรงกับประเภทสถานที่ “อุโมงค์” โดยตรง กรณี

นี้แสดงความสัมพันธ์ระหว่างคำตอบกับข้อมูลนำเข้าเกิดได้จากทั้งมิติประเภทสถานที่และมิติบริบททางประวัติศาสตร์

4.1.2 ผลการเปรียบเทียบคะแนนความคล้ายคลึงโคไซน์ของการทดสอบด้วยชื่อสถานที่

เพื่อแสดงกระจายตัวของค่าคะแนนความคล้ายคลึงโคไซน์ของกรณีศึกษาต่าง ๆ ผู้วิจัยได้กำหนดชุดคำค้นหาทดสอบชื่อสถานที่เฉพาะจำนวน 10 คำค้นหา ไปประมวลผลผ่านฟังก์ชันการแนะนำของแบบจำลองทั้ง 4 วิธี ได้แก่ TF-IDF, Thai2Vec, FastText และ Sentence-BERT ภายใต้เงื่อนไขเดียวกัน โดยพิจารณาค่าความคล้ายคลึงโคไซน์ของผลลัพธ์อันดับที่ 1 (Top-1) ของแต่ละคำค้นหา ผลการทดลองแสดง ดังรูปที่ 15

แผนภูมิแสดงค่าความคล้ายคลึงโคไซน์ Top-1 จากการค้นหาด้วยชื่อสถานที่



รูปที่ 15 แผนภูมิแสดงค่าความคล้ายคลึงโคไซน์จากการชุดคำค้นหาด้วยชื่อสถานที่

จากรูปที่ 15 พบว่า FastText มีค่าความคล้ายคลึงโคไซน์ของผลลัพธ์อันดับที่ 1 อยู่สูงที่สุดที่สุดโดยรวม เมื่อเปรียบเทียบกับรูปแบบการจำลองแบบอื่น โดยมีค่าอยู่ในระดับประมาณ 0.89 - 0.95 ในชุดการทดสอบทั้ง 10 กรณี ขณะที่ Thai2Vec และ Sentence-BERT มีค่าความคล้ายคลึงอยู่ในระดับใกล้เคียงกัน แต่มีการเปลี่ยนแปลงของคะแนนระหว่างคำค้นหาแตกต่างกัน ส่วน TF-IDF มีค่าความคล้ายคลึงต่ำกว่าแบบจำลองที่ใช้การแทนข้อความด้วยเวกเตอร์เชิงความหมายอย่างชัดเจน โดยมีค่าประมาณ 0.12-0.30

เมื่อพิจารณาความแตกต่างของผลลัพธ์ในแต่ละคำค้นหา พบว่า FastText ให้คะแนน Top-1 สูงกว่าแบบจำลองอื่นในหลายกรณี สะท้อนว่าเวกเตอร์ที่สร้างจาก FastText สามารถจับความสัมพันธ์ของคำในชื่อสถานที่และข้อมูลที่เกี่ยวข้องได้ดีในชุดข้อมูลที่นำมาทดสอบ อย่างไรก็ตาม ในบางคำค้นหา เช่น กรณีที่มีสถานที่หรือบริบทเฉพาะ Sentence-BERT หรือ Thai2Vec อาจให้ผลลัพธ์ที่มีคะแนนใกล้เคียงหรือแตกต่างจาก FastText ดังนั้น การพิจารณาประสิทธิภาพของระบบจึงไม่ควรพิจารณาจากคะแนน Cosine Similarity เพียงอย่างเดียว แต่ควรพิจารณาร่วมกับรายการสถานที่ที่ได้รับการแนะนำ ลำดับของผลการแนะนำ และความเหมาะสมของสถานที่ที่แนะนำด้วย

เพื่อสรุปแนวโน้มของคะแนนความคล้ายคลึงโคไซน์ ผู้วิจัยได้นำคะแนน Top-1 จากชุดคำค้นหาด้วยชื่อสถานที่ทั้ง 10 กรณี มาคำนวณค่าเฉลี่ยและช่วงคะแนนของแต่ละแบบจำลอง ดังแสดงในตารางที่ 19

ตารางที่ 19 ค่าเฉลี่ย Top-1 Cosine Similarity จากการค้นหาโดยชื่อสถานที่ (N=10)

แบบจำลอง	Mean Top-1 Score	ช่วง Top-1 Score
TF-IDF	0.2168	0.1283 - 0.2964
Thai2Vec	0.8898	0.8722 - 0.9129
FastText	0.9223	0.8941 - 0.9453
Sentence-BERT	0.8733	0.8095 - 0.9104

จากตารางที่ 19 พบว่า FastText มีค่าเฉลี่ย Top-1 Cosine Similarity สูงที่สุด เท่ากับ 0.9223 รองลงมาคือ Thai2Vec เท่ากับ 0.8898, Sentence-BERT เท่ากับ 0.8733 และ TF-IDF เท่ากับ 0.2205 ตามลำดับ

เมื่อพิจารณาช่วงคะแนน พบว่า FastText มีช่วงคะแนน Top-1 อยู่ระหว่าง 0.8941–0.9453 ซึ่งอยู่ในระดับสูงและมีคะแนนสูงอย่างสม่ำเสมอในทั้ง 10 กรณี ขณะที่ Thai2Vec มีช่วงคะแนน 0.8722–0.9129 และ Sentence-BERT มีช่วงคะแนน 0.8095–0.9104 ส่วน TF-IDF มีช่วงคะแนน 0.1283–0.2964 ซึ่งต่ำกว่าแบบจำลองอื่นอย่างชัดเจน

ดังนั้น ผลการทดลองในกรณี By Name แสดงให้เห็นว่า หากพิจารณาจากค่าเฉลี่ยของ Top-1 Cosine Similarity แล้ว FastText ให้ค่าความคล้ายคลึงสูงสุด รองลงมาคือ Thai2Vec, Sentence-BERT และ TF-IDF ตามลำดับ อย่างไรก็ตาม ค่า Cosine Similarity ของแต่ละแบบจำลองไม่ควรนำมาเปรียบเทียบในเชิง “คุณภาพของความคล้ายคลึง” โดยตรงเพียงอย่างเดียว เนื่องจากแต่ละแบบจำลองสร้างตัวแทนข้อความและมีลักษณะของพื้นที่เวกเตอร์แตกต่างกัน การประเมินผลโดยรวมจึงควรพิจารณาร่วมกับ ความถูกต้องของรายการสถานที่ที่แนะนำ ลำดับของผลการแนะนำ และความเหมาะสมของสถานที่ที่ได้รับการแนะนำด้วย

4.1.3 ผลการเปรียบเทียบระยะเวลาในการสร้างแบบจำลอง

ผู้วิจัยได้นำข้อมูลสถานที่ท่องเที่ยวที่ผ่านกระบวนการเตรียมข้อมูลจำนวน 7,478 รายการ มาสร้างแบบจำลองระบบแนะนำด้วยแบบจำลองทั้ง 4 วิธี ได้แก่ TF-IDF, Thai2Vec, FastText และ Sentence-BERT โดยแต่ละแบบจำลองใช้วิธีการสร้างตัวแทนข้อมูลข้อความ และกระบวนการประมวลผลที่แตกต่างกัน ส่งผลให้ระยะเวลาในการสร้างแบบจำลองที่ต่างกัน

ตารางที่ 20 ผลการเปรียบเทียบระยะเวลาในการสร้างแบบจำลองจากการทดสอบด้วยชื่อสถานที่ (N = 10)

แบบจำลอง	ช่วงระยะเวลาในการสร้างแบบจำลอง	ค่าเฉลี่ยเวลาในการสร้างแบบจำลอง
TF-IDF	3.36s - 4.98s	3.677s
Thai2Vec	3.73s - 4.36s	3.973s
FastText	126.03s - 136.75s	130.157s
Sentence-BERT	24.9s - 41.49s	28.338s

จากตารางที่ 20 พบว่า TF-IDF ใช้ระยะเวลาในการสร้างแบบจำลองเฉลี่ยน้อยที่สุด เท่ากับ 3.677 วินาที รองลงมาคือ Thai2Vec ซึ่งใช้เวลาเฉลี่ย 3.973 วินาที ขณะที่ Sentence-BERT ใช้เวลาเฉลี่ย 28.338 วินาที และ FastText ใช้ระยะเวลาในการสร้างแบบจำลองเฉลี่ยมากที่สุด เท่ากับ 130.157 วินาที

เมื่อพิจารณาช่วงระยะเวลาในการสร้างแบบจำลอง พบว่า TF-IDF มีระยะเวลาอยู่ระหว่าง 3.36 - 4.98 วินาที และ Thai2Vec อยู่ระหว่าง 3.73 - 4.36 วินาที ซึ่งมีระยะเวลาใกล้เคียงกันและใช้เวลาในการ

ประมวลผลค่อนข้างน้อยเมื่อเปรียบเทียบกับแบบจำลองอื่น ส่วน Sentence-BERT มีระยะเวลาอยู่ระหว่าง 24.90 - 41.49 วินาที ขณะที่ FastText มีระยะเวลาอยู่ระหว่าง 126.03 - 136.75 วินาที ซึ่งสูงกว่าแบบจำลองอื่นอย่างชัดเจน

ทั้งนี้ ความแตกต่างของระยะเวลาในการสร้างแบบจำลองอาจเกิดจาก ลักษณะของกระบวนการประมวลผลและการสร้างตัวแทนข้อมูลข้อความของแต่ละแบบจำลอง โดย TF-IDF อาศัยการคำนวณความถี่และน้ำหนักของคำในเอกสาร ขณะที่แบบจำลองที่ใช้เวกเตอร์คำหรือเวกเตอร์ข้อความมีขั้นตอนการประมวลผลที่ซับซ้อนมากกว่า ดังนั้น ผลการทดลองด้านระยะเวลาจึงสะท้อนให้เห็นถึงความแตกต่างด้านต้นทุนการประมวลผลของแต่ละแบบจำลอง

เมื่อพิจารณาพร้อมกับผลการทดลองด้าน Top-1 Cosine Similarity ในหัวข้อก่อนหน้านี้ พบว่า FastText มีค่าเฉลี่ยความคล้ายคลึงสูงสุด แต่ใช้เวลาในการสร้างแบบจำลองมากที่สุด ขณะที่ TF-IDF ใช้เวลาในการสร้างแบบจำลองน้อยที่สุด แต่มีค่าเฉลี่ย Top-1 Cosine Similarity ต่ำที่สุด ดังนั้น การเลือกแบบจำลองสำหรับระบบแนะนำจึงควรพิจารณาทั้งด้านประสิทธิภาพของผลการแนะนำและระยะเวลาในการประมวลผล ไม่ควรพิจารณาจากค่าความคล้ายคลึงเพียงด้านเดียว

4.2 ผลการสร้างแบบจำลอง และการทดสอบด้วยข้อความ (By Text)

การทดสอบแบบ By Text เป็นการนำข้อความภาษาไทยที่อธิบายลักษณะของสถานที่หรือกิจกรรมที่ผู้ใช้ต้องการมาใช้เป็นข้อมูลนำเข้า จากนั้นระบบจะนำข้อความดังกล่าวไปเปรียบเทียบกับรายละเอียดของสถานที่ท่องเที่ยวในชุดข้อมูล และคำนวณค่าความคล้ายคลึงด้วย Cosine Similarity ก่อนจัดอันดับผลการแนะนำจำนวน 5 อันดับแรก

การทดสอบในรูปแบบนี้ใช้ข้อความค้นหาจำนวน 10 ข้อความ เพื่อเปรียบเทียบความสามารถของ TF-IDF, Thai2Vec, FastText และ Sentence-BERT ในการค้นหาสถานที่ที่มีเนื้อหาสอดคล้องกับความต้องการที่ระบุในข้อความ

4.2.1 ผลการทดสอบด้วยข้อความ (By Text)

ในการทดสอบด้วยข้อความ ผู้วิจัยได้นำข้อความภาษาไทยที่อธิบายลักษณะของสถานที่ท่องเที่ยวมาใช้เป็นข้อมูลนำเข้า และให้แต่ละแบบจำลองค้นหาสถานที่ที่มีเนื้อหาหรือรายละเอียดใกล้เคียงกับข้อความดังกล่าว โดยแสดงผลการแนะนำจำนวน 5 อันดับแรก พร้อมค่าความคล้ายคลึงของแต่ละรายการ

ตารางที่ 21 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “ย่านเมืองเก่าที่มีอาคารชิโนโปรตุกีส และผสมผสานหลายวัฒนธรรม”

TF-IDF		
อันดับ	ชื่อสถานที่	Score
1	เมืองเก่าสตูล	0.3308
2	หอยทะเลที่ตีนจังหวัดภูเก็ต	0.2662
3	หอนาฬิกาวงเวียนสุรินทร์	0.1907
4	เมืองเก่าภูเก็ต	0.1795
5	พิพิธภัณฑ์เมืองภูเก็ต	0.1694
Thai2Vec		
อันดับ	ชื่อสถานที่	Score
1	เมืองเก่าสตูล	0.7421
2	เมืองเก่าภูเก็ต	0.7293
3	เมืองเก่าตะกั่วป่า	0.7272
4	หอยทะเลที่ตีนจังหวัดภูเก็ต	0.7201
5	หอนาฬิกาวงเวียนสุรินทร์	0.7096
FastText		
อันดับ	ชื่อสถานที่	Score
1	หอนาฬิกาวงเวียนสุรินทร์	0.9004
2	หอยทะเลที่ตีนจังหวัดภูเก็ต	0.8853

3	เมืองเก่าภูเก็ต	0.8699
4	เมืองเก่าสตูล	0.8562
5	บ้านชินประชา	0.8495
Sentence-BERT		
อันดับ	ชื่อสถานที่	Score
1	เมืองเก่าสตูล	0.8224
2	หอยทะเลเปียนที่ตื้นจังหวัดภูเก็ต	0.8005
3	ย่านเมืองเก่าบ้านสิงห์ท่า	0.7552
4	ชุมชนท่องเที่ยวนครชุม	0.7506
5	คุ้มเจ้าหลวงเมืองแพร่	0.7212

TF-IDF, Thai2Vec และ Sentence-BERT ต่างแนะนำ “เมืองเก่าสตูล” ตรงกันเป็นอันดับ 1 ซึ่งเป็นย่านเมืองเก่าที่มีสถาปัตยกรรมชิโนโปรตุกีส และความหลากหลายทางวัฒนธรรมตรงตาม Query อย่างชัดเจน แสดงถึงความสอดคล้องเชิงความหมายใน 3 วิธี ขณะที่ FastText จัดให้ “หอนาฬิกาวงเวียนสุรินทร์” ซึ่งเป็นสถานที่ในย่านเมืองเก่าภูเก็ตเช่นกัน แต่เป็นจุดสังเกตเฉพาะจุด และจัด “เมืองเก่าสตูล” ไว้เพียงอันดับ 4 กรณีนี้แสดงว่าเมื่อ Query เป็นประโยคบรรยายเชิงพื้นที่ วิธีที่พิจารณาความหมายทั้งประโยคมีแนวโน้มจับคำตอบที่ตรงประเด็นได้ดีกว่า

ตารางที่ 22 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “นาชั้นบันไดที่เหมาะสมสำหรับเที่ยวในฤดูฝน หรือชมหมอก พร้อมสัมผัสวิถีชีวิตดั้งเดิม”

TF-IDF		
อันดับ	ชื่อสถานที่	Score
1	นาชั้นบันไดบ้านเวร	0.1916

2	ป่าบางเปี้ยว	0.1486
3	ชุมชนบ้านบางเสร่	0.1201
4	วิสาหกิจชุมชนท่องเที่ยวบ้านทะเลน้อยระยอง ฮี	0.1194
5	เปี้ยวซ้อ	0.1114
Thai2Vec		
อันดับ	ชื่อสถานที่	Score
1	ชายทะเลบ้านนาใต้	0.6455
2	นาชั้นบันไดบ้านเวร	0.6406
3	ควนทรนฮิลล์ นาทวี	0.6166
4	ผาหมวก-ผาหนอง	0.6159
5	ขุนแปะ	0.5943
FastText		
อันดับ	ชื่อสถานที่	Score
1	ป่าบางเปี้ยว	0.8619
2	นาชั้นบันไดบ้านเวร	0.8517
3	เปี้ยวซ้อ	0.8202
4	ดอยเขาม้ง	0.819
5	วนอุทยานน้ำตกตาควัน	0.818
Sentence-BERT		
อันดับ	ชื่อสถานที่	Score

1	นาชั้นบันไดบ้านเวร	0.7128
2	ป่าบงเปียง	0.6556
3	จุดชมวิwb้านเผ่าไทย	0.6431
4	คานชิงห้วยเคียน	0.635
5	ผาหมอกมียว	0.6182

TF-IDF และ Sentence-BERT ต่างแนะนำ “นาชั้นบันไดบ้านเวร” ตรงกัน ซึ่งตรงกับคำว่า “นาชั้นบันได” ใน Query โดยตรงทั้งในระดับคำและระดับความหมาย ขณะที่ FastText แนะนำ “ป่าบงเปียง” ซึ่งเป็นพื้นที่เกษตรบนภูเขาที่ใกล้เคียงแต่ไม่ใช่นาชั้นบันไดโดยตรง (ให้ค่าคะแนน “นาชั้นบันไดบ้านเวร” ไว้เป็นอันดับ 2) ส่วน Thai2Vec แนะนำ “ชายทะเลบ้านนาใต้” ซึ่งไม่มีความเกี่ยวข้องกับนาชั้นบันไดหรืออุทยานแต่อย่างใด สะท้อนข้อจำกัดของ Thai2Vec ในการจับความหมายของ Query ที่เป็นประโยคยาวและมีหลายเงื่อนไข

ตารางที่ 23 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “หมู่บ้านเล็กๆ ในแถบภาคเหนือที่เด่นในเรื่องผ้ามัดย้อม สวนผลไม้และอยู่ท่ามกลางธรรมชาติ”

TF-IDF		
อันดับ	ชื่อสถานที่	Score
1	อ่างเก็บน้ำร่องลึก	0.116
2	ชุมชนท่องเที่ยวบ้านโพธิ์ทอง	0.1146
3	ชุมชนท่องเที่ยวบ้านมุงเหนือ	0.1146
4	หมู่บ้านดอยสากด	0.1088
5	ทดสอบสร้าง โฟส 1	0.1069
Thai2Vec		

อันดับ	ชื่อสถานที่	Score
1	ชุมชนท่องเที่ยวบ้านมุงเหนือ	0.6875
2	ชุมชนท่องเที่ยวบ้านโพธิ์ทอง	0.6875
3	หมู่บ้านพุมเรียง	0.6838
4	หมู่บ้านท่องเที่ยวชาวดอย (ม้ง) เล่าลือ	0.6713
5	วิสาหกิจชุมชนท่องเที่ยวบ้านธรรมชาติล่าง จังหวัดตราด	0.6702
FastText		
อันดับ	ชื่อสถานที่	Score
1	บ้านป่าปู้	0.8679
2	โรงย้อมยายยอด	0.8623
3	ชุมชนท่องเที่ยวบ้านโพธิ์ทอง	0.8578
4	ชุมชนท่องเที่ยวบ้านมุงเหนือ	0.8578
5	กลุ่มแม่บ้านเกษตรกรนครถ้าเต่า	0.8442
Sentence-BERT		
อันดับ	ชื่อสถานที่	Score
1	หมู่บ้านแม่กำปอง	0.7136
2	สวนทุเรียนคุณป้าไสว	0.7111
3	โฮมสเตย์บ้านเขาเหล็ก	0.7104
4	วิสาหกิจชุมชนท่องเที่ยวเชิงเกษตรตำบลไม้เค็ด	0.7056
5	เมืองคอง	0.7024

แต่ละวิธีให้คำตอบที่แตกต่างกันอย่างชัดเจน โดย Sentence-BERT แนะนำ “หมู่บ้านแม่กำปอง” ซึ่งเป็นหมู่บ้านท่องเที่ยวเชิงธรรมชาติในภาคเหนือที่มีชื่อเสียงตรงตามบริบทของ Query มากที่สุด ขณะที่ Thai2Vec และ FastText แนะนำ “ชุมชนท่องเที่ยวบ้านมุงเหนือ” และ “บ้านป่าปู้” ตามลำดับ ซึ่งเป็นชุมชนท่องเที่ยวลักษณะใกล้เคียงกันแต่ไม่เน้นคำว่า “ผ้ามัดย้อม” หรือ “สวนผลไม้” อย่างชัดเจน ส่วน TF-IDF แนะนำ “อ่างเก็บน้ำร่องสัก” ด้วยคะแนนต่ำมาก (0.116) และไม่มีความเกี่ยวข้องเชิงเนื้อหาเกี่ยวกับ Query อย่างชัดเจน แสดงข้อจำกัดของ TF-IDF เมื่อ Query เป็นประโยคบรรยายที่ไม่มีคำเฉพาะซ้ำกับคำอธิบายสถานที่ในชุดข้อมูลโดยตรง

ตารางที่ 24 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “ดอยหรือป่าที่เหมาะสมสำหรับการเดินเขา หรือวิ่ง trail สำหรับผู้เริ่มต้น”

TF-IDF		
อันดับ	ชื่อสถานที่	Score
1	บึงน้ำทิพย์	0.1012
2	น้ำตกแม่เย็น	0.0969
3	ที่พักริมทางห้วยคั่นก้อม	0.0953
4	อ่างเก็บน้ำเขาจุก	0.0906
5	น้ำตกแม่ปล้อง	0.0858
Thai2Vec		
อันดับ	ชื่อสถานที่	Score
1	ดอยม่อนจอง	0.6661
2	น้ำตกพารมเย็น	0.6633
3	ดอยภูแว	0.6506
4	เขาดินสอ	0.6445

5	น้ำตกห้วยตึงเฒ่า (ตาดมุก)	0.6405
FastText		
อันดับ	ชื่อสถานที่	Score
1	ดอยภูแว	0.8292
2	ดอยผาผึ้ง	0.8198
3	น้ำตกแม่เย็น	0.8117
4	อุทยานแห่งชาติแจ้ซ้อน	0.8085
5	อุทยานแห่งชาติแม่เมย	0.8049
Sentence-BERT		
อันดับ	ชื่อสถานที่	Score
1	ภูวัด	0.728
2	เส้นทางเดินป่าเขาช้างเผือก อุทยานแห่งชาติทองผาภูมิ	0.7214
3	ดอยหัวหมู	0.7039
4	ดอยทุเล	0.703
5	ไฟล์ท ออฟ เดอะ กิบบอน	0.7

Thai2Vec และ FastText ต่างแนะนำสถานที่ประเภท “ดอย” ตรงกับคำใน Query โดยตรง (“ดอยม่อนจอง” และ “ดอยภูแว” ตามลำดับ) ซึ่งเป็นเส้นทางเดินป่า/เดินเขาที่มีชื่อเสียงในภาคเหนือ ส่วน Sentence-BERT แนะนำ “ภูวัด” ซึ่งเป็นภูเขาที่มีเส้นทางเดินเขาเช่นกัน แม้ไม่ใช่คำว่า “ดอย” ตรงตัว แสดงถึงความเข้าใจเชิงความหมายของกิจกรรม “เดินเขา” ขณะที่ TF-IDF แนะนำ “บึงน้ำทิพย์” ด้วยคะแนนต่ำมาก (0.1012) ซึ่งเป็นแหล่งน้ำไม่ใช่ภูเขาหรือเส้นทางเดินป่า สะท้อนข้อจำกัดของ TF-IDF เมื่อ Query ไม่มีคำสำคัญ ตรงกับคำอธิบายสถานที่ในชุดข้อมูล

ตารางที่ 25 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “ดำนํ้าดูปะการัง ใกล้กรุงเทพ และอาหารทะเลสดอร่อย”

TF-IDF		
อันดับ	ชื่อสถานที่	Score
1	เกาะแอม	0.3777
2	อ่าวขอนแก่น	0.2442
3	อ่าวทือ	0.2155
4	เกาะราชาใหญ่	0.2116
5	อ่าวลึก	0.1913
Thai2Vec		
อันดับ	ชื่อสถานที่	Score
1	อ่าวสยาม	0.643
2	หมู่บ้านน้ำตาลสด	0.608
3	อ่างเก็บน้ำเขาสามลิม	0.5404
4	อ่าวขอนแก่น	0.5383
5	หาดไทรตรัง	0.5306
FastText		
อันดับ	ชื่อสถานที่	Score
1	หมู่บ้านประมงอ่าวสลัด	0.805
2	หาดไม้ขาว	0.8042
3	เกาะปิตะโนและเกาะปิตะนอก	0.8032

4	วิสาหกิจชุมชนท่องเที่ยวบ้านน้ำเขียว	0.8002
5	เกาะแรด	0.7999
Sentence-BERT		
อันดับ	ชื่อสถานที่	Score
1	อ่าวลึก	0.7608
2	อ่าวสุเทพ	0.746
3	อ่าวหลา	0.7413
4	อ่าวเต่า	0.7408
5	เกาะพยาม	0.7289

ทุกวิธีแนะนำสถานที่ที่เกี่ยวข้องกับทะเลหรืออ่าวเช่นเดียวกับ Query แต่เน้นคนละแง่มุม โดย TF-IDF แนะนำ “เกาะแอร” ซึ่งมีคำอธิบายเกี่ยวกับการดำน้ำดูปะการังตรงกับคำในชุดข้อมูลโดยตรง ขณะที่ FastText แนะนำ “หมู่บ้านประมงอ่าวสลด” ซึ่งเชื่อมโยงกับมิติ “อาหารทะเล” ผ่านคำว่า “ประมง” ส่วน Sentence-BERT แนะนำ “อ่าวลึก” และ Thai2Vec แนะนำ “อ่าวสยาม” ซึ่งเป็นพื้นที่ชายฝั่งทะเลเช่นกันแต่ไม่ได้ระบุกิจกรรมดำน้ำดูปะการังหรืออาหารทะเลไว้อย่างชัดเจนในชื่อ กรณีนี้แสดงว่า Query ที่มีหลายเงื่อนไขร่วมกันทำให้แต่ละวิธีเลือกให้น้ำหนักกับคนละเงื่อนไข จึงได้คำตอบที่ต่างกัน

ตารางที่ 26 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “สถานที่ท่องเที่ยวใกล้กรุงเทพสำหรับคนชอบกิจกรรม adventure”

TF-IDF		
อันดับ	ชื่อสถานที่	Score
1	สมุยชิปไลน์	0.2017
2	ทรี บริดจ์ ชิปไลน์ สมุย	0.175

3	เดอะฮอर्सแมน คาเฟ่ แอนด์ ฟาร์ม	0.1297
4	มัจฉานูแลนด์ ณ เดอะศาลาयाเลเซอร์ปาร์ค	0.1297
5	เขาวงกต เขาค้อ	0.1252
Thai2Vec		
อันดับ	ชื่อสถานที่	Score
1	ท่องเที่ยวโดยชุมชน ลีเล็ด CBT Leeled	0.7203
2	ล่องแก่งลำน้ำสาน	0.6736
3	ชมรมอนุรักษ์พันธุ์ม้าพันธุ์พื้นบ้านสิรินธร	0.6516
4	ท่องเที่ยวชุมชนบ้านป่าหมาก	0.6342
5	ถนนคนเดินบางแสน	0.6298
FastText		
อันดับ	ชื่อสถานที่	Score
1	มัจฉานูแลนด์ ณ เดอะศาลาयाเลเซอร์ปาร์ค	0.8276
2	ทรัพย์ไพรวัลย์รีสอร์ทและศูนย์เรียนรู้ช้างทรัพย์ไพรวัลย์	0.802
3	ฟาร์มเดอเล็ก (Farm de Lek)	0.8009
4	ทรี บริดจ์ ซิปไลน์ สมุย	0.7985
5	ถนนคนเดิน ฟิชเชอร์แมน วิลเลจ	0.7976
Sentence-BERT		
อันดับ	ชื่อสถานที่	Score
1	พรีเมียม เอาท์เล็ท	0.849

2	เส้นทางปั่นจักรยานเลียบริมฝั่งโขงนครพนม	0.8414
3	สยามนิรมิตร	0.8329
4	สวนน้ำเขาโพธิ์	0.8306
5	เลเจนด์ สยาม	0.8301

ผลลัพธ์อันดับ 1 ของแต่ละวิธีมีความหลากหลายและสะท้อนความไม่แน่นอนของ Query ที่มีลักษณะกว้าง โดย TF-IDF แนะนำ “สมุยชิปไลน์” ซึ่งตรงกับคำว่า “adventure” แต่ตั้งอยู่ที่เกาะสมุย ไม่ตรงกับเงื่อนไข “ใกล้กรุงเทพ” ขณะที่ FastText แนะนำ “มัจฉาแลนด์ ณ เดอะศาลายาเลเซอร์ปาร์ค” ซึ่งอยู่ใกล้กรุงเทพและมีลักษณะเป็นสวนสนุกกิจกรรมกลางแจ้ง ส่วน Sentence-BERT แนะนำ “พรีเมียม เอาร์ทเล็ท” ซึ่งเป็นแหล่งช้อปปิ้งไม่เกี่ยวข้องกับกิจกรรม adventure โดยตรง และ Thai2Vec แนะนำแหล่งท่องเที่ยวชุมชนซึ่งก็ไม่ตรงกับความหมายกิจกรรมผจญภัยเช่นกัน กรณีนี้ชี้ว่าเมื่อ Query มีคำทับศัพท์ภาษาอังกฤษปนอยู่ ประสิทธิภาพของทุกวิธีอาจลดลง เนื่องจากข้อมูลอธิบายสถานที่ส่วนใหญ่เป็นภาษาไทย

ตารางที่ 27 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “สถานที่ท่องเที่ยวเมืองเก่าที่เป็นที่นิยมในหมู่ชาวจีน”

TF-IDF		
อันดับ	ชื่อสถานที่	Score
1	น้ำตกวังหินหอ	0.0935
2	ชมรมส่งเสริมการท่องเที่ยวโดยชุมชนตำบลในเวียง	0.0856
3	ชุมชนศรีรายา (ชุมชนเมืองเก่าเกาะลันตา)	0.0699
4	หาดกะหลิม	0.0673
5	ศูนย์ฝึกอบรมช่างสิบหมู่สุพรรณบุรี	0.0668
Thai2Vec		
อันดับ	ชื่อสถานที่	Score

1	ย่านเมืองเก่าบ้านสิงห์ท่า	0.7146
2	ตลาดบ้านใหม่	0.7078
3	เมืองบาดาล	0.7016
4	ตลาดเก่าหันคา	0.695
5	ตลาดเก่า และตรอกเล่งบัวเยอ เอื้อยะ	0.6934
FastText		
อันดับ	ชื่อสถานที่	Score
1	ย่านเมืองเก่าบ้านสิงห์ท่า	0.8354
2	วิสาหกิจชุมชนตลาดเก่าริมน้ำจันทบูร	0.8275
3	ศาลเจ้าแม่เบ็กไพร	0.8234
4	ชุมชนบ้านขุนสมุทรจีน	0.8118
5	เมืองเก่าสตูล	0.8062
Sentence-BERT		
อันดับ	ชื่อสถานที่	Score
1	ชุมชนศรีรายา (ชุมชนเมืองเก่าเกาะลันตา)	0.8232
2	ตรอกบ้านจีน	0.8162
3	หมู่บ้านศูนย์วัฒนธรรมจีนยูนนานบ้านสันติชล	0.8139
4	วิสาหกิจชุมชนตลาดเก่าริมน้ำจันทบูร	0.8132
5	ศาลเจ้ามูลนิธิธรรมกตัญญู (เสียนหล่อใต้เทียนกง)	0.8113

Thai2Vec และ FastText ต่างแนะนำ “ย่านเมืองเก่าบ้านสิงห์ท่า” ตรงกัน ซึ่งเป็นย่านเมืองเก่าที่มีชุมชนชาวจีนอาศัยอยู่ตรงตามบริบทของ Query ส่วน Sentence-BERT แนะนำ “ชุมชนศรีรายา (ชุมชนเมืองเก่าเกาะลันตา)” ซึ่งเป็นย่านเมืองเก่าที่มีวัฒนธรรมจีนผสมผสานเช่นกัน แสดงความเข้าใจเชิงความหมายของทั้งสามวิธีที่สอดคล้องกับ Query ได้ดี ขณะที่ TF-IDF แนะนำ “น้ำตกวังหินหอ” ด้วยคะแนนต่ำมากเพียง 0.0935 ซึ่งไม่มีความเกี่ยวข้องกับเมืองเก่าหรือวัฒนธรรมจีนเลย สะท้อนข้อจำกัดของ TF-IDF อย่างชัดเจนเมื่อ Query ไม่มีคำปรากฏตรงกับคำอธิบายสถานที่ในชุดข้อมูล

ตารางที่ 28 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “วัดเก่าที่มีประวัติศาสตร์ยาวนานมากกว่า 100 ปี ที่แสดงถึงการผสมผสานวัฒนธรรมต่างชาติ”

TF-IDF		
อันดับ	ชื่อสถานที่	Score
1	ชุมชนย่านเก่าวังกรด	0.1893
2	วัดราชโอรสาราม ราชวรวิหาร	0.1757
3	โบราณสถานบ่อแก่ง	0.1516
4	พิพิธภัณฑสถานแห่งชาติกรุงเทพมหานคร เขตสวนหลวง	0.1462
5	เมืองเก่าสตูล	0.1424
Thai2Vec		
อันดับ	ชื่อสถานที่	Score
1	วัดวาสุการาม	0.8369
2	วัดบวรเม่งคล ราชวรวิหาร	0.8318
3	วัดธรรมาราม	0.8257
4	บ้านเดียนฉาย	0.8252

5	วัดชุมพลคีรี	0.8178
FastText		
อันดับ	ชื่อสถานที่	Score
1	ล้ง 1919	0.909
2	บ้านเดื่อนฉาย	0.8645
3	อุทยานสามก๊ก	0.8628
4	เมืองเก่าสุล	0.8611
5	วัดบวกรกรหลวง	0.8603
Sentence-BERT		
อันดับ	ชื่อสถานที่	Score
1	วัดชุมพลคีรี	0.8091
2	วัดวาฬุการาม	0.7887
3	วัดตาลง	0.786
4	วัดราชโอรสาราม ราชวรวิหาร	0.7854
5	วัดศรีสวรรค์สังฆาราม (วัดถือน้ำ)	0.7834

Thai2Vec และ Sentence-BERT ต่างแนะนำวัดเก่าโดยตรง (“วัดวาฬุการาม” และ “วัดชุมพลคีรี” ตามลำดับ) ซึ่งตรงกับคำว่า “วัดเก่า” ใน Query ขณะที่ FastText แนะนำ “ล้ง 1919” ซึ่งเป็นท่าเรือประวัติศาสตร์ที่มีสถาปัตยกรรมผสมผสานจีน-ไทย ตรงกับเงื่อนไข “ผสมผสานวัฒนธรรมต่างชาติ” แม้จะไม่ใช่วัดก็ตาม ส่วน TF-IDF แนะนำ “ชุมชนย่านเก่าวังกรด” ซึ่งเป็นชุมชนเก่าแก่แต่ไม่ใช่วัดและไม่เน้นวัฒนธรรมต่างชาติอย่างชัดเจน กรณีนี้แสดงว่าเมื่อ Query มีหลายเงื่อนไขซ้อนกัน แต่ละวิธีอาจให้ความสำคัญกับคนละเงื่อนไข ทำให้ได้คำตอบต่างประเภทสถานที่กัน

ตารางที่ 29 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “ชุมชนริมน้ำที่เหมาะสมสำหรับการพักผ่อนและท่องเที่ยว”

TF-IDF		
อันดับ	ชื่อสถานที่	Score
1	หาดตะวันฉาย	0.2281
2	สวนธรรมชาติ	0.1777
3	อ่างเก็บน้ำห้วยสีโท	0.1756
4	อ่างเก็บน้ำวังบอน	0.175
5	เกาะราฮัม	0.1577
Thai2Vec		
อันดับ	ชื่อสถานที่	Score
1	วิสาหกิจชุมชนท่องเที่ยวชุมชนตำบลม่วงกลวง	0.8033
2	กลุ่มท่องเที่ยวชุมชนบ้านทุ่งนางดำ	0.8013
3	ชุมชนม่วงกลวง	0.7977
4	ชุมชนปากน้ำประแส	0.7839
5	ท่องเที่ยวโดยชุมชนบ้านบางโรง	0.7782
FastText		
อันดับ	ชื่อสถานที่	Score
1	ชุมชนท่องเที่ยวแหลมสัก	0.8208
2	วิสาหกิจท่องเที่ยวโดยชุมชนบ้านไร่สะท้อนถ้ำรงค์	0.816
3	บ้านเกวต	0.8092

4	วิสาหกิจชุมชนท่องเที่ยวชุมชนตำบลม่วงกลวง	0.8022
5	ชุมชนปากน้ำประแส	0.8006
Sentence-BERT		
อันดับ	ชื่อสถานที่	Score
1	อ่างเก็บน้ำมาบประชัน	0.8432
2	เขื่อนเรียงหิน	0.8299
3	อ่างเก็บน้ำชลประทานห้วยป่าแดง	0.8187
4	สวนตาสรณ์ สป่าปลาตอด	0.8164
5	บึงเกลือ	0.8123

Thai2Vec และ FastText ต่างแนะนำชุมชนท่องเที่ยวที่มีคำว่า “ชุมชน” ตรงกับ Query โดยตรง (“วิสาหกิจชุมชนท่องเที่ยวชุมชนตำบลม่วงกลวง” และ “ชุมชนท่องเที่ยวแหลมสัก” ตามลำดับ) ซึ่งเป็นชุมชนริมทะเลหรือริมน้ำที่รองรับการท่องเที่ยวเชิงพักผ่อนตรงตามบริบท ขณะที่ Sentence-BERT แนะนำ “อ่างเก็บน้ำมาบประชัน” ซึ่งเป็นแหล่งน้ำสำหรับพักผ่อนเช่นกันแต่ไม่ใช่ “ชุมชน” ตามที่ระบุใน Query ส่วน TF-IDF แนะนำ “หาดตะวันฉาย” ซึ่งเป็นสถานที่พักผ่อนริมน้ำเช่นกันแต่เป็นชายหาดมากกว่าชุมชน กรณีนี้แสดงว่า Thai2Vec และ FastText ให้น้ำหนักกับคำว่า “ชุมชน” ตรงกับ Query มากกว่าอีกสองวิธี

ตารางที่ 30 ผลการแนะนำสถานที่จากข้อมูลนำเข้า “สถานที่ท่องเที่ยวที่คล้ายกับอุทยานผาแต้ม หรือเขาปลาร้า มีภาพเขียนก่อนยุคประวัติศาสตร์”

TF-IDF		
อันดับ	ชื่อสถานที่	Score
1	เขาปลาร้า	0.1447
2	ภาพเขียนสีก่อนประวัติศาสตร์ที่ผาแต้ม	0.1344

3	ภูผาแต้ม	0.1317
4	วัดพระพุทธราย	0.1282
5	แหล่งโบราณคดีภูผายา	0.1121
Thai2Vec		
อันดับ	ชื่อสถานที่	Score
1	หอโบราณคดีเพชรบูรณ์อินทราชัย	0.7623
2	พิพิธภัณฑ์รถจักรยานยนต์โล้วเฮงหมง	0.7475
3	ศูนย์ศิลปวัฒนธรรมจังหวัดภูเก็ต	0.746
4	มณฑปรอยพระพุทธรบาท	0.7407
5	ภาพเขียนสีก่อนประวัติศาสตร์ที่ผาแต้ม	0.7404
FastText		
อันดับ	ชื่อสถานที่	Score
1	ภาพเขียนสีก่อนประวัติศาสตร์ที่ผาแต้ม	0.8516
2	ภูผาแต้ม	0.8505
3	แหล่งโบราณคดีภูผายา	0.843
4	ถ้ำฝ่ามือแดง	0.8391
5	เขาปลาร้า	0.8344
Sentence-BERT		
อันดับ	ชื่อสถานที่	Score
1	วัดพระพุทธราย	0.8028

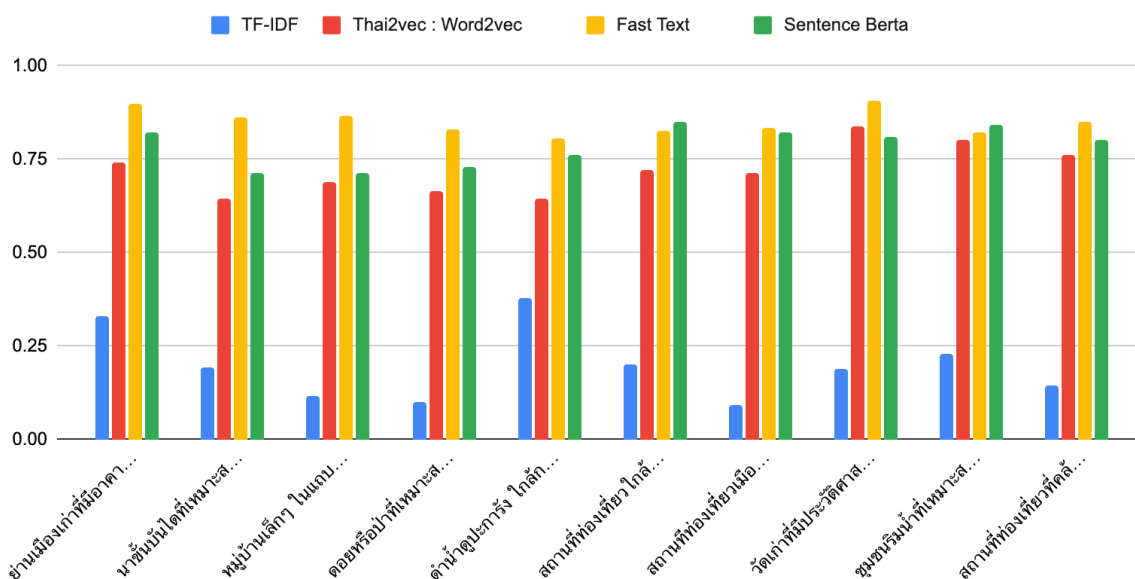
2	อุทยานประวัติศาสตร์พิมาย	0.7922
3	แหล่งโบราณคดีค่ายประจวบ	0.7894
4	ชมรมส่งเสริมการท่องเที่ยวโดยชุมชนตำบลในเวียง	0.7866
5	จุดถ่ายรูปเช็คอิน Street Art	0.7839

TF-IDF แนะนำ “เขาปลาร้า” ซึ่งเป็นชื่อสถานที่ที่ปรากฏตรงในคำถามของ Query โดยตรง ขณะที่ FastText แนะนำ “ภาพเขียนสีก่อนประวัติศาสตร์ที่ผาแต้ม” ซึ่งตรงกับทั้งชื่อสถานที่และลักษณะเนื้อหาที่ระบุใน Query อย่างครบถ้วนที่สุด ส่วน Sentence-BERT แนะนำ “วัดพระพุทธราย” ซึ่งมีภาพเขียนหรือร่องรอยประวัติศาสตร์เช่นกันแต่เป็นวัด ไม่ใช่แหล่งโบราณคดีธรรมชาติ และ Thai2Vec แนะนำ “หอโบราณคดีเพ็ชรบูรณ์ อินทราชัย” ซึ่งเกี่ยวข้องกับโบราณคดีเชิงพิพิธภัณฑ์มากกว่าภาพเขียนบนผาหินตามธรรมชาติ กรณีนี้ FastText จับความสัมพันธ์กับ Query ได้ตรงประเด็นที่สุดในบรรดาทั้งสี่วิธี เนื่องจากเชื่อมโยงทั้งชื่อเฉพาะและคำอธิบายเชิงเนื้อหาได้พร้อมกัน

4.2.2 ผลการเปรียบเทียบค่า Cosine ของ By Text

เพื่อแสดงกระจายตัวของค่าคะแนนความคล้ายคลึงโคไซน์ของกรณีศึกษาต่าง ๆ ผู้วิจัยได้กำหนดชุดคำค้นหาทดสอบในรูปแบบข้อความจำนวน 10 คำค้นหา ไปประมวลผลผ่านฟังก์ชันการแนะนำละนำไปประมวลผลผ่านฟังก์ชันการแนะนำของแบบจำลองทั้ง 4 วิธี ได้แก่ TF-IDF, Thai2Vec, FastText และ Sentence-BERT ภายใต้เงื่อนไขเดียวกัน โดยพิจารณาค่าความคล้ายคลึงโคไซน์ของผลลัพธ์อันดับที่ 1 (Top-1) จากแต่ละคำค้นหา ผลการทดลองแสดงดังรูปที่ 16

แผนภูมิแสดงค่าความคล้ายคลึงโคไซน์ Top-1 จากการค้นหาด้วยข้อความ



รูปที่ 16 แผนภูมิแสดงค่าความคล้ายคลึงโคไซน์จากการหาคำค้นหาด้วยข้อความ

จากรูปที่ 16 พบว่า ค่าความคล้ายคลึงโคไซน์ของแต่ละแบบจำลองมีความแตกต่างกันในแต่ละคำค้นหา โดยแบบจำลองที่ให้ค่าความคล้ายคลึงสูงกว่ามีแนวโน้มที่จะสามารถค้นหาสถานที่ท่องเที่ยวที่มีความสัมพันธ์กับข้อความคำค้นหาได้มากขึ้น อย่างไรก็ตาม คะแนนความคล้ายคลึงของแต่ละแบบจำลองเกิดจากการสร้างตัวแทนข้อความและการคำนวณความสัมพันธ์ในพื้นที่เวกเตอร์ที่แตกต่างกัน ดังนั้น การเปรียบเทียบผลการทดลองจึงพิจารณาร่วมกันทั้งด้านรายการสถานที่ที่ได้รับการแนะนำ ลำดับของผลการแนะนำ และความเหมาะสมของสถานที่ที่แนะนำ

เพื่อแสดงแนวโน้มของคะแนนความคล้ายคลึงโคไซน์ ผู้วิจัยได้นำคะแนนของผลลัพธ์อันดับที่ 1 จากการทดสอบด้วยข้อความทั้ง 10 กรณี มาคำนวณค่าเฉลี่ยและช่วงคะแนนของแต่ละแบบจำลอง ดังแสดงในตารางที่ 31

ตารางที่ 31 ค่าเฉลี่ย Top-1 Cosine Similarity จากการค้นหาโดยข้อความ

แบบจำลอง	Mean Top-1 Score	ช่วง Top-1 Score
TF-IDF	0.1975	0.0935 - 0.3777

Thai2Vec	0.7222	0.6430 - 0.8369
FastText	0.8509	0.8050 - 0.9090
Sentence-BERT	0.7865	0.7128 - 0.8490

จากตารางที่ 31 พบว่า FastText มีค่าเฉลี่ย Top-1 Cosine Similarity สูงที่สุด เท่ากับ 0.8509 และมีช่วงคะแนนอยู่ระหว่าง 0.8050 - 0.9090 รองลงมาคือ Sentence-BERT มีค่าเฉลี่ยเท่ากับ 0.7865 และมีช่วงคะแนนอยู่ระหว่าง 0.7128 - 0.8490 และ Thai2Vec มีค่าเฉลี่ยเท่ากับ 0.7222 โดยมีช่วงคะแนนอยู่ระหว่าง 0.6430 - 0.8369 ส่วน TF-IDF มีค่าเฉลี่ยต่ำที่สุดเท่ากับ 0.1975 และมีช่วงคะแนนอยู่ระหว่าง 0.0935 - 0.3777

จากผลการทดลอง พบว่าแบบจำลองทั้ง 4 วิธีให้ผลการแนะนำสถานที่ท่องเที่ยวที่แตกต่างกัน ทั้งในด้านรายการสถานที่ ลำดับของผลการแนะนำ และค่าความคล้ายคลึง โดย TF-IDF และ FastText สามารถแนะนำสถานที่ที่มีความเกี่ยวข้องกับคำสำคัญในข้อความนำเข้าได้อย่างชัดเจน ขณะที่ Thai2Vec และ Sentence-BERT ให้ผลการแนะนำที่แตกต่างกันตามวิธีการสร้างตัวแทนข้อความของแต่ละแบบจำลอง

สำหรับ TF-IDF ผลการแนะนำอันดับต้นประกอบด้วย “เขาปลาร้า” “ภาพเขียนสีก่อนประวัติศาสตร์ที่ผาแต้ม” และ “ภูผาแต้ม” ซึ่งมีคำหรือเนื้อหาที่เกี่ยวข้องกับข้อความนำเข้าโดยตรง แสดงให้เห็นว่า TF-IDF สามารถค้นหาสถานที่ที่มีคำสำคัญสอดคล้องกับข้อความที่ป้อนเข้าสู่ระบบได้ อย่างไรก็ตาม ค่าความคล้ายคลึงที่ได้มีระดับต่ำกว่าแบบจำลองอื่นอย่างชัดเจน

สำหรับ Thai2Vec ผลการแนะนำมีความเกี่ยวข้องกับแหล่งโบราณคดี ประวัติศาสตร์ หรือ ศิลปวัฒนธรรม เช่น “หอโบราณคดีเพ็ชรบูรณ์อินทราชัย” และ “ภาพเขียนสีก่อนประวัติศาสตร์ที่ผาแต้ม” แสดงให้เห็นว่าแบบจำลองสามารถค้นหาสถานที่ที่มีความสัมพันธ์กับบริบทของข้อความนำเข้าได้ อย่างไรก็ตาม ผลการแนะนำในอันดับต้นบางรายการอาจมีความสัมพันธ์กับข้อความนำเข้าไม่ชัดเจนเมื่อเปรียบเทียบกับแบบจำลองอื่น

สำหรับ FastText มีค่าเฉลี่ย Top-1 Cosine Similarity สูงที่สุดเท่ากับ 0.8509 โดยสามารถแนะนำสถานที่ที่มีความเกี่ยวข้องกับข้อความนำเข้าได้อย่างชัดเจน เช่น “ภาพเขียนสีก่อนประวัติศาสตร์ที่ผาแต้ม” “ภูผาแต้ม” “แหล่งโบราณคดีภูผายา” และ “ถ้ำฝ่ามือแดง” ซึ่งมีลักษณะเกี่ยวข้องกับแหล่งโบราณคดีหรือ ภาพเขียนในอดีต

ส่วน Sentence-BERT มีค่าเฉลี่ย Top-1 Cosine Similarity เท่ากับ 0.7865 และสามารถแนะนำสถานที่ที่มีความสัมพันธ์กับบริบทหรือความหมายของข้อความนำเข้า เช่น “วัดพระพุทธฉาย” “อุทยานประวัติศาสตร์พิมาย” และ “แหล่งโบราณคดีค่ายประตูผา” อย่างไรก็ตาม สถานที่บางรายการ เช่น “ชมรม

ส่งเสริมการท่องเที่ยวโดยชุมชนตำบลในเวียง” และ “จุดถ่ายรูปเช็คอิน Street Art” อาจมีความสอดคล้องกับข้อความนำเข้าในระดับที่แตกต่างกัน

ดังนั้น เมื่อพิจารณาจากผลการทดสอบด้วยข้อความทั้ง 10 กรณี พบว่า FastText มีค่าเฉลี่ย Top-1 Cosine Similarity สูงที่สุด รองลงมาคือ Sentence-BERT, Thai2Vec และ TF-IDF ตามลำดับ ซึ่งแสดงให้เห็นว่า FastText ให้คะแนนความคล้ายคลึงของผลลัพธ์อันดับที่ 1 สูงที่สุดในการทดสอบรูปแบบ By Text อย่างไรก็ตาม การพิจารณาประสิทธิภาพของระบบแนะนำไม่ควรพิจารณาจากค่า Cosine Similarity เพียงอย่างเดียว แต่ควรพิจารณาร่วมกับรายการสถานที่ที่ได้รับการแนะนำ ลำดับของผลการแนะนำ และความเหมาะสมของสถานที่ที่แนะนำด้วย

4.2.3 ผลการเปรียบเทียบระยะเวลาในการสร้างแบบจำลอง

ในการทดลอง ผู้วิจัยได้บันทึกระยะเวลาในการสร้างแบบจำลองของแต่ละวิธี เพื่อเปรียบเทียบความแตกต่างด้านระยะเวลาที่ใช้ในการประมวลผลข้อมูล โดยการทดสอบด้วยข้อความจำนวน 10 กรณี ได้บันทึกระยะเวลาในการสร้างแบบจำลองในแต่ละครั้ง และนำมาคำนวณช่วงระยะเวลาและค่าเฉลี่ย ดังแสดงในตารางที่ 32

ตารางที่ 32 ผลการเปรียบเทียบระยะเวลาในการสร้างแบบจำลอง และทดสอบด้วยข้อความ (N = 10)

แบบจำลอง	ช่วงระยะเวลาในการสร้างแบบจำลอง	ค่าเฉลี่ยเวลาในการสร้างแบบจำลอง
TF-IDF	3.258s - 3.93s	3.5543s
Thai2Vec	3.702s - 4.242s	3.9567s
FastText	125.751s - 135.624s	129.6425s
Sentence-BERT	25.113s - 28.212s	26.0812s

จากตารางพบว่า TF-IDF ใช้ระยะเวลาเฉลี่ยในการสร้างแบบจำลองน้อยที่สุด คือ 3.5543 วินาที รองลงมาคือ Thai2Vec ซึ่งใช้ระยะเวลาเฉลี่ย 3.9567 วินาที ส่วน Sentence-BERT ใช้ระยะเวลาเฉลี่ย 26.0812 วินาที ขณะที่ FastText ใช้ระยะเวลาเฉลี่ยสูงสุด คือ 129.6425 วินาที

ผลการทดลองแสดงให้เห็นว่าแต่ละแบบจำลองมีระยะเวลาในการสร้างแบบจำลองแตกต่างกัน โดย FastText ใช้ระยะเวลาในการประมวลผลสูงกว่าแบบจำลองอื่นในการทดลองครั้งนี้ ขณะที่ TF-IDF และ Thai2Vec ใช้ระยะเวลาในการสร้างแบบจำลองใกล้เคียงกันและน้อยกว่าแบบจำลองอื่น อย่างไรก็ตาม การพิจารณาประสิทธิภาพของระบบแนะนำควรพิจารณาพร้อมกับคุณภาพและความสอดคล้องของผลการแนะนำ ไม่ควรพิจารณาจากระยะเวลาในการสร้างแบบจำลองเพียงอย่างเดียว

บทที่ 5

สรุปผลการวิจัย อภิปรายผล และข้อเสนอแนะ

การวิจัยครั้งนี้มีวัตถุประสงค์เพื่อศึกษาและพัฒนาระบบแนะนำสถานที่ท่องเที่ยวภายในประเทศไทย โดยประยุกต์ใช้เทคนิคการกรองแบบอิงเนื้อหา (Content-Based Filtering) ในการสร้างคำแนะนำจากข้อมูลเนื้อหา และรายละเอียดของสถานที่ท่องเที่ยว

ชุดข้อมูลที่ใช้ในการวิจัยเป็นข้อมูลแหล่งท่องเที่ยวภายในประเทศไทยจำนวน 8,848 รายการ ซึ่งผ่านกระบวนการเตรียมข้อมูล ได้แก่ การจัดการข้อมูลที่มีค่าขาดหาย การคัดกรองข้อมูลที่มีค่าไม่เหมาะสม การทำความสะอาดข้อมูลที่มีโครงสร้าง HTML และการกำจัดข้อมูลซ้ำ ส่งผลให้เหลือข้อมูลจำนวน 7,478 รายการ สำหรับใช้ในการสร้างแบบจำลอง

ในการวิจัยครั้งนี้ได้เปรียบเทียบแบบจำลองจำนวน 4 วิธี ได้แก่ TF-IDF, Thai2Vec, FastText และ Sentence-BERT โดยใช้วิธี Cosine Similarity ในการคำนวณความคล้ายคลึงระหว่างข้อมูลนำเข้าและข้อมูลสถานที่ท่องเที่ยว ผลการทดลองแบ่งออกเป็น 2 รูปแบบ ได้แก่ การทดสอบด้วยชื่อสถานที่ (By Name) และการทดสอบด้วยข้อความ (By Text) จำนวนรูปแบบละ 10 กรณี รวมทั้งมีการเปรียบเทียบระยะเวลาในการสร้างแบบจำลองของแต่ละวิธี

จากผลการดำเนินการวิจัยที่ได้นำเสนอในบทที่ 4 ผู้วิจัยนำผลการทดลองมาสรุปและอภิปราย เพื่อพิจารณาความแตกต่างของผลการแนะนำจากแบบจำลองทั้ง 4 วิธี ทั้งในด้านสถานที่ที่ได้รับการแนะนำ ลำดับของผลการแนะนำ ค่าความคล้ายคลึงแบบ Cosine Similarity และระยะเวลาในการสร้างแบบจำลอง นอกจากนี้ ยังนำเสนอข้อจำกัดของการวิจัยและข้อเสนอแนะสำหรับการพัฒนาระบบแนะนำสถานที่ท่องเที่ยวในอนาคต โดยแบ่งเนื้อหาออกเป็น 4 ส่วน ดังนี้

1. สรุปผลการวิจัย
2. อภิปรายผลการวิจัย
3. ข้อจำกัดการวิจัย
4. ข้อเสนอแนะ

5.1 สรุปผลการวิจัย

5.1.1 สรุปผลการทดสอบด้วยชื่อสถานที่ (By Name)

จากผลการทดสอบด้วยชื่อสถานที่ (By Name) จำนวน 10 กรณี พบว่า FastText ให้ค่าเฉลี่ยของคะแนนความคล้ายคลึงโคไซน์อันดับ 1 (Top-1) สูงที่สุดที่ 0.9223 รองลงมาคือ Thai2Vec (0.8898) และ Sentence-BERT (0.8733) ขณะที่ TF-IDF มีค่าเฉลี่ยต่ำที่สุดที่ 0.2205 อย่างไรก็ตาม ค่าดังกล่าวสะท้อนความเข้มของค่าความคล้ายคลึงภายในพื้นที่คุณลักษณะ (feature space) ของแต่ละวิธี มากกว่าจะบ่งชี้คุณภาพของคำแนะนำโดยตรง ตัวอย่างเช่น กรณี “ตลาดน้ำคลองแห” TF-IDF แนะนำ “ตลาดน้ำท่าคา” ด้วยคะแนนเพียง 0.2015 ขณะที่ Thai2Vec และ FastText แนะนำ “ตลาดน้ำบางคล้า” ด้วยคะแนน 0.9041 และ 0.9282 ตามลำดับ ซึ่งทั้งสามคำตอบยังคงอยู่ในกลุ่มประเภทตลาดน้ำเช่นเดียวกัน แสดงให้เห็นว่าคะแนนที่ต่ำของ TF-IDF ไม่ได้หมายความว่าคำแนะนำนั้นไม่เหมาะสมเสมอไป

เมื่อพิจารณาความสอดคล้องของผลลัพธ์ระหว่างแบบจำลอง พบว่า Thai2Vec และ FastText ซึ่งเป็นเทคนิคการแทนความหมายระดับคำ (word embedding) ให้ผลลัพธ์อันดับ 1 ตรงกันในหลายกรณี เช่น กรณี “อุทยานแห่งชาติดอยอินทนนท์” ที่ทั้ง Thai2Vec, FastText และ Sentence-BERT ต่างแนะนำ “ดอยหลวงเชียงดาว” เป็นอันดับ 1 เหมือนกัน และกรณี “พระธาตุมรุกขนคร” ที่ TF-IDF, Thai2Vec และ FastText ต่างแนะนำ “วัดพระธาตุท่าอุเทน” ซึ่งแสดงให้เห็นว่าความสัมพันธ์ระหว่างสถานที่บางคู่มีความแข็งแกร่งเพียงพอที่จะถูกตรวจพบได้ด้วยเทคนิคการแทนข้อความที่แตกต่างกัน

โดยสรุป ลำดับประสิทธิภาพเชิงคุณภาพจากการทดสอบด้วยชื่อสถานที่ในชุดข้อมูลนี้ เรียงจากดีที่สุดไปน้อยที่สุดคือ FastText, Thai2Vec, Sentence-BERT และ TF-IDF ตามลำดับ โดย TF-IDF ยังคงมีข้อได้เปรียบด้านความเรียบง่ายและความเร็ว รวมถึงสามารถให้ผลลัพธ์ที่เหมาะสมในกรณีที่สถานที่ต้นทางและสถานที่เป้าหมายมีคำสำคัญร่วมกันอย่างชัดเจน เช่น กรณีตลาดน้ำหรือพระธาตุ

5.1.2 สรุปผลการทดสอบด้วยข้อความ (By Text)

จากผลการทดสอบด้วยข้อความ (By Text) จำนวน 10 กรณี พบว่า FastText ยังคงให้ค่าเฉลี่ยคะแนน Top-1 สูงที่สุดที่ 0.8509 รองลงมาคือ Sentence-BERT (0.7865), Thai2Vec (0.7222) และ TF-IDF (0.1975) ตามลำดับ ซึ่งค่าเฉลี่ยของทุกแบบจำลองในการทดสอบด้วยข้อความมีแนวโน้มต่ำกว่าการทดสอบด้วยชื่อสถานที่ เนื่องจากข้อความค้นหาแบบอิสระ (free-text query) มักประกอบด้วยหลายเงื่อนไขและมีลักษณะเป็นภาษาธรรมชาติมากกว่าชื่อเฉพาะ

ข้อสังเกตที่สำคัญคือ ลำดับของแบบจำลองที่ดีเป็นอันดับ 2 เปลี่ยนจาก Thai2Vec ในการทดสอบด้วยชื่อสถานที่ มาเป็น Sentence-BERT ในการทดสอบด้วยข้อความ เนื่องจาก Sentence-BERT มีความสามารถในการพิจารณาความหมายของข้อความทั้งประโยคซึ่งเหมาะสมกับลักษณะของ Query ที่เป็นประโยคมากกว่าคำเดี่ยว ตัวอย่างเช่น กรณี Query “ย่านเมืองเก่าที่มีอาคารชิโนโปรตุกีสและผสมผสานหลาย

วัฒนธรรม” ซึ่งมุ่งหมายถึง “เมืองเก่าสตูล” พบว่า TF-IDF, Thai2Vec และ Sentence-BERT ต่างให้เมืองเก่าสตูลเป็นอันดับ 1 ตรงกัน ขณะที่ FastText จัดไว้เพียงอันดับ 4 และกรณี Query เกี่ยวกับนาขั้นบันไดในฤดูฝน TF-IDF และ Sentence-BERT ต่างให้ “นาขั้นบันไดบ้านเวร” เป็นอันดับ 1 ตรงตามเป้าหมายเช่นกัน

ในทางกลับกัน เมื่อ Query มีลักษณะเป็นความต้องการเชิงความหมายหรือประกอบด้วยหลายเงื่อนไขร่วมกัน เช่น “ดำนํ้าคูปะการัง ใกล้กรุงเทพ อาหารทะเลสดอร่อย” แต่ละแบบจำลองให้ผลลัพธ์อันดับ 1 ที่แตกต่างกันอย่างชัดเจน ซึ่งสะท้อนว่าแต่ละเทคนิคให้น้ำหนักกับองค์ประกอบของ Query ไม่เหมือนกัน โดยสรุปลำดับประสิทธิภาพเชิงคุณภาพจากการทดสอบด้วยข้อความคือ FastText, Sentence-BERT, Thai2Vec และ TF-IDF ตามลำดับ ซึ่งเป็นลำดับที่แตกต่างจากการทดสอบด้วยชื่อสถานที่เล็กน้อยในตำแหน่งที่ 2 และ 3

5.1.3 สรุปผลการเปรียบเทียบระยะเวลาในการสร้างแบบจำลอง

เมื่อเปรียบเทียบระยะเวลาเฉลี่ยในการสร้างแบบจำลอง (Fit time) ของทั้งการทดสอบด้วยชื่อสถานที่และด้วยข้อความ พบว่า TF-IDF และ Thai2Vec ใช้เวลาน้อยที่สุดและใกล้เคียงกัน คือประมาณ 3.5–4.0 วินาที (TF-IDF เฉลี่ย 3.677 และ 3.5543 วินาที และ Thai2Vec เฉลี่ย 3.973 และ 3.9567 วินาที ในการทดสอบด้วยชื่อสถานที่และด้วยข้อความตามลำดับ) รองลงมาคือ Sentence-BERT ซึ่งใช้เวลาประมาณ 26–28 วินาที ขณะที่ FastText ใช้เวลานานที่สุดคือประมาณ 129–130 วินาที ซึ่งสูงกว่า TF-IDF และ Thai2Vec ราว 30 เท่า

ผลดังกล่าวแสดงให้เห็นถึงข้อแลกเปลี่ยน (trade-off) ระหว่างคุณภาพของคำแนะนำกับต้นทุนด้านเวลาในการประมวลผลอย่างชัดเจน กล่าวคือ FastText ซึ่งให้ผลลัพธ์ด้านความคล้ายคลึงและความสอดคล้องเชิงเนื้อหาที่ดีที่สุดในทั้งสองรูปแบบการทดสอบ กลับมีต้นทุนด้านเวลาสูงที่สุด ขณะที่ TF-IDF และ Thai2Vec ซึ่งประมวลผลได้รวดเร็ว กลับมีข้อจำกัดด้านความสามารถในการจับความหมายเชิงบริบท ข้อสังเกตนี้มีความสำคัญต่อการเลือกใช้งานจริง โดยเฉพาะระบบที่ต้องให้บริการแบบ real-time ซึ่งอาจต้องพิจารณาความสมดุลระหว่างคุณภาพผลลัพธ์กับระยะเวลาตอบสนอง

5.2 อภิปรายผลการวิจัย

5.2.1 การเปรียบเทียบผลการแนะนำของแบบจำลอง

ผลการเปรียบเทียบรายการแนะนำระหว่างแบบจำลองทั้ง 4 วิธี แสดงให้เห็นว่าความแตกต่างของผลลัพธ์ไม่ได้เกิดจากข้อผิดพลาดของแบบจำลองใดแบบจำลองหนึ่ง แต่เกิดจากหลักการนิยามความคล้ายคลึงที่ต่างกันของแต่ละเทคนิค กล่าวคือ TF-IDF พิจารณาความคล้ายคลึงจากการปรากฏร่วมกันของคำในระดับพื้นผิว (lexical level) โดยไม่คำนึงถึงความหมายเชิงบริบท Thai2Vec และ FastText แทนความหมายของคำใน

รูปเวกเตอร์เชิงความหมาย (semantic word embedding) จึงสามารถเชื่อมโยงคำที่มีความหมายใกล้เคียงกันได้ แม้จะไม่ใช่คำเดียวกัน โดย FastText มีข้อได้เปรียบเพิ่มเติมจากการใช้ subword information ทำให้จัดการคำประสมและชื่อเฉพาะซึ่งพบมากในข้อมูลการท่องเที่ยวไทยได้ดีกว่า Thai2Vec ส่วน Sentence-BERT แทนความหมายของข้อความทั้งประโยค จึงมีข้อได้เปรียบเมื่อ Query มีลักษณะเป็นประโยคภาษาธรรมชาติ

จากการสังเกตพบว่า Thai2Vec และ FastText ซึ่งมีพื้นฐานเป็น word embedding เช่นเดียวกัน มักให้ผลลัพธ์อันดับ 1 ตรงกันบ่อยครั้งที่สุด ขณะที่ TF-IDF และ Sentence-BERT ซึ่งมีหลักการแทนข้อความที่แตกต่างกันมากที่สุด กลับสามารถให้ผลลัพธ์ตรงกันได้เช่นกันในกรณีที่ Query หรือชื่อสถานที่ต้นทางมีคำเฉพาะเจาะจงที่โดดเด่นมาก เช่น กรณี “เมืองเก่าสตูล” และ “นาชั้นบันไดบ้านเวร” ซึ่งชี้ให้เห็นว่าความสอดคล้องของผลลัพธ์ระหว่างแบบจำลองไม่ได้ขึ้นอยู่กับความคล้ายคลึงของกลไกภายในเพียงอย่างเดียว แต่ยังขึ้นอยู่กับลักษณะของข้อมูลนำเข้าด้วย

5.2.2 การอธิบายผลค่า Cosine Similarity

ค่าความคล้ายคลึงโคไซน์ (Cosine Similarity) ที่ได้จากแบบจำลองทั้ง 4 วิธี ไม่สามารถนำมาเปรียบเทียบขนาดกันโดยตรงระหว่างวิธี เนื่องจากแต่ละวิธีสร้างเวกเตอร์คุณลักษณะจากพื้นที่คุณลักษณะ (feature space) ที่แตกต่างกัน โดย TF-IDF สร้างเวกเตอร์จากความถี่ของคำที่มีมิติสูงและกระจายตัว (sparse) ทำให้ค่าคะแนนความคล้ายคลึงโดยทั่วไปมีค่าต่ำกว่าฐานของแบบจำลองอื่นอย่างเป็นระบบ ขณะที่ Thai2Vec, FastText และ Sentence-BERT สร้างเวกเตอร์แบบหนาแน่น (dense) ในปริภูมิที่มีมิติต่ำกว่า จึงมักให้ค่าความคล้ายคลึงที่สูงกว่าโดยธรรมชาติของโครงสร้างเวกเตอร์เอง

ตัวอย่างที่แสดงให้เห็นข้อจำกัดนี้อย่างชัดเจนคือกรณี “วัดพระแก้ว” ซึ่งทุกวิธีต่างแนะนำวัดเป็นอันดับ 1 เหมือนกัน (TF-IDF แนะนำ “วัดราชโอรสาราม ราชวรวิหาร”, Thai2Vec แนะนำ “วัดกลาง”, FastText แนะนำ “วัดมหาธาตุ” และ Sentence-BERT แนะนำ “วัดม่วยต่อ”) แต่ TF-IDF ให้คะแนนเพียง 0.1790 ขณะที่อีกสามวิธีให้คะแนนสูงถึง 0.90 ขึ้นไป ทั้งที่คำตอบทั้งสี่มีความสอดคล้องเชิงหมวดหมู่กับข้อมูลนำเข้าเท่ากัน แสดงให้เห็นว่าค่าคะแนนที่ต่ำของ TF-IDF สะท้อนลักษณะของพื้นที่คุณลักษณะแบบ sparse มากกว่าจะบ่งบอกว่าคำแนะนำนั้นมีความเกี่ยวข้องน้อยกว่า ดังนั้น การประเมินคุณภาพของแบบจำลองจึงควรพิจารณาจากคุณสมบัติของผลลัพธ์ของรายการแนะนำร่วมกับค่าคะแนนภายในแบบจำลองเดียวกัน มากกว่าจะเปรียบเทียบขนาดของคะแนนข้ามแบบจำลองโดยตรง และในการประเมินประสิทธิภาพเชิงปริมาณที่แม่นยำยิ่งขึ้นในอนาคต ควรใช้ตัวชี้วัดที่อิงจากชุดข้อมูลอ้างอิงความเกี่ยวข้อง (relevance ground truth) เช่น Precision, Recall หรือ NDCG

5.2.3 การอธิบายผลด้านระยะเวลาในการสร้างแบบจำลอง

ผลการเปรียบเทียบด้านระยะเวลาในการสร้างแบบจำลองแสดงให้เห็นความสัมพันธ์ผกผันกับคุณภาพของผลลัพธ์ที่ได้ในภาพรวม กล่าวคือ TF-IDF และ Thai2Vec ใช้ระยะเวลาเพียง 3–4 วินาทีในการสร้างแบบจำลอง เนื่องจากอาศัยการคำนวณทางสถิติของคำหรือการค้นคืนเวกเตอร์คำที่ฝึกไว้ล่วงหน้าเป็นหลัก ซึ่งมีความซับซ้อนในการคำนวณต่ำ ขณะที่ Sentence-BERT ต้องประมวลผลข้อความผ่านโครงข่ายประสาทเทียมแบบ Transformer จึงใช้เวลาประมาณ 26–28 วินาที และ FastText ซึ่งต้องคำนวณเวกเตอร์จาก subword หรือ character n-grams ของคำจำนวนมาก ใช้เวลานานที่สุดคือประมาณ 129–130 วินาที

ข้อสังเกตนี้มีนัยสำคัญต่อการนำระบบแนะนำไปประยุกต์ใช้จริง กล่าวคือ หากระบบต้องประมวลผลแบบทันที (real-time) หรือมีการอัปเดตข้อมูลบ่อยครั้ง TF-IDF และ Thai2Vec จะมีความเหมาะสมมากกว่าเนื่องจากใช้นิยามน้อย แม้จะมีข้อจำกัดด้านความเข้าใจเชิงความหมาย ในขณะที่ FastText ซึ่งให้ผลลัพธ์ที่มีคุณภาพดีที่สุดในการทดลองนี้ อาจเหมาะสมกับระบบที่สร้างแบบจำลองล่วงหน้า (offline) และไม่จำเป็นต้องปรับปรุงแบบจำลองบ่อยครั้ง ดังนั้น การเลือกใช้แบบจำลองในทางปฏิบัติจึงควรพิจารณาทั้งด้านคุณภาพของคำแนะนำและข้อจำกัดด้านทรัพยากรและเวลาในการประมวลผลร่วมกัน

5.3 ข้อจำกัดการวิจัย

การวิจัยครั้งนี้มีข้อจำกัดบางประการ ดังนี้

1) การประเมินผลยังไม่มีข้อมูลอ้างอิงคำตอบที่ถูกต้อง

การประเมินผลในงานวิจัยครั้งนี้อาศัยการเปรียบเทียบรายการสถานที่ที่ได้รับการแนะนำ ลำดับของผลการแนะนำ ค่าความคล้ายคลึง และความสอดคล้องของผลลัพธ์กับข้อมูลนำเข้าเป็นหลัก โดยยังไม่มีชุดข้อมูลอ้างอิงที่กำหนดว่าสถานที่ใดเป็นคำแนะนำที่ถูกต้องหรือเหมาะสมที่สุดสำหรับแต่ละกรณีทดสอบ ดังนั้น จึงไม่สามารถคำนวณตัวชี้วัดเชิงปริมาณ เช่น Precision, Recall หรือ NDCG ได้โดยตรง

2) ขึ้นอยู่กับคุณภาพของข้อมูลรายละเอียดสถานที่ท่องเที่ยว

ระบบแนะนำแบบอิงเนื้อหา มีการสร้างคำแนะนำจากข้อมูลรายละเอียดของสถานที่ท่องเที่ยว ดังนั้น คุณภาพ ความครบถ้วน และความถูกต้องของข้อมูลข้อความจึงมีผลต่อคุณภาพของคำแนะนำ หากข้อมูลของสถานที่บางแห่งมีรายละเอียดน้อยหรือไม่ครบถ้วน อาจส่งผลให้แบบจำลองไม่สามารถสร้างตัวแทนเนื้อหาของสถานที่นั้นได้อย่างเหมาะสม

3) จำนวนกรณีทดสอบมีจำกัด

ในการทดลองครั้งนี้มีการทดสอบด้วยชื่อสถานที่จำนวน 10 รายการ และข้อความจำนวน 10 ข้อความ ซึ่งช่วยให้สามารถเปรียบเทียบแนวโน้มของผลลัพธ์ระหว่างแบบจำลองได้ อย่างไรก็ตาม จำนวนกรณีทดสอบดังกล่าวยังไม่ครอบคลุมความต้องการของผู้ใช้ในทุกลักษณะ

4) การเปรียบเทียบค่า Cosine Similarity ระหว่างแบบจำลองโดยตรงมีข้อจำกัด

เนื่องจากแต่ละแบบจำลองสร้างเวกเตอร์จากวิธีการและพื้นที่คุณลักษณะที่ต่างกัน ขนาดของค่า Cosine Similarity จึงไม่ควรใช้เป็นเกณฑ์เพียงอย่างเดียวในการตัดสินว่าแบบจำลองใดมีประสิทธิภาพสูงกว่า

5.4 ข้อเสนอแนะ

5.4.1 ข้อเสนอแนะสำหรับการพัฒนาระบบในอนาคต

ควรเพิ่มข้อมูลประกอบสำหรับใช้ในการสร้างคำแนะนำ เช่น จังหวัด พิกัดทางภูมิศาสตร์ คะแนนรีวิว หรือข้อมูลความสนใจของผู้ใช้ เพื่อเพิ่มความหลากหลายและความเฉพาะเจาะจงของผลการแนะนำ นอกจากนี้ อาจนำเทคนิค Collaborative Filtering และ Hybrid Recommendation System มาประยุกต์ใช้ร่วมกับ Content-Based Filtering เพื่อช่วยให้ระบบสามารถสร้างคำแนะนำที่เหมาะสมกับผู้ใช้แต่ละรายมากขึ้น

5.4.2 ข้อเสนอแนะด้านการประเมินผล

ในการวิจัยครั้งต่อไป ควรเพิ่มจำนวนกรณีทดสอบให้ครอบคลุมลักษณะของสถานที่ท่องเที่ยวและความต้องการของผู้ใช้ที่หลากหลายมากขึ้น รวมทั้งอาจให้ผู้ใช้หรือผู้เชี่ยวชาญด้านการท่องเที่ยวเข้ามาประเมินความเหมาะสมของผลการแนะนำ การมีข้อมูลอ้างอิงหรือการประเมินจากผู้ใช้งานจะช่วยให้สามารถใช้ตัวชี้วัดเชิงปริมาณเพิ่มเติม เช่น Precision@K, Recall@K, Mean Reciprocal Rank (MRR) หรือ Normalized Discounted Cumulative Gain (NDCG) เพื่อประเมินประสิทธิภาพของระบบแนะนำได้อย่างเป็นระบบมากขึ้น

นอกจากนี้ ควรวัดระยะเวลาในการประมวลผลโดยแยกระหว่างระยะเวลาในการสร้างดัชนีข้อมูล และระยะเวลาในการตอบสนองต่อคำค้น เพื่อให้สามารถเปรียบเทียบต้นทุนเชิงคำนวณของแต่ละแบบจำลองได้อย่างเหมาะสม และนำไปใช้ประกอบการเลือกแบบจำลองสำหรับระบบจริงได้

5.4.3 ข้อเสนอแนะด้านข้อมูลและแบบจำลอง

ควรเพิ่มการปรับปรุงข้อมูลข้อความ เช่น การปรับมาตรฐานคำ การจัดการคำสะกดหลายรูปแบบ คำพ้องความหมาย และการเพิ่มข้อมูลคำอธิบายของสถานที่ที่มีรายละเอียดไม่เพียงพอ เนื่องจากคุณภาพและความครบถ้วนของข้อมูลมีผลโดยตรงต่อคุณภาพของระบบแนะนำที่ระบบสามารถสร้างได้

สำหรับแบบจำลอง ควรศึกษาและเปรียบเทียบแบบจำลองภาษาไทย หรือแบบจำลองเชิงความหมายอื่นเพิ่มเติม เช่น WangchanBERTa หรือ multilinggua-e5-base ซึ่งอาจช่วยให้ระบบเข้าใจความหมายเชิงบริบท และความหมายของภาษาไทยได้ดียิ่งขึ้น โดยเฉพาะในกรณีที่คำค้น และคำอธิบายของสถานที่ใช้คำแตกต่างกันแต่มีความหมายใกล้เคียงกัน นอกจากนี้ ควรศึกษาผลของการปรับค่าพารามิเตอร์ของแต่ละแบบจำลอง เช่น จำนวนมิติของเวกเตอร์ ขนาดหน้าต่างบริบท จำนวนรอบการฝึก และวิธีการรวมเวกเตอร์ระดับคำเป็นระดับเอกสาร เพื่อศึกษาและวิเคราะห์ผลกระทบต่อคุณภาพของคำแนะนำและระยะเวลาในการประมวลผล

บรรณานุกรม

- กระทรวงการท่องเที่ยวและกีฬา. (2560). รายงานวิสัยทัศน์การท่องเที่ยวไทย พ.ศ. 2579. สืบค้นจาก https://secretary.mots.go.th/policy/download/article/article_20170511133807.pdf.
- Fayelyn. (2568). ระบบแนะนำสินค้า (Recommendation System) คืออะไร? ใช้งานอย่างไรใน E-Commerce. สืบค้นจาก <https://sennalabs.com/blog/what-is-a-recommendation-system>.
- แบ็คยาร์ด จำกัด. (2568). การประมวลผลภาษาธรรมชาติ (NLP) คืออะไร? ตัวช่วยในการสื่อสารระหว่างคอมพิวเตอร์กับมนุษย์. สืบค้นจาก <https://www.backyard.in.th/post/get-to-know-nlp>.
- กอบกฤตย์ วิริยะยุทธกร. (2568). Thai NLP คืออะไร? คู่มือการประมวลผลภาษาธรรมชาติฉบับสมบูรณ์สำหรับผู้เริ่มต้น. สืบค้นจาก <https://iapp.co.th/th/blog/what-is-thai-nlp-natural-language-processing-beginners-guide>.
- นิภาภรณ์ พันธนาม และ วราภรณ์ วิทยานนท์. (2563). ระบบแนะนำสินค้าอาหารโดยใช้ระบบแนะนำแบบผสมผสาน (Food Recommendation System Using a Hybrid Recommendation Method). สารนิพนธ์ปริญญาวิทยาศาสตรมหาบัณฑิต, มหาวิทยาลัยศรีนครินทรวิโรฒ. กรุงเทพฯ. สืบค้นจาก <https://ir-ithesis.swu.ac.th/handle/123456789/1166>.
- Bojanowski, P., Grave, E., Joulin, A., & Mikolov, T. (2017). Enriching word vectors with subword information. Transactions of the Association for Computational Linguistics, 5, 135–146. https://doi.org/10.1162/tacl_a_00051.
- Borràs, J., Moreno, A., & Valls, A. (2014). Intelligent tourism recommender systems: A survey. Expert Systems with Applications, 41(16), 7370–7389. <https://doi.org/10.1016/j.eswa.2014.06.007>.
- Efimov, S. (2023). Large Language Models: SBERT – Sentence-BERT. สืบค้นจาก <https://towardsdatascience.com/sbert-deb3d4aef8a4/>.
- Grave, E., Bojanowski, P., Gupta, P., Joulin, A., & Mikolov, T. (2018). Learning word vectors for 157 languages. In Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018). European Language Resources Association.
- Hamid, R. A., Albahri, A. S., Alwan, J. K., Al-qaysi, Z. T., Albahri, O. S., Zaidan, A. A., Alnoor, A., Alamoodi, A. H., & Zaidan, B. B. (2021). How smart is e-tourism? A systematic review of

- smart tourism recommendation system applying data management. *Computer Science Review*, 39, 100337. <https://doi.org/10.1016/j.cosrev.2020.100337>.
- Joulin, A., Grave, E., Bojanowski, P., & Mikolov, T. (2017). Bag of tricks for efficient text classification. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers* (pp. 427–431). Association for Computational Linguistics.
- Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *Proceedings of the International Conference on Learning Representations (ICLR 2013)*.
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G., & Dean, J. (2013). Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems*, 26.
- Phatthiyaphaibun, W., Chaovavanich, K., Polpanumas, C., Suriyawongkul, A., Lowphansirikul, L., Chormai, P., Limkonchotiwat, P., Suntornpit, T., & Udomcharoenchaikit, C. (2023). PyThaiNLP: Thai natural language processing in Python. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023)*(pp. 25–36). Association for Computational Linguistics.
- PyThaiNLP. (n.d.). Thai2Vec embeddings examples. PyThaiNLP Tutorials. สืบค้นจาก https://pythainlp.org/tutorials/notebooks/word2vec_examples.html
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 3982-3992. <https://arxiv.org/abs/1908.10084>.
- Wang, Z., Höpken, W., & Jannach, D. (2025). A survey on point-of-interest recommendations leveraging heterogeneous data. *Information Technology & Tourism*, 27, 29–73. <https://doi.org/10.1007/s40558-024-00301-3>.

ภาคผนวก

รายละเอียด Code การทำระบบแนะนำสถานที่ท่องเที่ยวภายในประเทศไทยโดยใช้การกรองแบบอิงเนื้อหา (Content-Based Filtering)

```

from __future__ import annotations

import argparse
import contextlib
import html
import re
import os
import sys
import time
import unicodedata

from dataclasses import dataclass, field
from pathlib import Path
from typing import Callable, Iterable

import numpy as np
import pandas as pd

from bs4 import BeautifulSoup

# TF-IDF
from sklearn.feature_extraction.text import TfidfVectorizer

# Word2Vec / Thai2Vec
from gensim.models import Word2Vec
from gensim.models.keyedvectors import KeyedVectors

# FastText
from gensim.models import FastText
from gensim.models.keyedvectors import FastTextKeyedVectors

# Cosine Similarity
from sklearn.metrics.pairwise import cosine_similarity

# UTF-8 OUTPUT
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")

if hasattr(sys.stderr, "reconfigure"):
    sys.stderr.reconfigure(encoding="utf-8")

# CONSTANTS
COLUMNS = [
    "place_id",
    "name",
    "category",
    "description_th",
    "description_en",
    "location",
    # "rating",
    # "review",
]

COLUMN_MAP = {
    "ATT_ID": "place_id",

```

```

    "ATT_NAME_TH": "name",
    "ATT_CATEGORY_LABEL": "category",
    "ATT_DETAIL_TH": "description_th",
    "ATT_DETAIL_EN": "description_en",
    "PROVINCE_NAME_TH": "location",
    # "rating",
    # "review",
}

DEFAULT_CSV_PATH = Path(__file__).with_name(
    "attraction.csv"
)

DEFAULT_SENTENCE_BERT_MODEL = (
    "sentence-transformers/"
    "paraphrase-multilingual-MiniLM-L12-v2"
)

INSTALL_MESSAGE = (
    "Sentence-BERT requires the sentence-transformers package. "
    "Install it with: pip install sentence-transformers"
)

DOMAIN_STOPWORDS = {
    "ที่",
    "มี",
    "และ",
    "หรือ",
    "ใน",
    "ของ",
    "เป็น",
    "กับ",
    "ได้",
    "ให้",
    "จาก",
    "สำหรับ",
    "สามารถ",
    "นักท่องเที่ยว",
    "สถานที่",
    "เหมาะ",
    "เหมาะสำหรับ",
    "การ",
    "ชม",
    "เที่ยว",
}

# CLEAN HTML
def clean_html(text: object) -> str:
    if pd.isna(text):
        return ""
    soup = BeautifulSoup(str(text), "html.parser")
    for element in soup(["script", "style"]):
        element.decompose()
    return soup.get_text(separator=" ", strip=True)

# DATA CLEANING
def clean_dataset(df: pd.DataFrame) -> pd.DataFrame:

```

```

print(f"Original dataset shape: {df.shape}")
# Drop rows with no ATT_DETAIL_TH
df = df.dropna(subset=["ATT_DETAIL_TH"], how="all")
print(f"After dropping NaN ATT_DETAIL_TH: {df.shape}")

# Remove placeholder "-" values
df = df[df["ATT_DETAIL_TH"].astype(str).str.strip() != "-"]
df = df[df["ATT_DETAIL_TH"].astype(str).str.strip() != "*"]
print(f"After removing '-' and '*' ATT_DETAIL_TH: {df.shape}")

# Strip HTML markup
df["ATT_DETAIL_TH"] = df["ATT_DETAIL_TH"].apply(clean_html)
df["ATT_DETAIL_EN"] = df["ATT_DETAIL_EN"].apply(clean_html)

# Remove rows with empty ATT_DETAIL_TH after HTML cleaning
df = df[df["ATT_DETAIL_TH"].astype(str).str.strip() != ""]
print(f"After removing empty ATT_DETAIL_TH: {df.shape}")

# Remove duplicates
before_dup = len(df)
df = df.drop_duplicates(subset=["ATT_NAME_TH", "ATT_DETAIL_TH"], keep="first")
after_dup = len(df)

print(f"Removed duplicates: {before_dup - after_dup}")
print(f"After duplicate removal: {df.shape}")
return df

# SUPPRESS GENSIM STDERR
@contextlib.contextmanager
def suppress_native_stderr():
    """
    Silence low-level C/Cython stderr writes
    during Word2Vec/FastText training.
    """
    stderr_fd = sys.stderr.fileno()
    saved_fd = os.dup(stderr_fd)
    devnull_fd = os.open(os.devnull, os.O_WRONLY)

    try:
        sys.stderr.flush()
        os.dup2(devnull_fd, stderr_fd)
        yield
    finally:
        sys.stderr.flush()
        os.dup2(saved_fd, stderr_fd)
        os.close(devnull_fd)
        os.close(saved_fd)

# TEXT PREPROCESSING
def clean_thai_text(text: object) -> str:
    if pd.isna(text):
        return ""
    value = html.unescape(str(text))
    value = unicodedata.normalize("NFC", value).lower()

    value = re.sub(r"https?://\S+|www\.\S+", " ", value)
    value = re.sub(r"<[>]+>", " ", value)

```

```

value = re.sub(r"[0-9]+", " ", value)
value = re.sub(r"^\u0E00-\u0E7Fa-zA-Z\s]", " ", value)
value = re.sub(r"\s+", " ", value).strip()
return value

# TOKENIZER
def get_thai_word_tokenizer(stopwords: Iterable[str]):
    try:
        from pythainlp import word_tokenize
        from pythainlp.corpus.common import thai_stopwords
        stopword_set = (set(thai_stopwords()).union(stopwords))

        def tokenize(text: str) -> list[str]:
            cleaned = clean_thai_text(text)
            tokens = word_tokenize(cleaned, engine="newmm", keep_whitespace=False)
            return [token.strip() for token in tokens if (len(token.strip()) > 1 and
token.strip() not in stopword_set)]
            return tokenize

    except ImportError:
        stopword_set = set(stopwords)
        def tokenize_without_pythainlp(text: str) -> list[str]:
            cleaned = clean_thai_text(text)

            return [token for token in cleaned.split() if (len(token) > 1 and token not in
stopword_set)]
        return tokenize_without_pythainlp

def get_char_ngram_tokenizer(min_n: int = 3, max_n: int = 5) -> Callable[[str], list[str]]:
    def tokenize(text: str) -> list[str]:
        cleaned = clean_thai_text(text)
        ngrams: list[str] = []
        for word in cleaned.split():
            padded = f" {word} "
            for n in range(min_n, max_n + 1):
                ngrams.extend(padded[i:i + n] for i in range(max(0, len(padded) - n + 1)))
        return ngrams
    return tokenize

# BASE RECOMMENDER
@dataclass
class BaseRecommender:
    df: pd.DataFrame
    analyzer: str = "word"
    text_columns: list[str] = field(default_factory=lambda: ["category", "description_th"])

    column_weights: dict[str, int] = field(default_factory=lambda: {"category": 3,
"description_th": 1})
    stopwords: set[str] = field(default_factory=lambda: set(DOMAIN_STOPWORDS))
    fitted_df: pd.DataFrame | None = None

    # COMMON FEATURE TEXT
    def _build_feature_text(self, row: pd.Series) -> str:
        parts: list[str] = []
        for column in self.text_columns:
            value = clean_thai_text(row.get(column, ""))
            weight = max(1, int(self.column_weights.get(column, 1)))
            parts.extend([value] * weight)
        return " ".join(parts)

```

```

# COMMON PLACE ID
def recommend_by_place_id(self, place_id: int, top_n: int = 5) -> pd.DataFrame:
    self._ensure_fitted()
    matches = (self.fitted_df.index[ self.fitted_df["place_id"] == place_id].tolist())

    # if not found place id
    if not matches:
        raise ValueError(f"place_id not found: {place_id}")

    return self._similar_items(matches[0], top_n=top_n)

# COMMON NAME
def recommend_by_name(self, name: str, top_n: int = 5) -> pd.DataFrame:
    self._ensure_fitted()
    exact = (self.fitted_df.index[self.fitted_df["name"]== name].tolist())

    if exact:
        return self._similar_items(exact[0],top_n=top_n)

    # in case name not match
    return self._recommend_by_unknown_name(name, top_n)

# COMMON TEXT
def recommend_by_text(self, query: str, top_n: int = 5) -> pd.DataFrame:
    self._ensure_fitted()
    return self._recommend_by_text(query, top_n)

# ABSTRACT METHODS
def _recommend_by_unknown_name(self, name: str, top_n: int) -> pd.DataFrame:
    raise NotImplementedError

def _recommend_by_text(self, query: str, top_n: int) -> pd.DataFrame:
    raise NotImplementedError

def _similar_items(self, row_index: int,top_n: int) -> pd.DataFrame:
    raise NotImplementedError

def _ensure_fitted(self) -> None:
    raise NotImplementedError

# All Model Implement BaseRecommender Class!
# TF-IDF (Term Frequency - Inverse Document Frequency)
@dataclass
class TourismTFIDFRecommender(BaseRecommender):
    analyzer: str = "char"
    vectorizer: (TfidfVectorizer | None) = None
    tfidf_matrix: object | None = None

    # FIT
    def fit(self) -> "TourismTFIDFRecommender":
        self.fitted_df = (self.df.copy().reset_index(drop=True))

        corpus = (self.fitted_df.apply(self._build_feature_text, axis=1).tolist())

        if self.analyzer == "word":
            self.vectorizer = (
                TfidfVectorizer(
                    tokenizer=

```

```

        get_thai_word_tokenizer(
            self.stopwords
        ),
        token_pattern=None,
        ngram_range=(1, 2),
        min_df=1,
        max_df=0.9,
        sublinear_tf=True,
        norm="l2",
    )
)

elif self.analyzer == "char":
    self.vectorizer = (
        TfidfVectorizer(
            preprocessor=clean_thai_text,
            analyzer="char_wb",
            ngram_range=(3, 5),
            min_df=1,
            max_df=0.95,
            sublinear_tf=True,
            norm="l2",
        )
    )

else:
    raise ValueError("analyzer must be 'char' or 'word'")

self.tfidf_matrix = (self.vectorizer.fit_transform(corpus))
return self

# UNKNOWN NAME
def _recommend_by_unknown_name(self, name: str, top_n: int) -> pd.DataFrame:
    name_vectors =
    (self.vectorizer.transform(self.fitted_df["name"].map(clean_thai_text)))

    query_vector = (self.vectorizer.transform([clean_thai_text(name)]))
    best_index = int(np.argmax(cosine_similarity(query_vector, name_vectors)[0]))

    return self._similar_items(best_index, top_n=top_n)

# TEXT QUERY
def _recommend_by_text(self, query: str, top_n: int) -> pd.DataFrame:
    query_vector = (self.vectorizer.transform([clean_thai_text(query)]))

    scores = cosine_similarity(query_vector, self.tfidf_matrix)[0]
    ranked_indices = (scores.argsort()[::-1][:top_n])
    return self._format_results(ranked_indices, scores)

# SIMILAR ITEMS
def _similar_items(self, row_index: int, top_n: int) -> pd.DataFrame:
    scores = cosine_similarity(self.tfidf_matrix[row_index], self.tfidf_matrix)[0]
    ranked_indices = [idx for idx in scores.argsort()[::-1] if idx != row_index][:top_n]

    return self._format_results(ranked_indices, scores, source_index=row_index)

# FORMAT
def _format_results(
    self,

```

```

        ranked_indices: Iterable[int],
        scores: np.ndarray,
        source_index: int | None = None
    ) -> pd.DataFrame:
        ranked_indices = list(ranked_indices)
        result = (self.fitted_df.iloc[ranked_indices].copy())
        result.insert(0, "similarity_score", [float(scores[idx]) for idx in ranked_indices])

        if source_index is not None:
            source_name = (self.fitted_df.loc[source_index, "name"])
            result.insert(0, "source_name", source_name)

        return result[
            [
                *(
                    ["source_name"] if source_index is not None else [],
                    "similarity_score",
                    "place_id",
                    "name",
                    "category",
                    "location",
                    "description_th",
                )
            ]
        ]

# CHECK FIT
def _ensure_fitted(self) -> None:
    if (self.vectorizer is None or self.tfidf_matrix is None or self.fitted_df is None):
        raise RuntimeError("Call fit() before requesting recommendations.")

# THAI2VEC
@dataclass
class TourismThai2VecRecommender(BaseRecommender):
    analyzer: str = "word"
    thai2vec_model_name: str = ("thai2fit_wv")
    vector_size: int = 100
    window: int = 5
    min_count: int = 1
    epochs: int = 300
    seed: int = 42
    embedding_model: (KeyedVectors | None) = None
    item_matrix: (np.ndarray | None) = None
    tokenizer: (Callable[[str], list[str]] | None) = None
    embedding_source: (str | None) = None

# FIT
def fit(self) -> "TourismThai2VecRecommender":
    self.fitted_df = (self.df.copy().reset_index(drop=True))
    corpus = (self.fitted_df.apply(self._build_feature_text, axis=1).tolist())

    if self.analyzer == "word":
        self.tokenizer = (get_thai_word_tokenizer(self.stopwords))
        self.embedding_model = (self._load_pretrained_thai2vec())
    elif self.analyzer == "char":
        self.tokenizer = (get_char_ngram_tokenizer())
        self.embedding_model = None
    else:
        raise ValueError("analyzer must be 'char' or 'word'")

```

```

        tokenized_corpus = [self.tokenizer(text) for text in corpus]
        tokenized_names = [self.tokenizer(name) for name in
(self.fitted_df["name"].map(clean_thai_text))]
        training_sentences = [tokens for tokens in [*tokenized_corpus, *tokenized_names] if
tokens]

        if not training_sentences:
            raise ValueError(
                "No usable tokens were found "
                "for Word2Vec training."
            )

        if self.embedding_model is None:
            self.embedding_model = (self._train_local_word2vec(training_sentences))

        self.item_matrix = np.vstack([self._tokens_to_vector(tokens) for tokens in
tokenized_corpus])
        return self

    # UNKNOWN NAME
    def _recommend_by_unknown_name(self, name: str, top_n: int) -> pd.DataFrame:
        name_vectors = np.vstack([self._text_to_vector(value) for value in
(self.fitted_df["name"])]])
        query_vector = (self._text_to_vector(name).reshape(1, -1))

        best_index = int(np.argmax(cosine_similarity(query_vector, name_vectors)[0]))

        return self._similar_items(best_index, top_n=top_n)

    # TEXT QUERY
    def _recommend_by_text(self, query: str, top_n: int) -> pd.DataFrame:
        query_vector = (self._text_to_vector(query).reshape(1, -1))
        scores = cosine_similarity(query_vector, self.item_matrix)[0]

        ranked_indices = (scores.argsort()[::-1][:top_n])
        return self._format_results(ranked_indices, scores)

    # SIMILAR ITEMS
    def _similar_items(self, row_index: int, top_n: int) -> pd.DataFrame:
        source_vector = (self.item_matrix[row_index].reshape(1, -1))

        scores = cosine_similarity(source_vector, self.item_matrix)[0]
        ranked_indices = [ idx for idx in scores.argsort()[::-1] if idx != row_index
][:top_n]

        return self._format_results(ranked_indices, scores, source_index=row_index)

    # LOAD PRETRAINED THAI2VEC
    def _load_pretrained_thai2vec(self) -> KeyedVectors | None:
        try:
            from pythainlp.word_vector import WordVector
            self.embedding_source = (self.thai2vec_model_name)
            return (WordVector(self.thai2vec_model_name).get_model())

        except Exception:
            self.embedding_source = ("local_word2vec")
            return None

    # TRAIN LOCAL WORD2VEC

```

```

def _train_local_word2vec(self, sentences: list[list[str]]) -> KeyedVectors:
    model = Word2Vec(
        sentences=sentences,
        vector_size=self.vector_size,
        window=self.window,
        min_count=self.min_count,
        workers=os.cpu_count() or 4,
        sg=1,
        epochs=self.epochs,
        seed=self.seed,
    )
    self.embedding_source = ("local_word2vec")

    return model.wv

# TEXT -> VECTOR
def _text_to_vector(self, text: object) -> np.ndarray:
    return self._tokens_to_vector(self.tokenizer(str(text)))
# TOKENS -> VECTOR
def _tokens_to_vector(self, tokens: Iterable[str]) -> np.ndarray:
    vectors = [self.embedding_model[token] for token in tokens if token in
self.embedding_model]

    if not vectors:
        return np.zeros(self.embedding_model.vector_size, dtype=np.float32)

    return np.mean(vectors, axis=0)

# FORMAT
def _format_results(
    self,
    ranked_indices: Iterable[int],
    scores: np.ndarray,
    source_index: int | None = None
) -> pd.DataFrame:
    ranked_indices = list(ranked_indices)
    result = (self.fitted_df.iloc[ranked_indices].copy())
    result.insert(0, "similarity_score", [float(scores[idx]) for idx in ranked_indices])

    if source_index is not None:
        source_name = (self.fitted_df.loc[source_index, "name"])
        result.insert(0, "source_name", source_name)

    return result[
        [
            *(
                ["source_name"] if source_index is not None else [],
                "similarity_score",
                "place_id",
                "name",
                "category",
                "location",
                "description_th",
            )
        ]
    ]

# CHECK FIT
def _ensure_fitted(self) -> None:
    if (self.embedding_model is None or self.item_matrix is None or self.fitted_df is

```

```

None or self.tokenizer is None):
    raise RuntimeError("Call fit() before requesting recommendations.")

# FASTTEXT
@dataclass
class TourismFastTextRecommender(BaseRecommender):
    analyzer: str = "word"
    vector_size: int = 100
    window: int = 5
    min_count: int = 1
    epochs: int = 300
    min_n: int = 3
    max_n: int = 6
    seed: int = 42
    embedding_model: (FastTextKeyedVectors | None) = None
    item_matrix: (np.ndarray | None) = None
    tokenizer: (Callable[[str], list[str]] | None) = None

    # FIT
    def fit(self) -> "TourismFastTextRecommender":
        self.fitted_df = (self.df.copy().reset_index(drop=True))
        corpus = (self.fitted_df.apply(self._build_feature_text, axis=1).tolist())

        if self.analyzer == "word":
            self.tokenizer = (get_thai_word_tokenizer(self.stopwords))
        elif self.analyzer == "char":
            self.tokenizer = (get_char_ngram_tokenizer())
        else:
            raise ValueError("analyzer must be 'char' or 'word'")

        tokenized_corpus = [self.tokenizer(text) for text in corpus]
        tokenized_names = [self.tokenizer(name) for name in
(self.fitted_df["name"].map(clean_thai_text))]
        training_sentences = [ tokens for tokens in [ *tokenized_corpus, *tokenized_names ]
if tokens]

        if not training_sentences:
            raise ValueError(
                "No usable tokens were found "
                "for FastText training."
            )

        self.embedding_model = (self._train_fasttext(training_sentences))
        self.item_matrix = np.vstack([self._tokens_to_vector(tokens) for tokens in
tokenized_corpus])
        return self

    # UNKNOWN NAME
    def _recommend_by_unknown_name(self, name: str, top_n: int) -> pd.DataFrame:
        name_vectors = np.vstack([self._text_to_vector(value) for value in
(self.fitted_df["name"])]])

        query_vector = (self._text_to_vector(name).reshape(1, -1))
        best_index = int(np.argmax(cosine_similarity(query_vector, name_vectors)[0]))

        return self._similar_items(best_index, top_n=top_n)

    # TEXT QUERY
    def _recommend_by_text(self, query: str, top_n: int) -> pd.DataFrame:

```

```

        query_vector = (self._text_to_vector(query).reshape(1, -1))

        scores = cosine_similarity(query_vector, self.item_matrix)[0]
        ranked_indices = (scores.argsort()[::-1][:top_n])

        return self._format_results(ranked_indices, scores)

# SIMILAR ITEMS
def _similar_items(self, row_index: int, top_n: int) -> pd.DataFrame:
    source_vector = (self.item_matrix[row_index].reshape(1, -1))
    scores = cosine_similarity(source_vector, self.item_matrix)[0]
    ranked_indices = [idx for idx in scores.argsort()[::-1] if idx != row_index][:top_n]

    return self._format_results(ranked_indices, scores, source_index=row_index)

# TRAIN FASTTEXT
def _train_fasttext(self, sentences: list[list[str]]) -> FastTextKeyedVectors:
    model = FastText(
        sentences=sentences,
        vector_size=self.vector_size,
        window=self.window,
        min_count=self.min_count,
        workers=os.cpu_count() or 4,
        sg=1,
        epochs=self.epochs,
        min_n=self.min_n,
        max_n=self.max_n,
        seed=self.seed,
    )
    return model.wv

# TEXT -> VECTOR
def _text_to_vector(self, text: object) -> np.ndarray:
    return self._tokens_to_vector(self.tokenizer(str(text)))

# TOKENS -> VECTOR
def _tokens_to_vector(self, tokens: Iterable[str]) -> np.ndarray:
    vectors = [self.embedding_model[token] for token in tokens if token]

    if not vectors:
        return np.zeros(self.embedding_model.vector_size, dtype=np.float32)

    return np.mean(vectors, axis=0)

# FORMAT
def _format_results(
    self,
    ranked_indices: Iterable[int],
    scores: np.ndarray,
    source_index: int | None = None
) -> pd.DataFrame:
    ranked_indices = list(ranked_indices)
    result = (self.fitted_df.iloc[ranked_indices].copy())
    result.insert(0, "similarity_score", [float(scores[idx]) for idx in ranked_indices])

    if source_index is not None:
        source_name = (self.fitted_df.loc[source_index, "name"])
        result.insert(0, "source_name", source_name)

```

```

        return result[
            [
                *(
                    ["source_name" if source_index is not None else []],
                    "similarity_score",
                    "place_id",
                    "name",
                    "category",
                    "location",
                    "description_th",
                )
            ]
        ]

# CHECK FIT
def _ensure_fitted(self) -> None:
    if (self.embedding_model is None or self.item_matrix is None or self.fitted_df is
        None or self.tokenizer is None):
        raise RuntimeError("Call fit() before requesting recommendations.")

# SENTENCE-BERT
@dataclass
class TourismSentenceBERTRecommender(BaseRecommender):
    analyzer: str = "sentence"
    text_columns: list[str] = field(default_factory=lambda: ["category", "description_th"])

    model_name: str = (DEFAULT_SENTENCE_BERT_MODEL)

    batch_size: int = 32
    sentence_model: (object | None) = None
    sentence_matrix: (np.ndarray | None) = None

# FIT
def fit(self) -> "TourismSentenceBERTRecommender":
    if self.analyzer not in {"sentence", "word", "char"}:
        raise ValueError(
            "analyzer must be "
            "'sentence', 'word', or 'char'"
        )

    self.fitted_df = (self.df.copy().reset_index(drop=True))
    corpus = (self.fitted_df.apply(self._build_feature_text, axis=1).tolist())

    self.sentence_model = (self._load_sentence_bert_model())
    self.sentence_matrix = (self._encode_texts(corpus))
    return self

# BUILD FEATURE TEXT
def _build_feature_text(self, row: pd.Series) -> str:
    parts: list[str] = []
    for column in self.text_columns:
        value = clean_thai_text(row.get(column, ""))
        if value:
            parts.append(value)

    return " ".join(parts)

# UNKNOWN NAME
def _recommend_by_unknown_name(self, name: str, top_n: int) -> pd.DataFrame:

```

```

        name_vectors =
(self._encode_texts(self.fitted_df["name"].map(clean_thai_text).tolist()))
        query_vector = (self._encode_texts([clean_thai_text(name)]))

        best_index = int(np.argmax(cosine_similarity(query_vector, name_vectors)[0]))
        return self._similar_items(best_index, top_n=top_n)

# TEXT QUERY
def _recommend_by_text(self, query: str, top_n: int) -> pd.DataFrame:
    query_vector = (self._encode_texts([clean_thai_text(query)]))

    scores = cosine_similarity(query_vector, self.sentence_matrix)[0]
    ranked_indices = (scores.argsort()[::-1][:top_n])

    return self._format_results(ranked_indices, scores)

# SIMILAR ITEMS
def _similar_items(self, row_index: int, top_n: int) -> pd.DataFrame:
    source_vector = (self.sentence_matrix[row_index].reshape(1, -1))
    scores = cosine_similarity(source_vector, self.sentence_matrix)[0]
    ranked_indices = [idx for idx in scores.argsort()[::-1] if idx != row_index] [:top_n]

    return self._format_results(ranked_indices, scores, source_index=row_index)

# LOAD MODEL
def _load_sentence_bert_model(self) -> object:
    try:
        from sentence_transformers import (SentenceTransformer)

    except ImportError as exc:
        raise ImportError(INSTALL_MESSAGE) from exc

    try:
        return SentenceTransformer(self.model_name)

    except Exception as exc:
        raise RuntimeError(
            f"Could not load Sentence-BERT "
            f"model {self.model_name!r}. "
            "Check your internet connection, "
            "Hugging Face cache, or pass a "
            "local model path with --model-name."
        ) from exc

# ENCODE
def _encode_texts(self, texts: list[str]) -> np.ndarray:
    embeddings = (
        self.sentence_model.encode(
            texts,
            batch_size=self.batch_size,
            convert_to_numpy=True,
            normalize_embeddings=True,
            show_progress_bar=False,
        )
    )

    return np.asarray(embeddings, dtype=np.float32)

# FORMAT

```

```

def _format_results(
    self,
    ranked_indices: Iterable[int],
    scores: np.ndarray,
    source_index: int | None = None
) -> pd.DataFrame:

    ranked_indices = list(ranked_indices)
    result = (self.fitted_df.iloc[ranked_indices].copy())
    result.insert(0, "similarity_score", [float(scores[idx]) for idx in ranked_indices])

    if source_index is not None:
        source_name = (self.fitted_df.loc[source_index, "name"])
        result.insert(0, "source_name", source_name)

    return result[
        *(
            [
                "source_name" if source_index is not None else [],
                "similarity_score",
                "place_id",
                "name",
                "category",
                "location",
                "description_th",
            ]
        )
    ]

# CHECK FIT
def _ensure_fitted(self) -> None:
    if (self.sentence_model is None or self.sentence_matrix is None or self.fitted_df is
None):
        raise RuntimeError("Call fit() before requesting recommendations.")

# LOAD DATA
def load_dataframe(csv_path: str | None = None) -> pd.DataFrame:
    path = (Path(csv_path) if csv_path else DEFAULT_CSV_PATH)

    if not path.exists():
        raise FileNotFoundError(f"CSV file not found: {path}")

    print(f"Loading dataset -> {path}")

    df = pd.read_csv(path, low_memory=False)

    # CLEAN RAW HTML DATA (before renaming columns)
    df = clean_dataset(df)
    df = df.rename(columns=COLUMN_MAP)

    missing = (set(COLUMNS) - set(df.columns))
    if missing:
        raise ValueError(f"CSV is missing columns: {sorted(missing)}")

    df = df[COLUMNS].copy()
    df = df.reset_index(drop=True)
    print(f"Final dataset shape: {df.shape}")
    return df

# PRINT RESULTS

```

```

def print_results(title: str, results: pd.DataFrame) -> None:
    print(f"\n{title}")
    print("=" * len(title))

    view = results.copy()
    view["description_th"] = (view["description_th"].str.slice(0, 80)+ "...")
    print(view.to_string(index=False))

# BUILD RECOMMENDER
def build_recommender(model: str, df: pd.DataFrame, analyzer: str):
    """
    Factory:
    build + fit a recommender by name.

    Returns:
        recommender,
        model_name,
        fit_seconds,
        error
    """

    started = time.perf_counter()
    try:
        with suppress_native_stderr():
            if model == "thai2vec":
                recommender = (TourismThai2VecRecommender(df=df, analyzer=analyzer).fit())
                model_name = "Thai2Vec"

            elif model == "fasttext":
                recommender = (TourismFastTextRecommender(df=df, analyzer=analyzer).fit())
                model_name = "Fasttext"

            elif model == "berta":
                recommender = (TourismSentenceBERTRecommender(df=df,
analyzer=analyzer).fit())
                model_name = "Sentence-BERT"

            else:
                recommender = (TourismTFIDFRecommender(df=df, analyzer="char").fit())
                model_name = "TF-IDF"

            elapsed = (time.perf_counter() - started)
            return (recommender, model_name, elapsed, None)

    except Exception as exc:
        elapsed = (time.perf_counter() - started)

        return (None, model, elapsed, exc)

# SIDE-BY-SIDE COMPARISON
def build_side_by_side_comparison(
    results_by_model: dict[str, pd.DataFrame],
    top_n: int
) -> pd.DataFrame:

    order = [
        "TF-IDF",
        "Thai2Vec",
        "Fasttext",
    ]

```

```

        "Sentence-BERT"
    ]

    labels = {
        "TF-IDF": "TF-IDF",
        "Fasttext": "FastText",
        "Thai2Vec": "Thai2Vec",
        "Sentence-BERT": "BERT"
    }

    data: dict[str, pd.Series] = {}
    for key in order:
        label = labels[key]
        model_df = (results_by_model.get(key))

        if model_df is not None:
            names = (model_df["name"].reset_index(drop=True))
            scores = (model_df["similarity_score"].reset_index(drop=True))

            else:
                names = pd.Series(dtype=object)
                scores = pd.Series(dtype=float)

            data[label] = (names.reindex(range(top_n)))
            data[f"{label} Score"] = (scores.reindex(range(top_n)))

    return pd.DataFrame(data)

# MODEL RANKING
def build_model_ranking(summary_df: pd.DataFrame) -> pd.DataFrame:
    """
    Rank models by top1_score
    from highest to lowest.
    """

    ranking = (summary_df.copy())

    ranking = ranking[ranking["top1_score"].notna()].copy()
    ranking = (ranking.sort_values(by="top1_score", ascending=False).reset_index(drop=True))

    ranking.insert(0, "Score Rank", range(1, len(ranking) + 1))
    ranking = ranking[
        [
            "Score Rank",
            "model",
            "top1_name",
            "top1_score",
            "fit_seconds",
            "query_seconds",
            "status",
        ]
    ]
    return ranking

# EXCEL NUMBER FORMAT
def apply_4f_number_format(worksheet, df: pd.DataFrame) -> None:
    float_cols = [i for i, dtype in enumerate(df.dtypes) if
pd.api.types.is_float_dtype(dtype)]

```

```

    for col_idx in float_cols:
        excel_col = (col_idx + 1)
        for row in range(2, worksheet.max_row + 1):
            worksheet.cell(row=row, column=excel_col).number_format = ("0.0000")

# SAVE RESULTS
def save_results(
    results: pd.DataFrame,
    csv_path: str | None,
    excel_path: str | None
) -> None:
    """
    Save a single model's results
    to CSV and/or Excel.
    """
    if csv_path:
        results.to_csv(csv_path, index=False, encoding="utf-8-sig")
        print(f"Saved CSV -> {csv_path}")

    if excel_path:
        Path(excel_path).parent.mkdir(parents=True, exist_ok=True)

        with pd.ExcelWriter(excel_path, engine="openpyxl") as writer:
            results.to_excel(writer, index=False, sheet_name="results")

            apply_4f_number_format(writer.sheets["results"], results)
            print(f"Saved Excel -> {excel_path}")

# SAVE ALL RESULTS To Excel or Csv files
def save_all_results(
    results_by_model: dict[str, pd.DataFrame],
    summary_df: pd.DataFrame,
    csv_path: str | None,
    excel_path: str | None,
    comparison_df: pd.DataFrame | None = None,
    ranking_df: pd.DataFrame | None = None,
) -> None:
    """
    Save every model's results.

    CSV:
        combined result
        comparison CSV

    Excel:
        summary
        model_ranking
        comparison
        individual model sheets
    """
    if csv_path:
        combined = pd.concat([df.assign(model=name) for name, df in
results_by_model.items()], ignore_index=True)

        cols = (["model"] + [c for c in combined.columns if c != "model"])
        Path(csv_path).parent.mkdir(parents=True, exist_ok=True)

        combined[cols].to_csv(csv_path, index=False, encoding="utf-8-sig")
        print(f"Saved CSV -> {csv_path}")

```

```

        if comparison_df is not None:
            csv_path_obj = Path(csv_path)
            comparison_path =
(csv_path_obj.with_name(f"{csv_path_obj.stem}_comparison{csv_path_obj.suffix}"))
            comparison_df.to_csv(comparison_path, index=False, encoding="utf-8-sig")
            print(f"Saved CSV -> {comparison_path}")

    if excel_path:
        Path(excel_path).parent.mkdir(parents=True, exist_ok=True)

        with pd.ExcelWriter(excel_path, engine="openpyxl") as writer:
            # SUMMARY
            summary_df.to_excel(writer, index=False, sheet_name="summary")

            # MODEL RANKING
            if ranking_df is not None:
                ranking_df.to_excel(writer, index=False, sheet_name="model_ranking")
                apply_4f_number_format(writer.sheets["model_ranking"], ranking_df)

            # COMPARISON
            if comparison_df is not None:
                comparison_df.to_excel(writer, index=False, sheet_name="comparison")
                apply_4f_number_format(writer.sheets["comparison"], comparison_df)

            # MODEL RESULTS
            for name, df in (results_by_model.items()):
                sheet_name = re.sub(r"[\\/*?:\\[\\]]", "_", name)[:31]

                df.to_excel(writer, index=False, sheet_name=sheet_name)

                apply_4f_number_format(writer.sheets[sheet_name], df)
            print(f"Saved Excel -> {excel_path}")

# GET PLACE NAME
def get_place_name(df: pd.DataFrame, place_id: int) -> str | None:
    matches = (df.index[df["place_id"]== place_id].tolist())

    if not matches:
        return None
    return df.loc[matches[0], "name"]

# RUN ALL MODELS
def run_all_models(df: pd.DataFrame, args: argparse.Namespace) -> None:
    model_ids = ["tfidf", "thai2vec", "fasttext", "berta"]

    sample_name = (args.name or "อุทยานแห่งชาติดอยอินทนนท์")
    sample_text = (args.text or "ทะเล น้ำใส ดำน้ำ พักผ่อน")

    summary_rows: list[dict] = []
    results_by_model: dict[str, pd.DataFrame] = {}

    for model_id in model_ids:
        (recommender, model_name, fit_seconds, fit_error) = build_recommender(model_id, df,
args.analyzer)

        if fit_error is not None:
            print(
                f"\n[{model_name}] "

```

```

        f"fit failed after "
        f"{fit_seconds:.4f}s: "
        f"{fit_error}"
    )

    summary_rows.append({
        "model": model_name,
        "fit_seconds": round(fit_seconds, 4),
        "query_seconds": None,
        "top1_name": None,
        "top1_score": None,
        "status": (
            f"error: {fit_error}"
        ),
    })
    continue

query_started = (time.perf_counter())

if args.place_id is not None:
    results = (recommender.recommend_by_place_id(args.place_id, top_n=args.top_n))

    place_name = (get_place_name(df, args.place_id))

    query_label = (
        f"{place_name} "
        f"(place_id={args.place_id})"
        if place_name
        else
        f"place_id={args.place_id}"
    )
    query_kind = "similar to"

elif args.name:
    results = (recommender.recommend_by_name(args.name, top_n=args.top_n))
    query_label = args.name
    query_kind = "similar to"

elif args.text:
    results = (recommender.recommend_by_text(args.text, top_n=args.top_n))
    query_label = args.text
    query_kind = "for query"

else:
    results = (recommender.recommend_by_name(sample_name, top_n=args.top_n))
    query_label = sample_name
    query_kind = "similar to"

query_seconds = (time.perf_counter() - query_started)

print_results(
    f"{model_name} "
    f"recommendations "
    f"{query_kind}: "
    f"{query_label}",
    results
)

print(

```

```

        f"[{model_name}] "
        f"fit: {fit_seconds:.3f}s | "
        f"query: {query_seconds:.4f}s"
    )

    results_by_model[model_name] = results
    top1 = results.iloc[0]
    summary_rows.append({
        "model": model_name,
        "fit_seconds": round(fit_seconds,4),
        "query_seconds": round(query_seconds,4),
        "top1_name": top1["name"],
        "top1_score": top1["similarity_score"],
        "status": "ok",
    })

# SIDE-BY-SIDE
comparison_df = (build_side_by_side_comparison(results_by_model, args.top_n))

print("\nSide-by-side comparison")
print("=" * len("Side-by-side comparison"))
print(comparison_df.to_string(index=False))

# SUMMARY
summary_df = pd.DataFrame(summary_rows)

print("\nModel comparison summary")
print("=" * len("Model comparison summary"))
print(summary_df.to_string(index=False))

# MODEL RANKING
ranking_df = (build_model_ranking(summary_df))

print("\nModel Ranking by Top-1 Score")
print("=" * 35)
print(ranking_df.to_string(index=False))

# SAVE
if (args.save_csv or args.save_excel):
    save_all_results(
        results_by_model,
        summary_df,
        args.save_csv,
        args.save_excel,
        comparison_df,
        ranking_df
    )

# ARGUMENT PARSER
def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        description=(
            "Content-based Thai tourism "
            "recommender using TF-IDF "
            "Thai2Vec, fastText or BERT."
        )
    )

```

```

parser.add_argument(
    "--csv",
    help=(
        "Optional CSV path. "
        f"If omitted, "
        f"{DEFAULT_CSV_PATH.name} "
        "is used."
    )
)

parser.add_argument(
    "--model",
    choices=[ "tfidf", "thai2vec", "fasttext", "berta", "all"],
    default="tfidf",
    help=(
        "Recommendation model: "
        "tfidf, thai2vec, fasttext, "
        "berta, or all "
        "(compare every model)."
    )
)

parser.add_argument(
    "--analyzer",
    choices=[
        "char",
        "word"
    ],
    default="word",
    help=(
        "TF-IDF: char or word. "
        "Thai2Vec: word is recommended."
    )
)

parser.add_argument(
    "--top-n",
    type=int,
    default=5,
    help=(
        "Number of recommendations "
        "to return."
    )
)

query = (
    parser
    .add_mutually_exclusive_group()
)

query.add_argument(
    "--place-id",
    type=int,
    help=(
        "Recommend places similar "
        "to this place_id."
    )
)

```

```

query.add_argument(
    "--name",
    help=(
        "Recommend places similar "
        "to this place name."
    )
)

query.add_argument(
    "--text",
    help=(
        "Recommend places from "
        "free-text Thai interest/query."
    )
)

parser.add_argument(
    "--save-csv",
    help=(
        "Save results to this CSV path. "
        "With --model all, all models "
        "are combined into one file "
        "with a 'model' column."
    )
)

parser.add_argument(
    "--save-excel",
    help=(
        "Save results to this .xlsx path. "
        "With --model all, each model "
        "gets its own sheet plus "
        "a 'summary' sheet."
    )
)

return parser

# MAIN
def main() -> None:
    args = (build_parser().parse_args())
    df = load_dataframe(args.csv)

    # ALL MODELS
    if args.model == "all":
        run_all_models(df, args)
        return

    # SINGLE MODEL
    (recommender, model_name, fit_seconds, fit_error) = build_recommender(args.model, df,
args.analyzer)

    if fit_error is not None:
        raise fit_error

    print(
        f"[{model_name}] "
        f"fit time: "
        f"{fit_seconds:.3f}s"
    )

```

```

# PLACE ID
if args.place_id is not None:
    results = (recommender.recommend_by_place_id(args.place_id, top_n=args.top_n))
    place_name = (get_place_name(df, args.place_id))

    title_label = (
        f"{place_name} "
        f"(place_id={args.place_id})"
        if place_name
        else
        f"place_id={args.place_id}"
    )

    print_results(
        f"{model_name} "
        f"recommendations "
        f"similar to "
        f"{title_label}",
        results
    )

    save_results(results, args.save_csv, args.save_excel)

# NAME
elif args.name:
    results = (recommender.recommend_by_name(args.name, top_n=args.top_n))

    print_results(
        f"{model_name} "
        f"recommendations "
        f"similar to "
        f"{args.name}",
        results
    )

    save_results(results, args.save_csv, args.save_excel)

# TEXT
elif args.text:
    results = (recommender.recommend_by_text(args.text, top_n=args.top_n))

    print_results(
        f"{model_name} "
        f"recommendations "
        f"for query: "
        f"{args.text}",
        results
    )

    save_results(results, args.save_csv, args.save_excel)

# DEFAULT EXAMPLES
else:
    examples = [
        (
            "อุทยานแห่งชาติดอยอินทนนท์",
            f"{model_name} recommendations similar to อุทยานแห่งชาติดอยอินทนนท์",
            recommender.recommend_by_name(

```

```

        "อุทยานแห่งชาติดอยอินทนนท์",
        top_n=args.top_n
    ),
    (
        "ทะเล น้ำใส ดำน้ำ พักผ่อน",
        f"{model_name} recommendations for: ทะเล น้ำใส ดำน้ำ พักผ่อน",
        recommender.recommend_by_text(
            "ทะเล น้ำใส ดำน้ำ พักผ่อน",
            top_n=args.top_n
        )
    ),
    (
        "วัดสองพี่น้อง",
        f"{model_name} recommendations similar to วัดสองพี่น้อง",
        recommender.recommend_by_name(
            "วัดสองพี่น้อง",
            top_n=args.top_n
        )
    ),
]

for _, title, results in examples:
    print_results( title, results )

if (args.save_csv or args.save_excel):
    combined = pd.concat( [ df.assign( query=query) for query, _, df in examples],
        ignore_index=True )
    cols = ( ["query"] + [ c for c in combined.columns if c != "query"])
    save_results(combined[cols], args.save_csv, args.save_excel)

# ENTRY POINT
if __name__ == "__main__":
    main()

```

ประวัติผู้เขียน

ชื่อ-สกุล

นางสาวกนกนาฏ มหาวณิช

สถานที่เกิด

กรุงเทพมหานคร

วุฒิการศึกษา

สถาปัตยกรรมภายในบัณฑิต สาขาวิชาสถาปัตยกรรมภายใน
คณะสถาปัตยกรรมศาสตร์และการผังเมือง มหาวิทยาลัยธรรมศาสตร์
ปีการศึกษา 2561

ปริญญาวิทยาศาสตรบัณฑิต สาขาวิทยาการคอมพิวเตอร์
คณะวิทยาศาสตร์ มหาวิทยาลัยรามคำแหง
ปีการศึกษา 2568

ประวัติผู้เขียน

ชื่อ-สกุล	นางสาวณัฐพรรณ แป้นทอง
สถานที่เกิด	อุทัยธานี
วุฒิการศึกษา	มัธยมศึกษาตอนปลาย (กศน.) ศูนย์การศึกษานอกระบบและการศึกษาตามอัธยาศัย อำเภอสว่างอารมณ์ จังหวัดอุทัยธานี ปีการศึกษา 2565
	ปริญญาวิทยาศาสตรบัณฑิต สาขาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์ มหาวิทยาลัยรามคำแหง ปีการศึกษา 2568